คณะเทคโนโลยีสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

Chapter 12: Image Segmentation Using Autoencoder Architectures

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

การประยุกต์ใช้งาน CNN สำหรับ Image

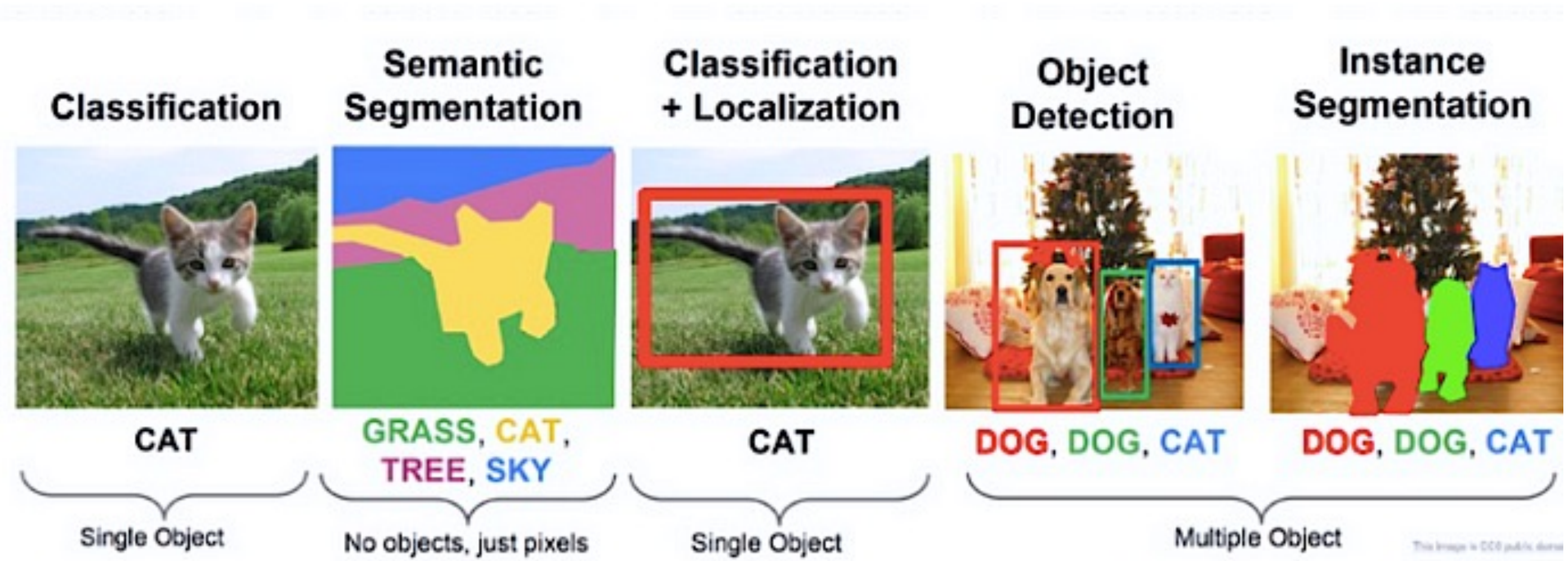


Figure 1: Different computer vision tasks. Source: Introduction to Artificial Intelligence and Computer Vision Revolution (https://www.slideshare.net/darian_f/introduction-to-the-artificial-intelligence-and-computer-vision-revolution).

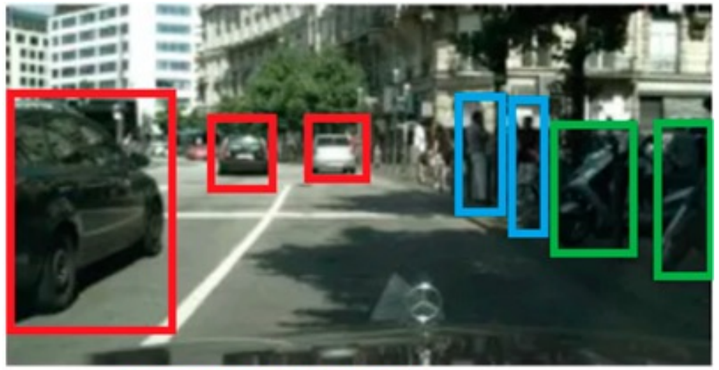
Image Classification



Car, Person, Motorcycle..

ตรวจจับวัตถุที่มีอยู่ในภาพโดยไม่พิจารณาตำแหน่งของวัตถุ

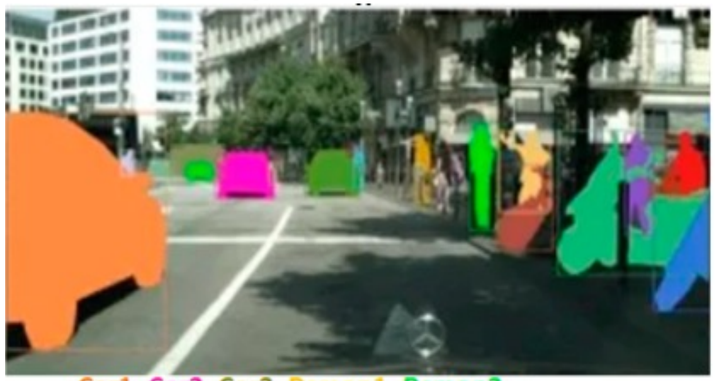
Object Detection



Car, Person, Motorcycle

ไม่เพียงแต่รับรู้ว่าวัตถุใดบ้างที่อยู่ในภาพ แต่ยังรู้ด้วยว่าวัตถุเหล่านั้นอยู่ที่ใดในภาพ สามารถจดจำวัตถุและระบุตำแหน่งด้วยกรอบสี่เหลี่ยมล้อมรอบ

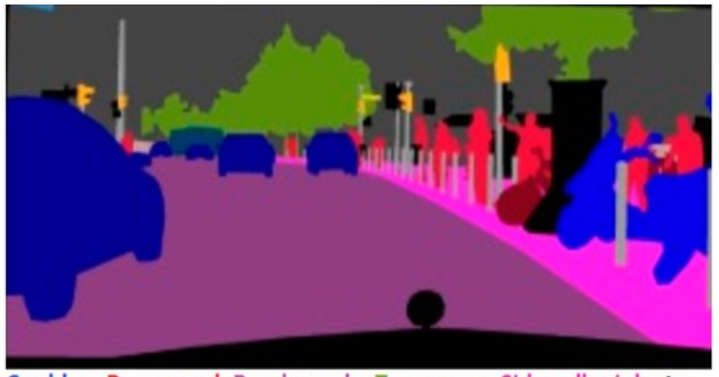
Instance Segmentation



Car1, Car2, Car3, Person1, Person2,..

พิกเซลของแต่ละวัตถุจะถูกระบุแยกกัน ในตัวอย่างนี้ พิกเซลของรถคันหนึ่งนำเสนอด้วยสีส้ม รถอีกคันนำเสนอด้วยสีชมพู และรถอีกคันนำเสนอด้วยสีเขียว แต่ background หรือ สิ่งที่ไม่ใช่วัตถุ จะไม่ได้ถูกทำ segmentation และไม่ได้บอก semantic ครอบคลุมทั้งภาพ (ยังเหลือ pixel ที่ไม่ถูก label)

Semantic Segmentation



Car:blue, Person:red, Road:purple, Tree:green, Sidewalk:pink etc.

การแบ่งส่วนเชิงความหมาย (semantic segmentation) จะจัดการกับวัตถุที่อยู่ในคลาสเดียวกันในภาพให้เป็นป้ายกำกับเดียวกัน ในขณะที่การแบ่งส่วนตามอินสแตนซ์ (instance segmentation) จะจัดการกับวัตถุที่อยู่ในคลาสเดียวกันเป็นอินสแตนซ์แยกกัน

Panoptic Segmentation

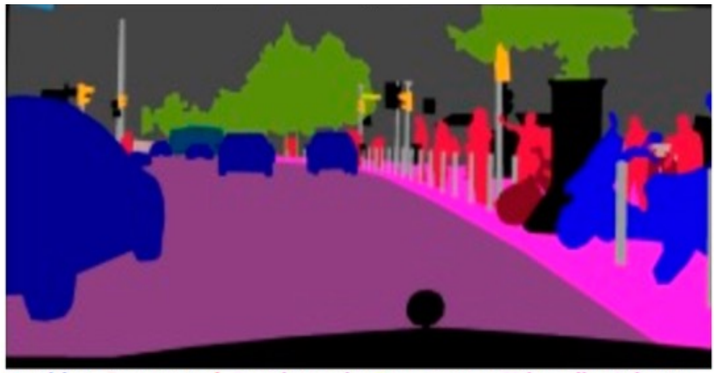


Car1,Car2, Car3, Person1, Person2,..

ผสานการแบ่งส่วนเชิงความหมายและการแบ่งส่วนเชิงอินสแตนซ์ เพื่อให้เข้าใจฉากได้อย่างครอบคลุมทั้งในระดับความหมาย (รถยนต์, อาคาร, ถนน, ฯลฯ) และระดับอินสแตนซ์ (รถยนต์ 1, รถยนต์ 2, ฯลฯ)

Emek Soylu B, Guzel MS, Bostanci GE, Ekinci F, Asuroglu T, Acici K. Deep-Learning-Based Approaches for Semantic Segmentation of Natural Scene Images: A Review. *Electronics*. 2023; 12(12):2730.

Semantic Segmentation



Stuff คือ สิ่งไม่มีขอบเขตชัดเจน เช่น ท้องฟ้า ถนน น้ำ จะทำ segmentation แบบ semantic (ทุก pixel ของ sky = class sky)

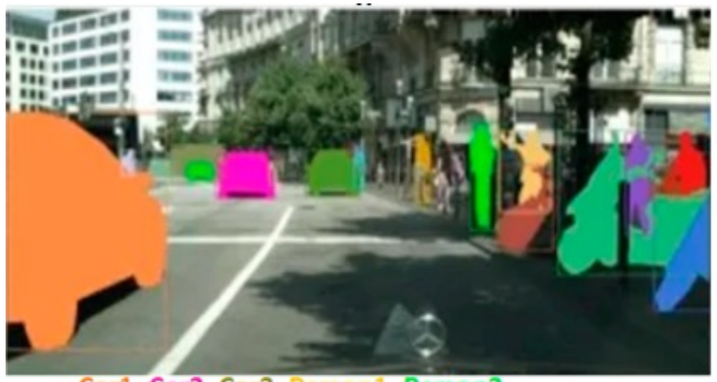
Panoptic Segmentation



คือ การรวม Semantic Segmentation และ Instance Segmentation เข้าด้วยกัน

ทุกพิกเซลในภาพ
ต้องถูกกำหนด label

Instance Segmentation



Thing คือ วัตถุที่นับเป็นชิ้น เช่น คน รถ เก้าอี้ จะทำ segmentation แบบ instance (แต่ละคน/รถ ได้ mask แยกกัน)

Emek Soylu B, Guzel MS, Bostanci GE, Ekinci F, Asuroglu T, Acici K. Deep-Learning-Based Approaches for Semantic Segmentation of Natural Scene Images: A Review. *Electronics*. 2023; 12(12):2730.



หัวข้อ

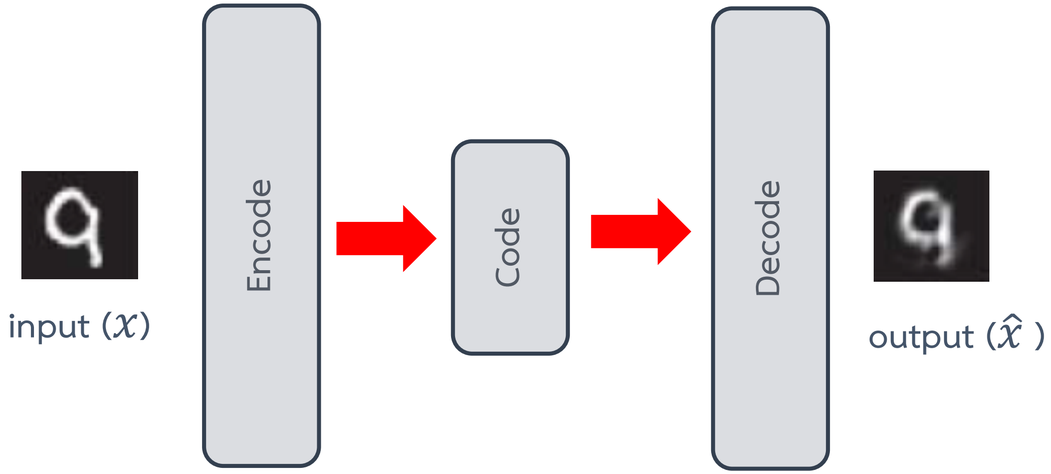
- แนวคิดสถาปัตยกรรม Autoencoder
- สถาปัตยกรรม Autoencoder ที่น่าสนใจ
- วิธีการใช้งานชุดคำสั่ง Segmentation Models



หัวข้อ

- แนวคิดสถาปัตยกรรม Autoencoder
- สถาปัตยกรรม Autoencoder ที่น่าสนใจ
- วิธีการใช้งานชุดคำสั่ง Segmentation Models

Autoencoder คืออะไร



คือ feedforward neural networks ประเภทหนึ่งที่ขนาดของข้อมูล input และ output มีขนาดเท่ากัน สถาปัตยกรรมแบบ Autoencoders จะทำการสกัดหรือบีบอัด input ให้อยู่ในรูปแบบรหัสข้อมูลที่มีมิติที่น้อยกว่า (Lower-dimensional Code) และใช้รหัสดังกล่าวเพื่อสร้าง output กลับมาจากรหัสดังกล่าว หรือสามารถกล่าวได้ว่าเป็นแบบจำลอง Neural Network ประเภทหนึ่งที่เรียนรู้ข้อมูล Input แล้วให้ผลลัพธ์ Output ออกมาเป็นข้อมูลเดิม (หรืออย่างน้อย width และ height เท่าเดิม) โดยทั่วไปจะประกอบด้วยกัน 3 ส่วน ได้แก่ encoder, code และ decoder ซึ่งการสร้างแบบจำลอง จำเป็นต้องอาศัยวิธีการ encoder วิธีการ decoder และ loss function เพื่อเปรียบเทียบ input กับ output

อ้างอิง https://towardsdatascience.com/applied-deep-learning-part-3-autoencoders-1c083af4d798



Autoencoder คืออะไร

สถาปัตยกรรมแบบ Autoencoders ได้รับความนิยมใช้งานทางด้านการลดขนาด (Dimensionality Reduction) หรือการบีบอัดข้อมูล (Data Compression) ที่มีคุณสมบัติที่สำคัญ 3 ประการ ได้แก่

- **ความเฉพาะของข้อมูล (Data-specific)**

แบบจำลอง Autoencoders สามารถบีบอัดข้อมูลที่คล้ายกับข้อมูลที่ได้รับการฝึกสอนเท่านั้นเนื่องจากแบบจำลอง Autoencoders ได้เรียนรู้คุณลักษณะเฉพาะจากชุดข้อมูลการฝึกสอนมา ทำให้แบบจำลอง Autoencoders จึงแตกต่างจากวิธีการบีบอัดข้อมูลมาตรฐานทั่วไป อาทิเช่น zip, winrar, 7zip ดังนั้น แบบจำลอง Autoencoders ที่ฝึกสอนจากชุดข้อมูลตัวเลขที่เขียนด้วยลายมือจะไม่สามารถบีบอัดภาพทิวทัศน์ได้

- **การบีบอัดแบบสูญเสีย (Lossy)**

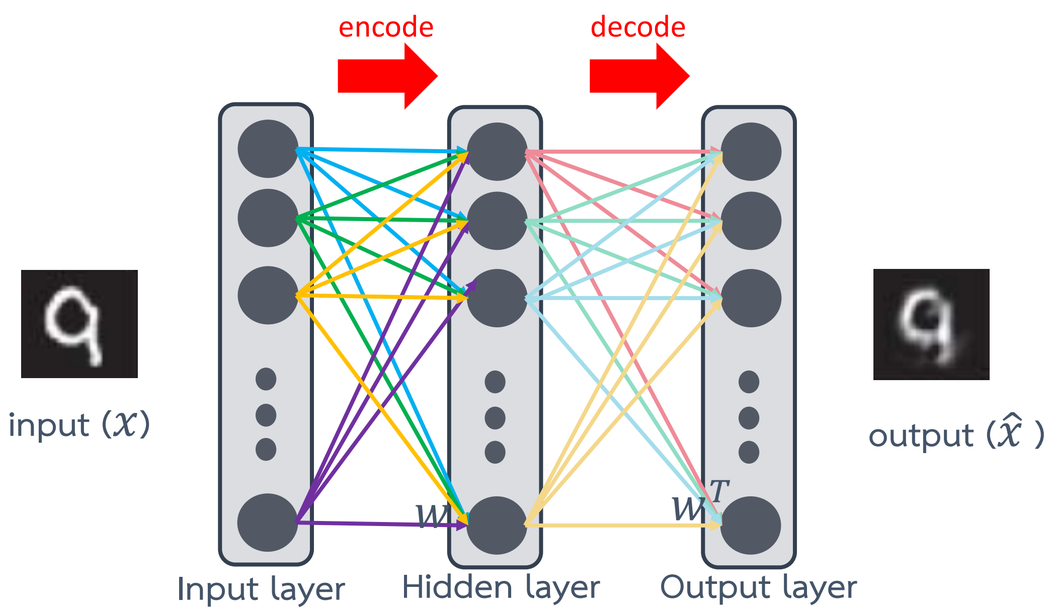
เป็นการตัดข้อมูลบางส่วนออกไปเพื่อลดขนาดของข้อมูล โดยข้อมูลที่ซ้ำซ้อนจะถูกตัดทิ้งอย่างถาวร ซึ่ง Output ของแบบจำลอง Autoencoders จะไม่เหมือนกับ Input ทุกประการ แต่จะเป็นลักษณะที่ใกล้เคียง (อาจจะเสื่อมโทรม)

- **ไม่มีผู้สอน (Unsupervised)**

การฝึกสอนแบบจำลอง Autoencoders ต้องอาศัยเพียง Input เท่านั้น และไม่จำเป็นต้องให้ Label (สำหรับงานบีบอัด แต่งานประเภทอื่น Label ยังคงมีความจำเป็น) ซึ่งถือว่าสถาปัตยกรรมแบบ Autoencoders มีลักษณะการเรียนรู้แบบไม่มี แต่แบบจำลองจะดำเนินการสร้าง Label จากข้อมูลฝึกสอนด้วยตนเอง เพื่อให้แบบจำลองมีความแม่นยำมากยิ่งขึ้น

อ้างอิง https://towardsdatascience.com/applied-deep-learning-part-3-autoencoders-1c083af4d798

หลักการ Autoencoder



ประกอบด้วย 3 ส่วน ได้แก่

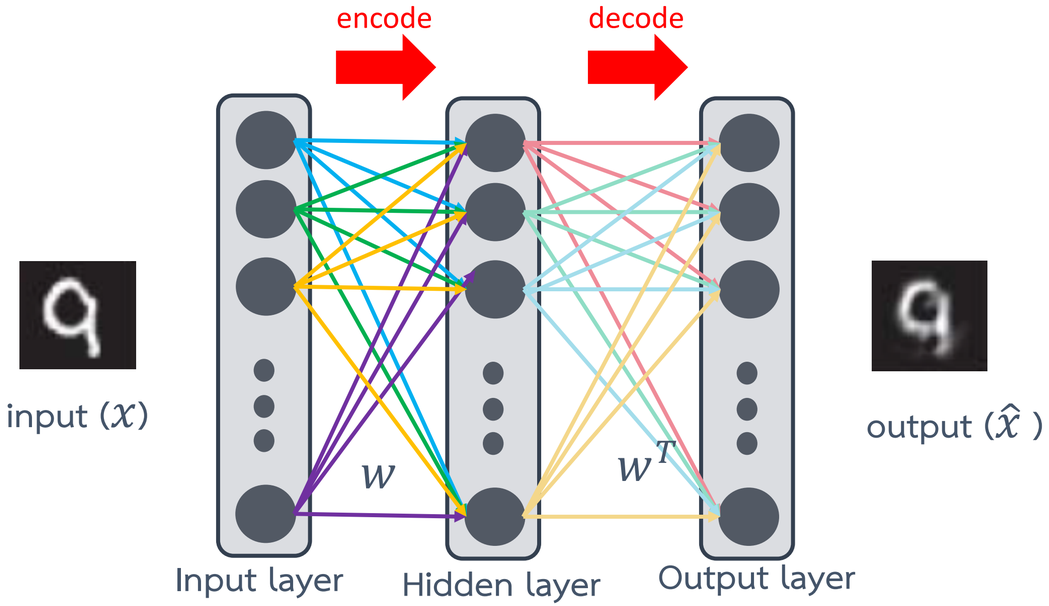
- การเข้ารหัส (Encoder) คือ การบีบอัดข้อมูลและสร้างรหัสข้อมูล
- รหัส (Code) คือ ตัวนำเสนอข้อมูล input แบบ "สรุป" หรือ "บีบอัด" ที่มีความกระชับ หรือที่เรียกว่า"การแสดงพื้นที่แบบแฝง (latent-space representation)"
- การถอดรหัส (Decoder) คือ กระบวนการสร้างอินพุทกลับมาใหม่จากรหัสข้อมูลที่บีบอัดได้

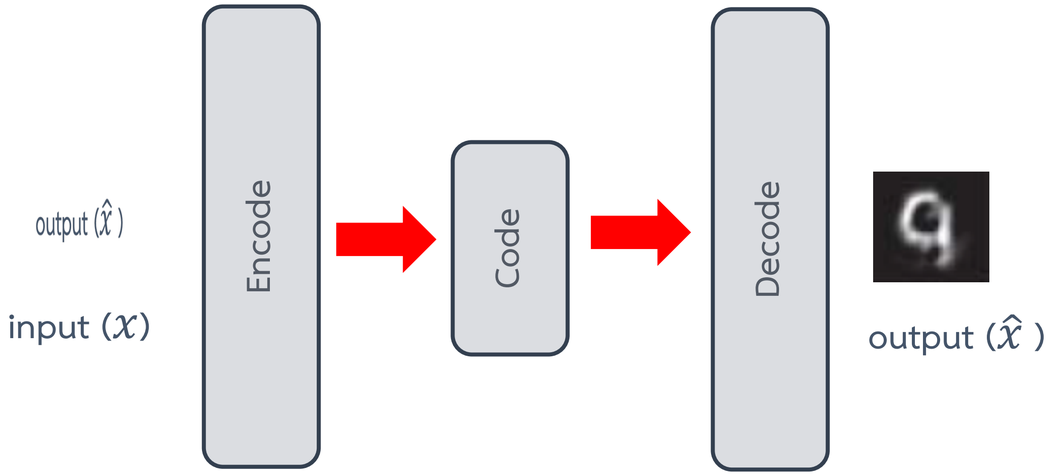
จุดประสงค์เพื่อ Minimize $(x - \hat{x})^2$ เข้าใกล้ 0 โดยที่

- กระบวนการ Encode คือ ขั้นตอนการนำเข้า x และส่งต่อให้ Hidden Layer
- ผลลัพธ์จาก Hidden Layer ถือว่าเป็น Code
- กระบวนการ Decode คือ ขั้นตอนการนำเข้า Code และส่งต่อให้ Output Layer

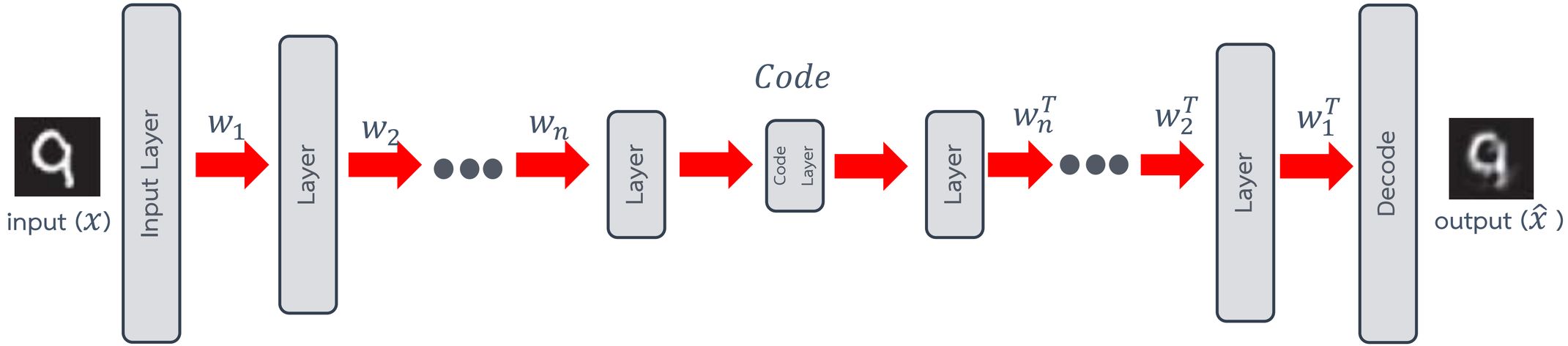
หลักการ Autoencoder

- โดยปกติแล้วขนาดของ x จะเท่ากับขนาด $\hat{x}$ *แต่ขนาดของ Code จะเล็กกว่า* x *และ* $\hat{x}$
- ค่าสัมประสิทธิ w และ w^T ในส่วนของ Encode และ Decode จะได้รับจากการฝึกสอน





หลักการ Autoencoder



- สำหรับ Deep Autoencoder สามารถมีจำนวนชั้นที่ลึกได้
- สถาปัตยกรรมของ Encode และ Decode ไม่จำเป็นต้องสมมาตร (Symmetric) กันเสมอไป
- สถาปัตยกรรมส่วน Encode นิยมค่อย ๆ ลดมิติของข้อมูลลงทุกเลเยอร์ด้วย Multilayer Neural Network และ Small Central Layer

อ้างอิง Hinton et al. "Reducing the dimensionality of data with neural networks." *Science* 313.5786 (2006): 504-507



หลักการ Autoencoder

ในการออกแบบสถาปัตยกรรม Deep Autoencoder สำหรับข้อมูลประเภทภาพ นิยมทำให้ในส่วนของ Encoder มักลดมิติของข้อมูลลงทุกเลเยอร์ ในขณะที่ในส่วนของ Decoder นิยมเพิ่มมิติของข้อมูลขึ้นทุกเลเยอร์ เนื่องจาก

- **การเรียนรู้คุณสมบัติที่สำคัญ (Feature Learning)** เป็นการลดมิติของข้อมูลในแต่ละเลเยอร์ของ Encoder ทำให้เครือข่ายสามารถเรียนรู้และสกัดคุณสมบัติที่สำคัญของภาพได้ดีขึ้น โดยเฉพาะคุณสมบัติที่เป็นเอกลักษณ์และสามารถนำไปใช้แยกแยะได้ในระดับสูง
- **การลดความซับซ้อนของข้อมูล (Dimensionality Reduction)** เป็นการค่อย ๆ ลดมิติของข้อมูลในส่วนของ Encoder ช่วยในการบีบอัดข้อมูล ทำให้ข้อมูลมีขนาดเล็กลงและง่ายต่อการประมวลผล ทั้งยังช่วยลด noise หรือข้อมูลที่ไม่สำคัญออกไป
- **การขยายข้อมูลกลับ (Reconstruction)** ในส่วนของ Decoder การค่อย ๆ เพิ่มมิติของข้อมูลในแต่ละเลเยอร์ช่วยให้สามารถขยายข้อมูลกลับมาเป็นรูปแบบดั้งเดิมได้อย่างละเอียดอ่อนและเป็นระเบียบมากขึ้น การเพิ่มมิติค่อยเป็นค่อยไปช่วยรักษาโครงสร้างและรายละเอียดของภาพ
- **การรักษาความสัมพันธ์เชิงพื้นที่ (Spatial Relationships)** การใช้ convolutional layers ในส่วนของ Encoder และ deconvolutional layers หรือ upsampling layers ในส่วนของ Decoder ช่วยรักษาความสัมพันธ์เชิงพื้นที่ของภาพ ทำให้การสร้างภาพใหม่มีความแม่นยำมากขึ้น
- **การเรียนรู้ที่ค่อยเป็นค่อยไป (Progressive Learning)** การค่อย ๆ ลดและเพิ่มมิติของข้อมูลช่วยให้เครือข่ายสามารถเรียนรู้และปรับตัวกับข้อมูลได้อย่างมีประสิทธิภาพ และสามารถสร้างภาพใหม่ได้ใกล้เคียงกับภาพต้นฉบับมากที่สุด

อ้างอิง Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. และ Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the Dimensionality of Data with Neural Networks. Science, 313(5786), 504-507.

หลักการ Autoencoder

ตัวอย่างการบีบอีดภาพ (Image Compression) ระหว่าง PCA และ Autoencoder

Original Image



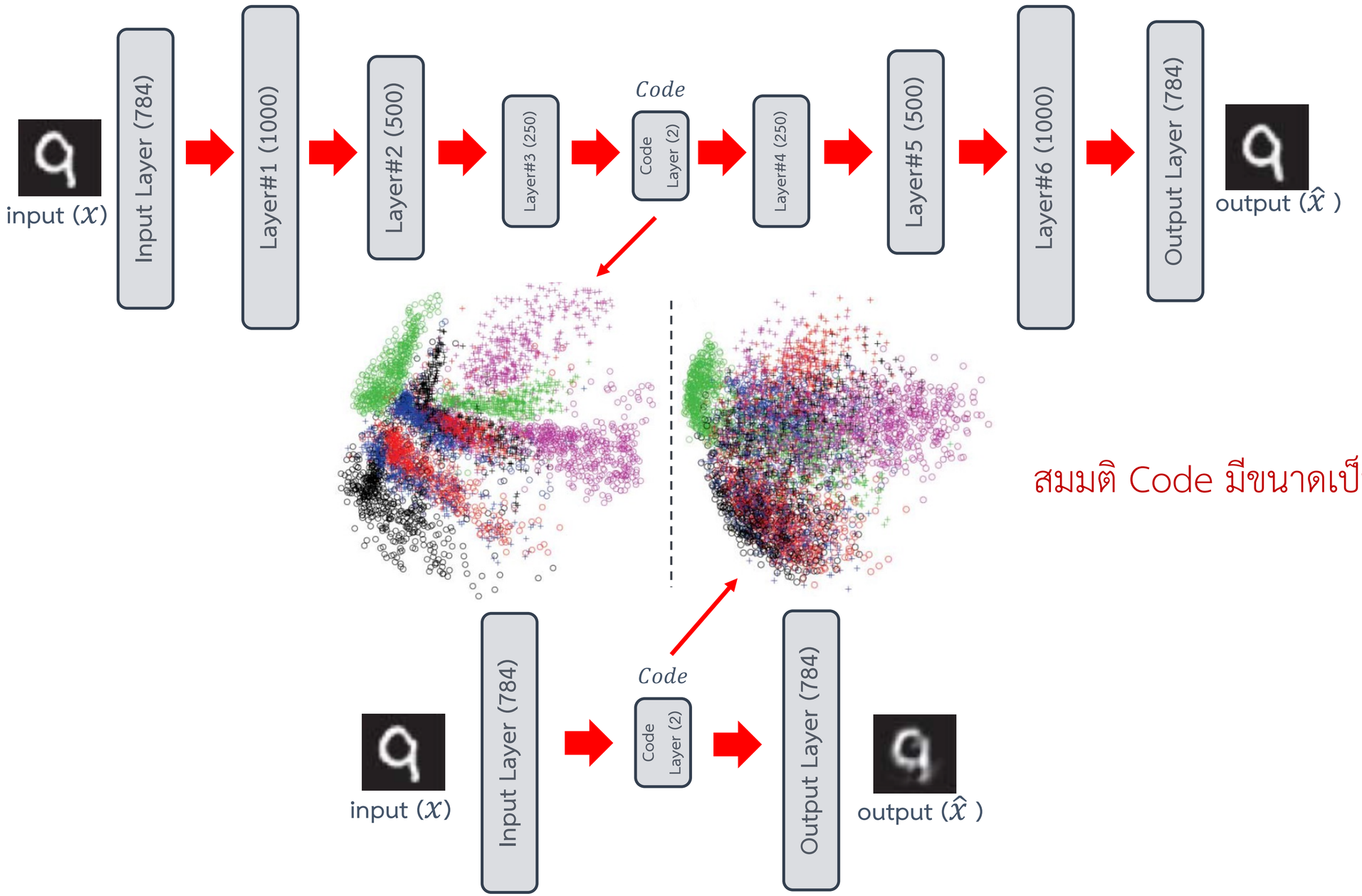
Principal Component Analysis



Deep Autoencoder

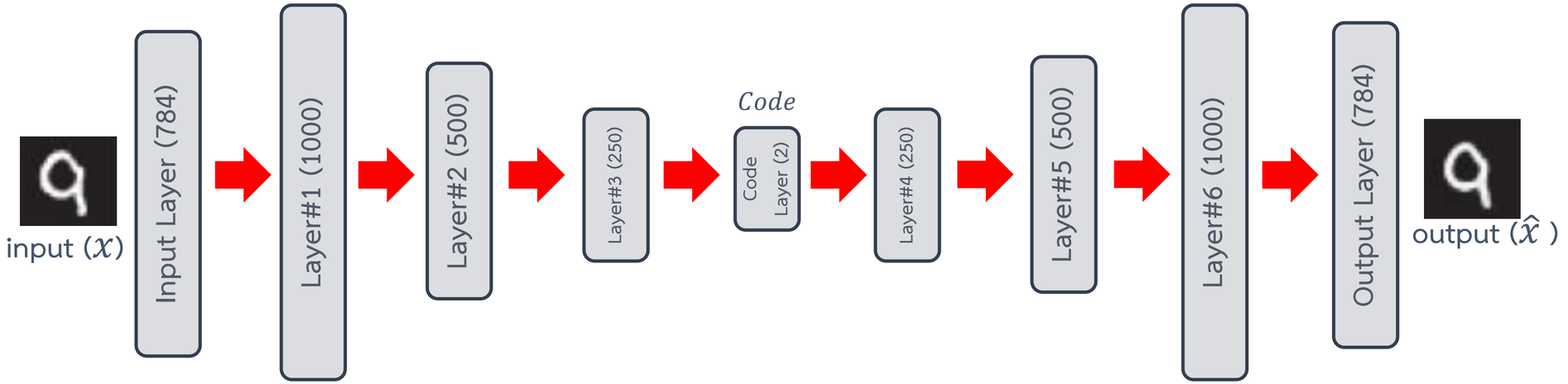


ถ้ามีการใช้ Component จำนวนเท่ากันจะพบว่า Autoencoder ให้ผลลัพธ์ที่ดีกว่า PCA ในกรณีที่ Autoencoder มีค่า Initial Weight ที่ดีและเหมาะสม



สมมติ Code มีขนาดเป็น 2 มิติ

หลักการ Autoencoder



- Encode และ Decode นิยมพัฒนาด้วย fully-connected feedforward neural networks แต่ถ้าสำหรับภาพจะเป็น Convolutional Neural Network
- Code เป็นเลเยอร์เดียวของ ANN ซึ่งจำนวนโหนดในเลเยอร์จะเรียกว่า code size ถือว่าเป็น Hyperparameter ที่ต้องกำหนดก่อนฝึกสอน ซึ่งพอจะเทียบเคียงได้กับการกำหนดจำนวน Principal Component ที่เลือกใน PCA

อ้างอิง Hinton et al. "Reducing the dimensionality of data with neural networks." *Science* 313.5786 (2006): 504-507



หลักการ Autoencoder

สถาปัตยกรรมแบบ Autoencoder จะมี Hyperparameter ที่สำคัญจำนวน 4 ตัว ได้แก่

Hyperparameter	คำอธิบาย
Code Size	จำนวนโหนดในชั้น Code Layer ขนาดที่เล็กลงส่งผลให้มีการบีบอัดมากขึ้น
Number of Layers	สถาปัตยกรรมแบบ Autoencoder สามารถมีความลึกได้เท่าที่ต้องการ โดยไม่คำนึงถึงขนาดของ Input และ Output
Number of Nodes per Layer	สถาปัตยกรรมแบบ Autoencoder มักจะมีการเรียงตัวแบบซ้อน (Stack) เนื่องจากเลเยอร์ต่าง ๆ จะถูกซ้อนกันทีละชั้น จำนวนโหนดต่อเลเยอร์จะลดลงตามแต่ละเลเยอร์ถัดไปในส่วนของ Encoder และเพิ่มกลับเข้าไปในส่วนของ Decoder นอกจากนี้ ฝั่ง Encoder และ Decoder มักจะสมมาตรกัน แต่ไม่จำเป็น
Loss Function	สถาปัตยกรรมแบบ Autoencoder มักนิยมใช้งานค่า mean squared error (MSE) หรือค่า binary cross-entropy เป็น Loss Function ถ้าหากค่า Input อยู่ในช่วง 0 ถึง 1 นิยมใช้งาน cross-entropy

สถาปัตยกรรมแบบ Autoencoder จะได้รับการฝึกสอนด้วยวิธีการ backpropagation เช่นเดียวกัน ANN และ CNN



ข้อควรระวัง

- ประสิทธิภาพของสถาปัตยกรรมแบบ Autoencoder ขึ้นกับการปรับค่า Hyperparameter ต่าง ๆ โดยที่ Code Size คือตัวที่สำคัญที่สุด การเพิ่มค่าของ Hyperparameter ส่งผลให้แบบจำลอง Autoencoder สามารถเรียนรู้การเข้ารหัสที่ซับซ้อนได้มากขึ้น แต่ไม่ควรปรับให้มีค่ามากเกินไป มิฉะนั้นแบบจำลองจะเรียนรู้ที่จะคัดลอก (Copy) Input ไปยัง Output โดยไม่ต้องเรียนรู้ที่จะสกัด Code ที่มีความสามารถในการนำเสนอข้อมูลได้ ส่งผลให้แบบจำลอง Autoencoder จะสร้าง Output ขึ้นมาใหม่เหมือนกับ Input ได้อย่างสมบูรณ์แบบ สิ่งนี้ทำให้เกิดเหตุการณ์ Overfitting ขึ้นมาได้
- ดังนั้น จึงนิยมให้ Code มีขนาดเล็กกว่าขนาดของ Input ส่งผลให้รูปแบบสถาปัตยกรรมเป็นรูปโบว์ หรือนาฬิกาทราย ทำให้แบบจำลอง Autoencoder ไม่สามารถคัดลอก Input ไปยัง Output ได้โดยตรง ถือว่าเป็นบังคับให้แบบจำลองเรียนรู้ Feature ที่เหมาะสมและสำคัญออกมาได้

อ้างอิง Hinton et al. "Reducing the dimensionality of data with neural networks." *Science* 313.5786 (2006): 504-507

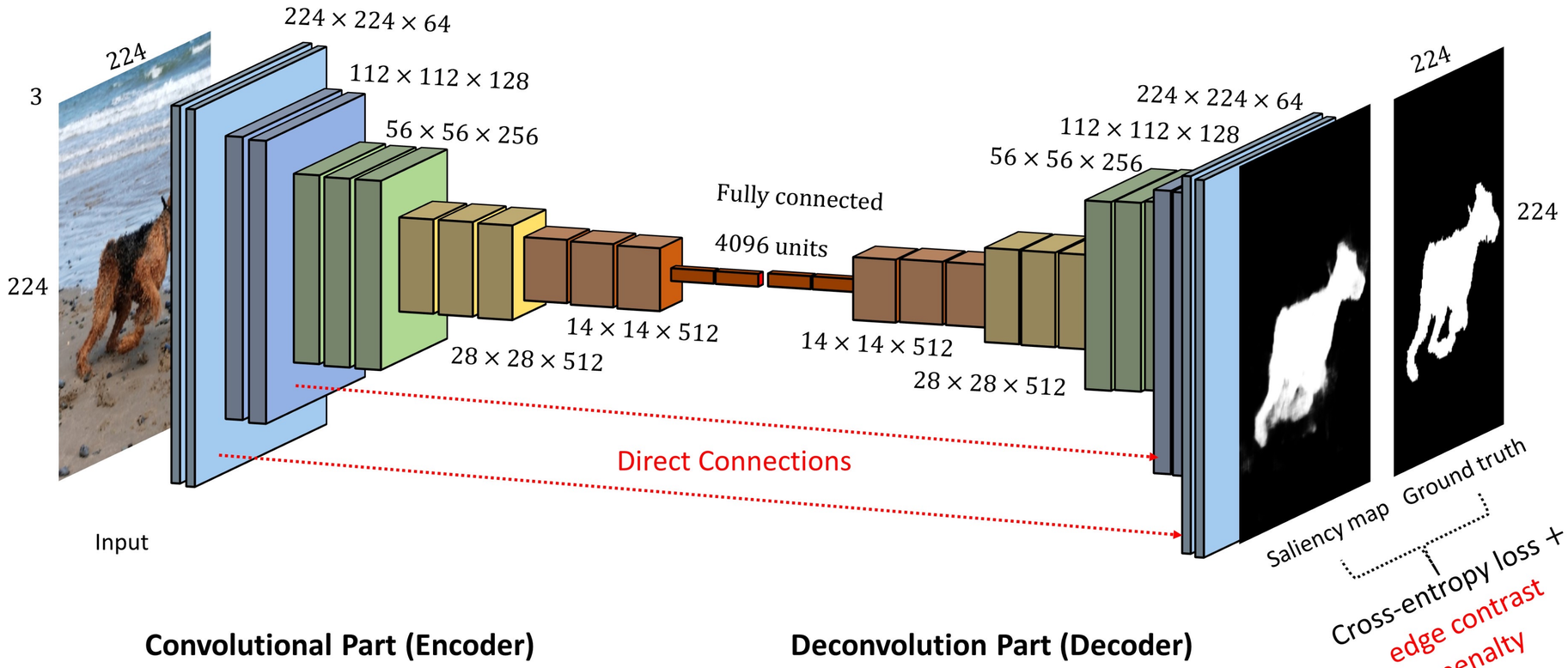


ข้อควรระวัง

อย่างไรก็ตาม หากข้อมูล Input มีรูปแบบ (Pattern) อาทิเช่น เลข “1” มีลักษณะเป็นเส้นตรงและเลข “0” มีลักษณะเป็นวงกลม แบบจำลองจะเรียนรู้รูปแบบและความสัมพันธ์ดังกล่าวได้ แต่หากข้อมูล Input เป็นรูปแบบสุ่มโดยสมบูรณ์ กล่าวคือไม่มีความสัมพันธ์ภายใน (Internal Correlation) หรือการอ้างอิง (Dependency) จะส่งผลให้แบบจำลอง Autoencoder ไม่สามารถสร้าง Output ใหม่จาก Code ได้อย่างสมบูรณ์

อ้างอิง Hinton et al. "Reducing the dimensionality of data with neural networks." *Science* 313.5786 (2006): 504-507

Convolutional Autoencoder



สถาปัตยกรรม Autoencoder ที่ใช้งาน Convolution Operation แทนการใช้งาน Fully Connected Layer ได้รับการคิดค้นโดย Zeiler และคณะในปี ค.ศ. 2013

อ้างอิง https://github.com/arthurmeyer/Saliency_Detection_Convolutional_Autoencoder



Convolutional Autoencoder

สถาปัตยกรรม Convolutional Autoencoder รองรับ Input ในรูปแบบ 3D Vectors หรือ Input Image ในสถาปัตยกรรมนี้ภาพ Input จะถูก Downsampling ให้อยู่ในรูปแบบ Latent Representation (หรือเรียกว่า Spatial Information) ที่มีขนาดเล็ก โดยสถาปัตยกรรม Convolutional Autoencoder สามารถนำไปประยุกต์ใช้งานทางด้าน Image and Video Compression, image denoising, และ image synthesis

เทคนิค Upsampling ใน Autoencoder

ใน CNN Autoencoder ส่วนของ Encoder จะลดขนาดของข้อมูล (Downsampling/Pooling) เพื่อลดมิติและดึงคุณลักษณะสำคัญออกมา แต่ในส่วนของ Decoder เราต้องการขยายภาพกลับ (Upsampling) ให้ได้ขนาดเท่าเดิม โดยมักจะใช้เทคนิคการขยายภาพกลับ ได้แก่

- การประมาณค่า (Interpolation) อาทิเช่น Nearest Neighbor Upsampling, Bilinear Interpolation,
- การคอนโวลูชันแบบย้อนกลับ (Transposed Convolution)
- การยกเลิกการพูลลิ่ง (Unpooling)

เทคนิคการ Nearest Neighbor Interpolation

กำหนดให้ข้อมูลเดิม x มีขนาด 2x2 และต้องการขยายขนาดเป็น 5x5

$$X = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \quad \Rightarrow \quad Y_{NN} = \begin{bmatrix} 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 30 & 30 & 40 & 40 & 40 \\ 30 & 30 & 40 & 40 & 40 \\ 30 & 30 & 40 & 40 & 40 \end{bmatrix}$$

ขั้นตอนที่ 1 คำนวณพิกัดต้นทาง src(row) และ src(column) แล้ว “ปัดไปพิกเซลอินพุตที่ใกล้ที่สุด”

พิจารณาจากเดิม [0,1] ไปเป็น [0,0.25,0.5,0.75,1]

ขั้นตอนที่ 2 ใช้กฎ nearest = $\lfloor src+0.5 \rfloor$ เพื่อคุมไม่ให้เกินขอบ

nearest = [$\lfloor 0+0.5 \rfloor$, $\lfloor 0.25+0.5 \rfloor$, $\lfloor 0.5+0.5 \rfloor$, $\lfloor 0.75+0.5 \rfloor$, $\lfloor 1+0.5 \rfloor$] = [0, 0, 1, 1, 1]

เทคนิคการ Nearest Neighbor Interpolation

กำหนดให้ข้อมูลเดิม x มีขนาด 2x2 เป็น Y ที่มีการขยายขนาดเป็น 5x5

$$X = \begin{matrix} (0,0) & (0,1) \\ \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \\ (1,0) & (1,1) \end{matrix}$$

$$Y_{NN} = \begin{matrix} 0, & 0, & 1, & 1, & 1 \\ \begin{bmatrix} 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 30 & 30 & 40 & 40 & 40 \\ 30 & 30 & 40 & 40 & 40 \\ 30 & 30 & 40 & 40 & 40 \end{bmatrix} \end{matrix} \begin{matrix} 0, \\ 0, \\ 1, \\ 1, \\ 1 \end{matrix}$$

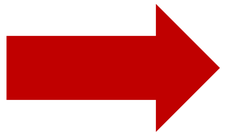
ขั้นตอนที่ 3 การแทนค่า

- เมื่อพิจารณาจาก src(col) = [0, 0, 1, 1, 1] แล้วจะกำหนดให้แถว 0 และ 1 ของ Y กำหนดให้ใช้แถวที่ 0 ของ X และแถว 2 ถึง 4 ของ Y กำหนดให้ใช้แถวที่ 1 ของ X
- เมื่อพิจารณาจาก src(row) = [0, 0, 1, 1, 1] แล้วจะกำหนดให้คอลัมน์ 0 และ 1 ของ Y กำหนดให้ใช้คอลัมน์ที่ 0 ของ X และคอลัมน์ 2 ถึง 4 ของ Y กำหนดให้ใช้คอลัมน์ที่ 1 ของ X

เทคนิคการ Bilinear Interpolation Interpolation

กำหนดให้ข้อมูลเดิม x มีขนาด 2x2 และต้องการขยายขนาดเป็น 5x5

$$X = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$



$$Y_{\text{bilinear}} = \begin{bmatrix} 10 & 12.5 & 15 & 17.5 & 20 \\ 15 & 17.5 & 20 & 22.5 & 25 \\ 20 & 22.5 & 25 & 27.5 & 30 \\ 25 & 27.5 & 30 & 32.5 & 35 \\ 30 & 32.5 & 35 & 37.5 & 40 \end{bmatrix}$$

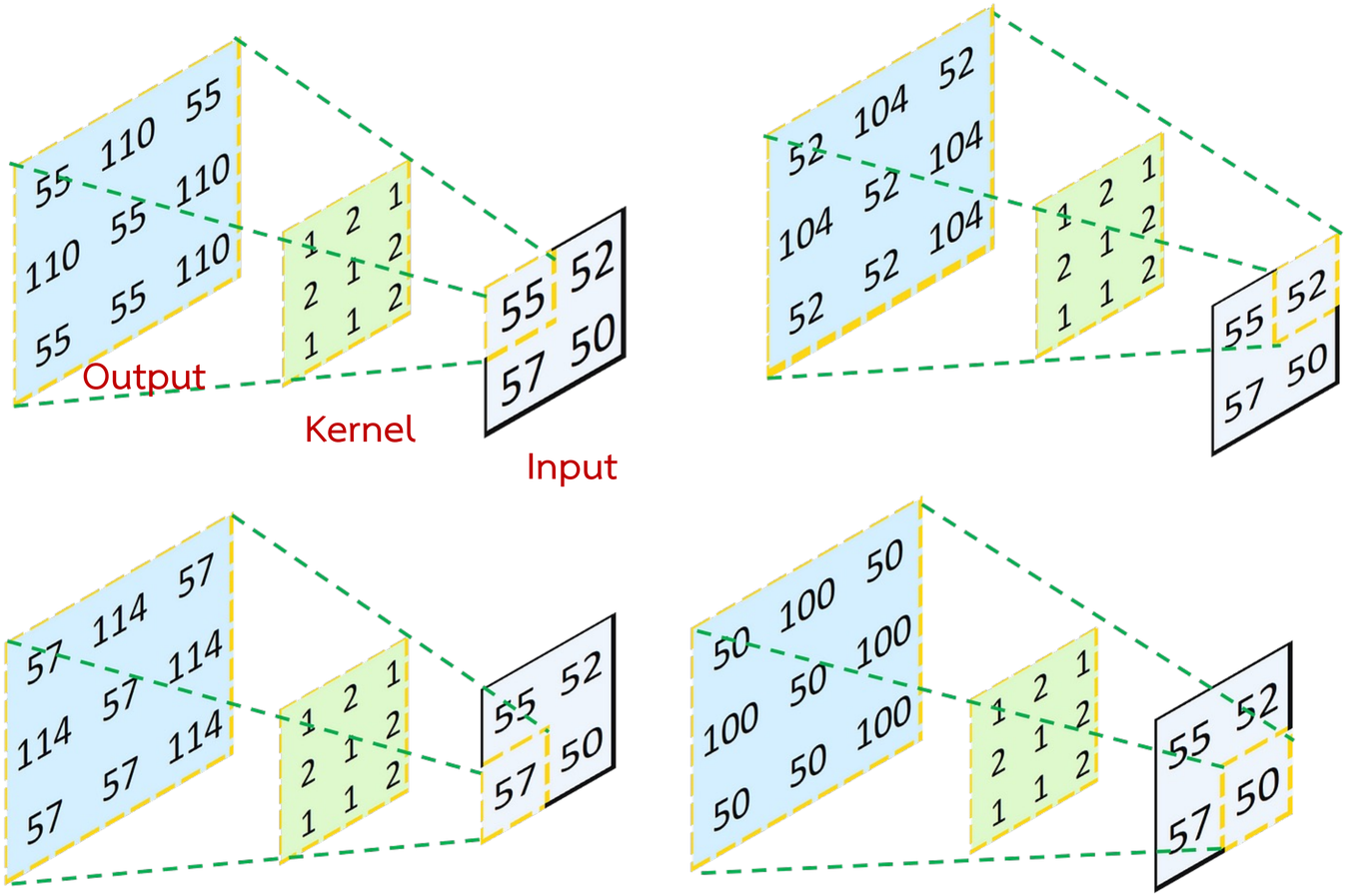
Transpose Convolutional Layer

Transpose Convolution (หรือ Deconvolution) เป็นกระบวนการทำกลับของการทำ Convolution ซึ่งกระบวนการ Transpose Convolution ด้วย Kernel ขนาด 3x3 จะแปลงข้อมูล Input ขนาด 1x1 ไปเป็นภาพผลลัพธ์ขนาด 3x3

หลักการทำงานของ Transpose Convolution จะมีหลักการทำงานคล้ายคลึงกัน Convolution โดยขยับ Kernel ไปคำนวณคูณแบบจุดต่อจุดบนภาพ Input ก่อนนำผลลัพธ์ที่ได้ทั้งหมดมาบวกกัน ผลลัพธ์ที่ได้จะมีขนาดใหญ่กว่าข้อมูล Input ขึ้นอยู่กับพารามิเตอร์เกี่ยวกับ stride และ padding

Transposed Convolutional Layer

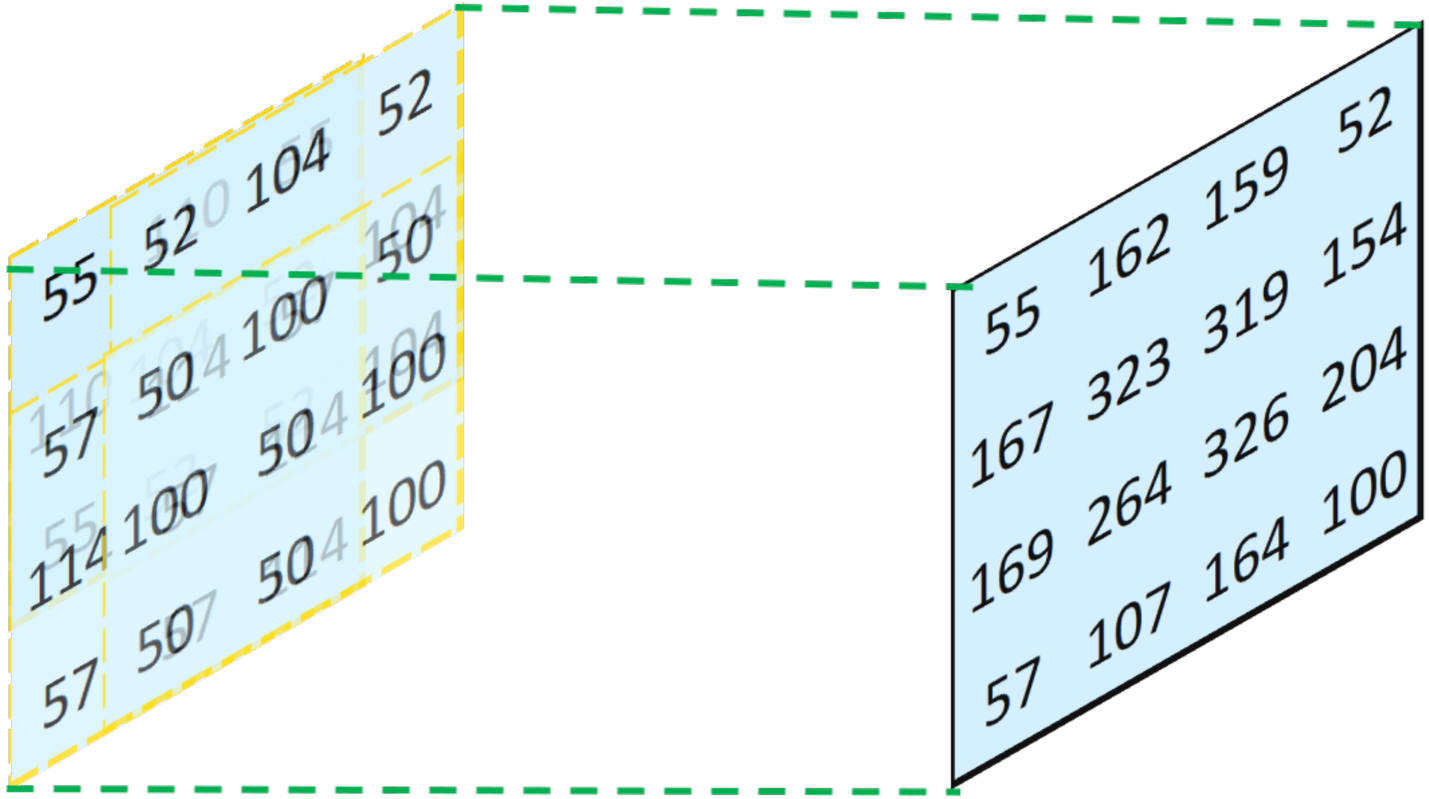
ขั้นตอนที่ 1 คูณกระจาย Input กับ Kernel ในแต่ละส่วนดังภาพ (stride = 1)



อ้างอิง https://towardsdatascience.com/understand-transposed-convolutions-and-build-your-own-transposed-convolution-layer-from-scratch-4f5d97b2967

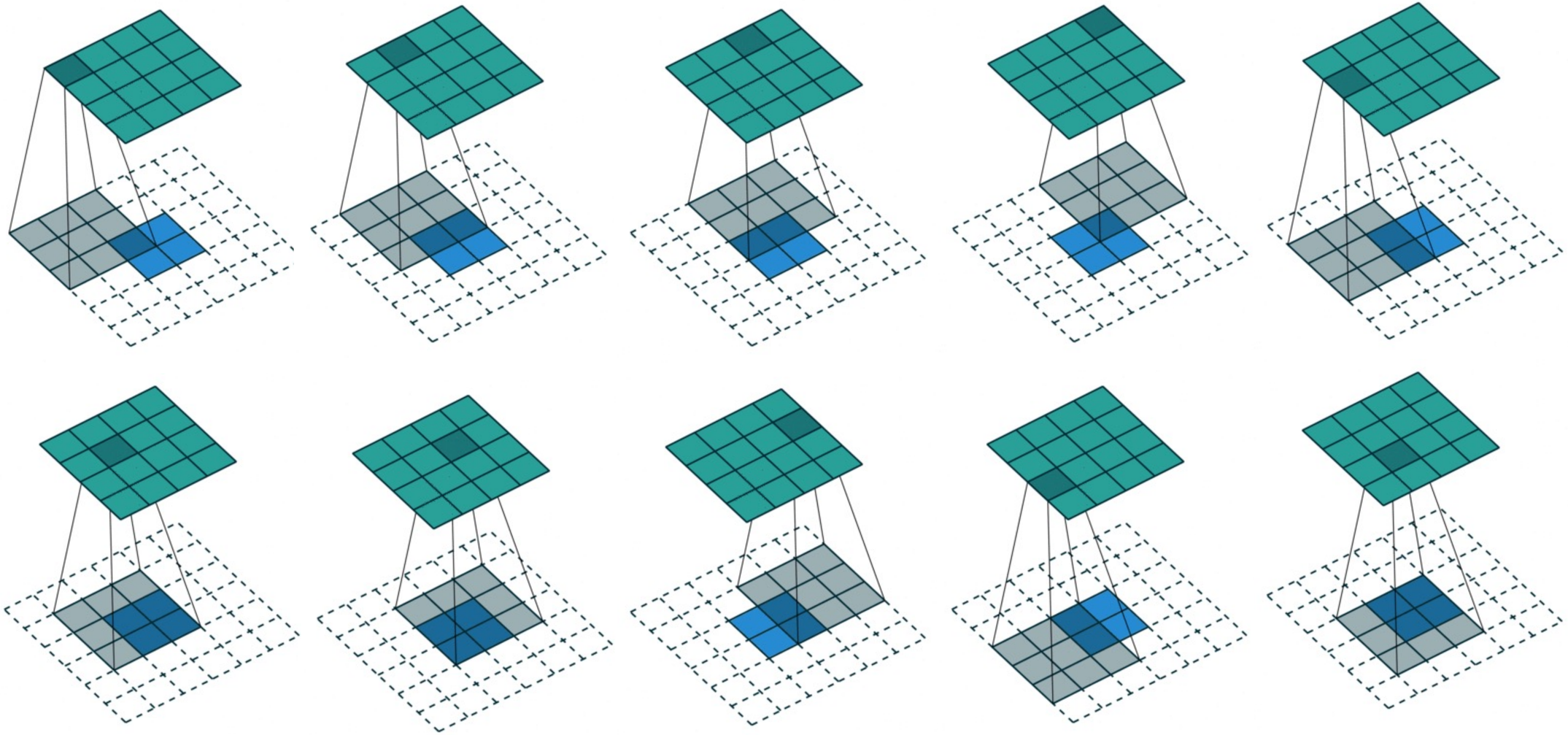
Transposed Convolutional Layer

ขั้นตอนที่ 2 บวกผลลัพธ์เข้าด้วยกันดังภาพ



อ้างอิง https://towardsdatascience.com/understand-transposed-convolutions-and-build-your-own-transposed-convolution-layer-from-scratch-4f5d97b2967

Transposed Convolutional Layer



อางอง https://medium.com/@marsxiang/convolutions-transposed-and-deconvolution-6430c358a5b6

Transposed Convolutional Layer

Input X =

1	1	1	1
1	1	1	1
3	1	2	2
3	1	2	2

Kernel K =

1	1	1
1	1	1
1	1	1

Padding = 0 • Stride = 2

ขั้นที่ 1: คูณแต่ละค่าด้วย Kernel

1 X = (1 1 1 1 / 1 1 1 1 / 3 1 2 2 / 3 1 2 2) → X[1,1] = 1

1 × (1 1 1 / 1 1 1 / 1 1 1) = (1 1 1 / 1 1 1 / 1 1 1)

2 X = (1 1 1 1 / 1 1 1 1 / 3 1 2 2 / 3 1 2 2) → X[3,1] = 3

3 × (1 1 1 / 1 1 1 / 1 1 1) = (3 3 3 / 3 3 3 / 3 3 3)

3 X = (1 1 1 1 / 1 1 1 1 / 3 1 2 2 / 3 1 2 2) → X[3,3] = 2

2 × (1 1 1 / 1 1 1 / 1 1 1) = (2 2 2 / 2 2 2 / 2 2 2)

ดัชนีเริ่มที่ 1: [แถว, คอลัมน์]

ผลคูณจากแต่ละตำแหน่งของ Input

X[1,1] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[1,2] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[1,3] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[1,4] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[2,1] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[2,2] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[2,3] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[2,4] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[3,1] = 3 (3 3 3 / 3 3 3 / 3 3 3)

X[3,2] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[3,3] = 2 (2 2 2 / 2 2 2 / 2 2 2)

X[3,4] = 2 (2 2 2 / 2 2 2 / 2 2 2)

X[4,1] = 3 (3 3 3 / 3 3 3 / 3 3 3)

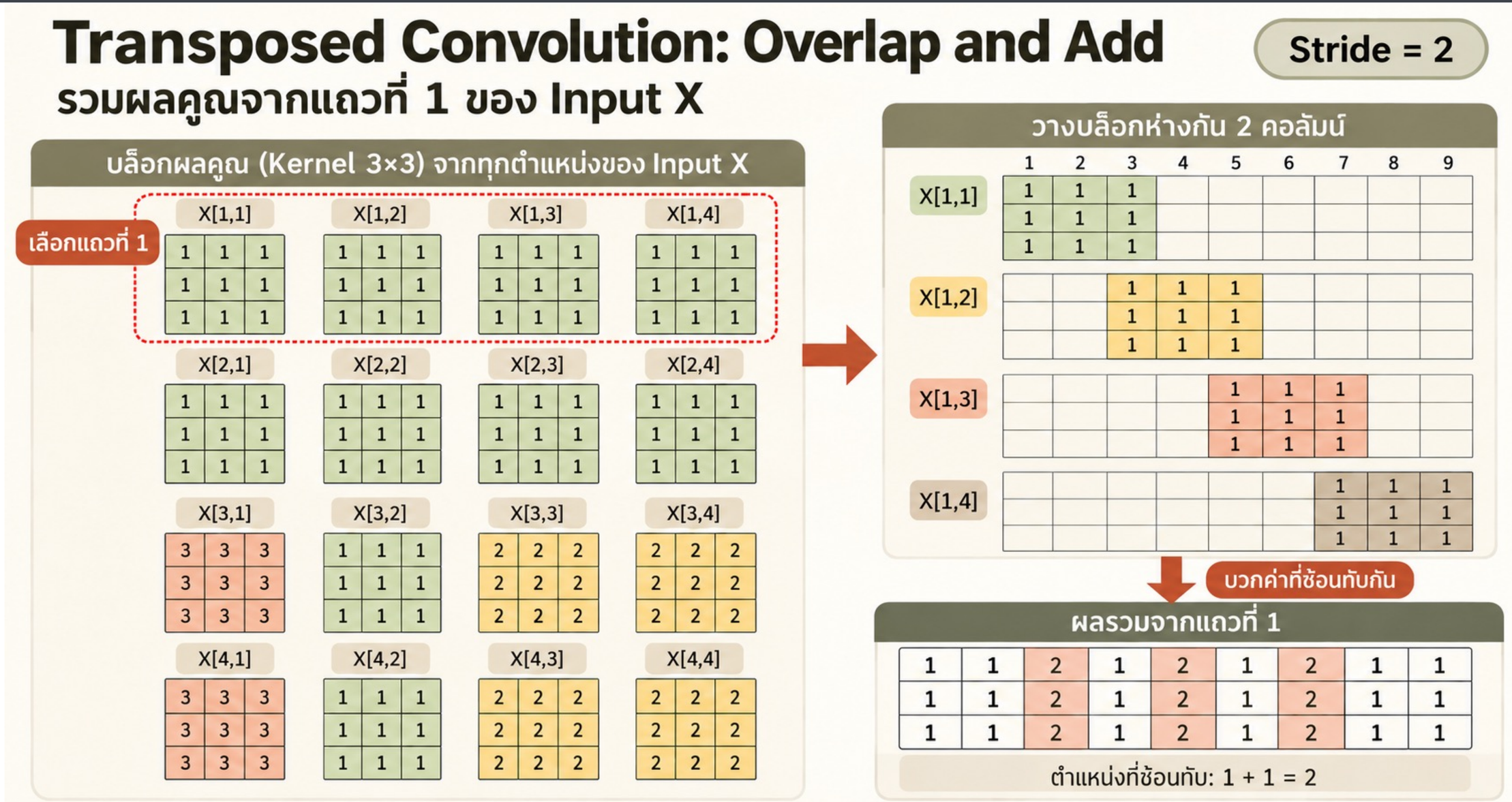
X[4,2] = 1 (1 1 1 / 1 1 1 / 1 1 1)

X[4,3] = 2 (2 2 2 / 2 2 2 / 2 2 2)

X[4,4] = 2 (2 2 2 / 2 2 2 / 2 2 2)

ขั้นถัดไป: วางผลคูณตาม **Stride** และบวกค่าที่ซ้อนทับกัน

Transposed Convolutional Layer



Transposed Convolutional Layer

Transposed Convolution: Overlap and Add

Stride = 2

รวมผลคูณจากแถวที่ 1 ของ Input X

Sum at overlapping positions

1	1	1+1	1	1+1	1	1+1	1	1
1	1	1+1	1	1+1	1	1+1	1	1
1	1	1+1	1	1+1	1	1+1	1	1

วางบล็อกห่างกัน 2 คอลัมน์

	1	2	3	4	5	6	7	8	9
X[1,1]	1	1	1						
	1	1	1						
	1	1	1						
X[1,2]			1	1	1				
			1	1	1				
			1	1	1				
X[1,3]					1	1	1		
					1	1	1		
					1	1	1		
X[1,4]							1	1	1
							1	1	1
							1	1	1

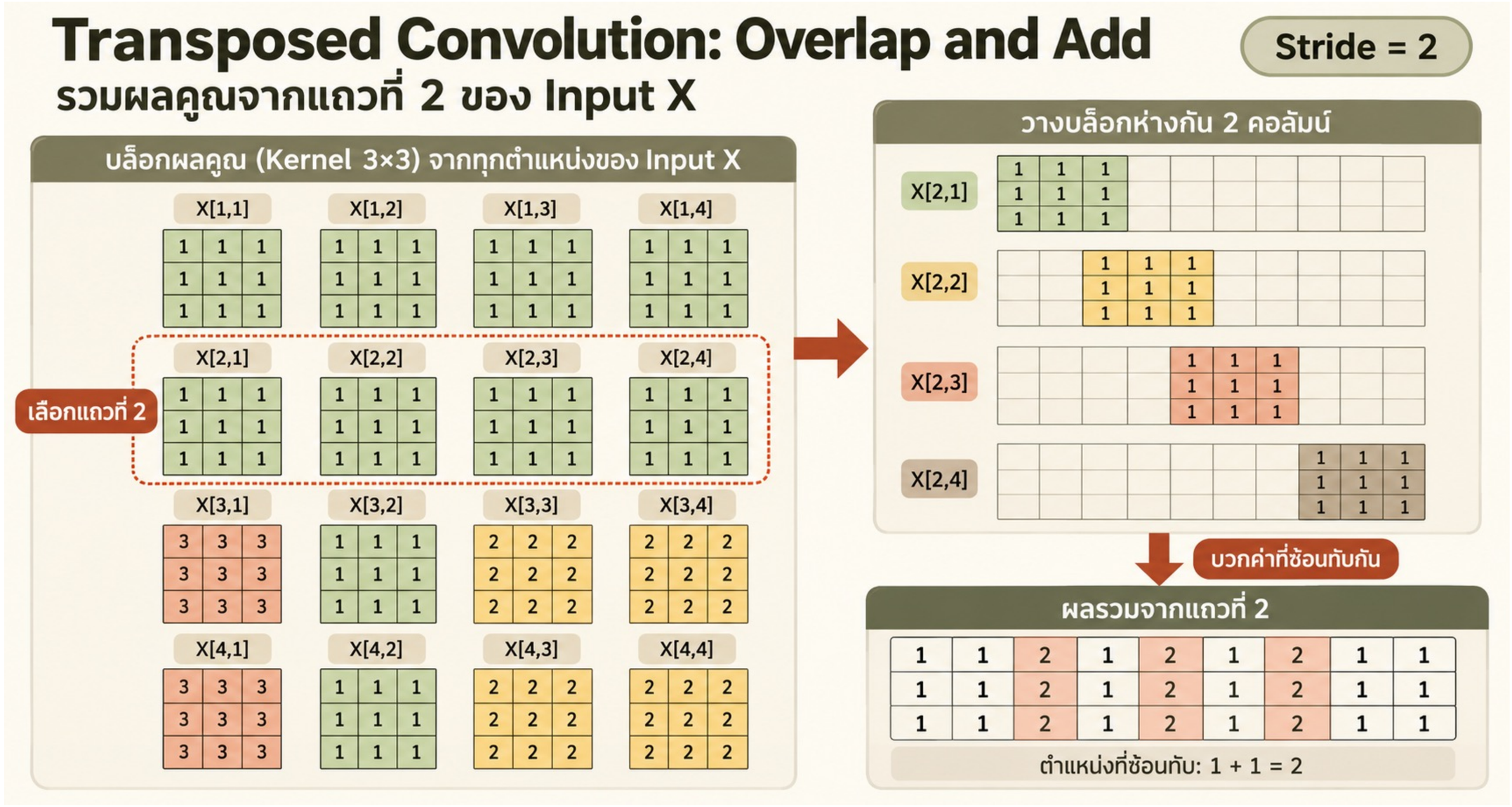
บวกค่าที่ซ้อนทับกัน

ผลรวมจากแถวที่ 1

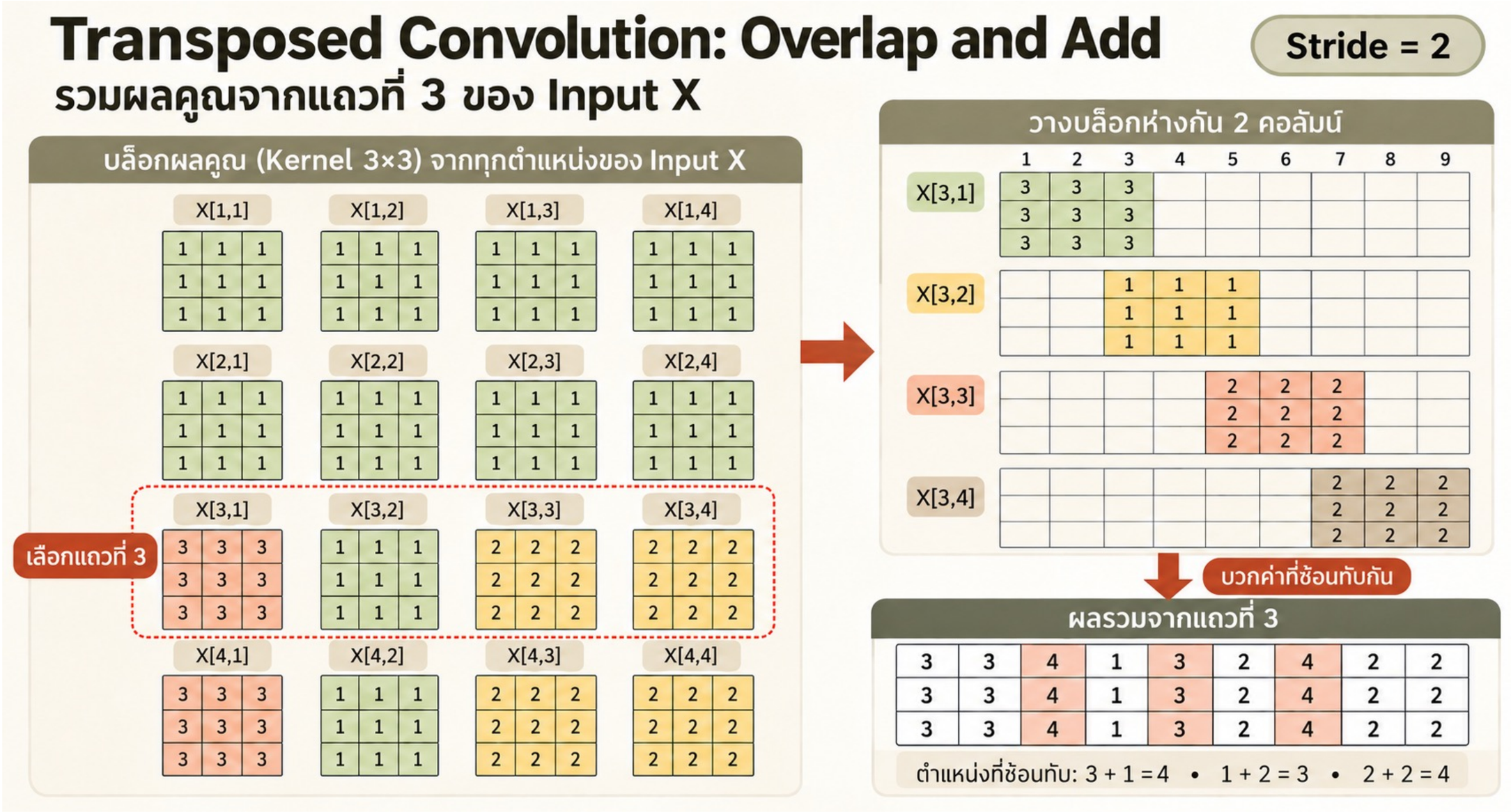
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ตำแหน่งที่ซ้อนทับ: 1 + 1 = 2

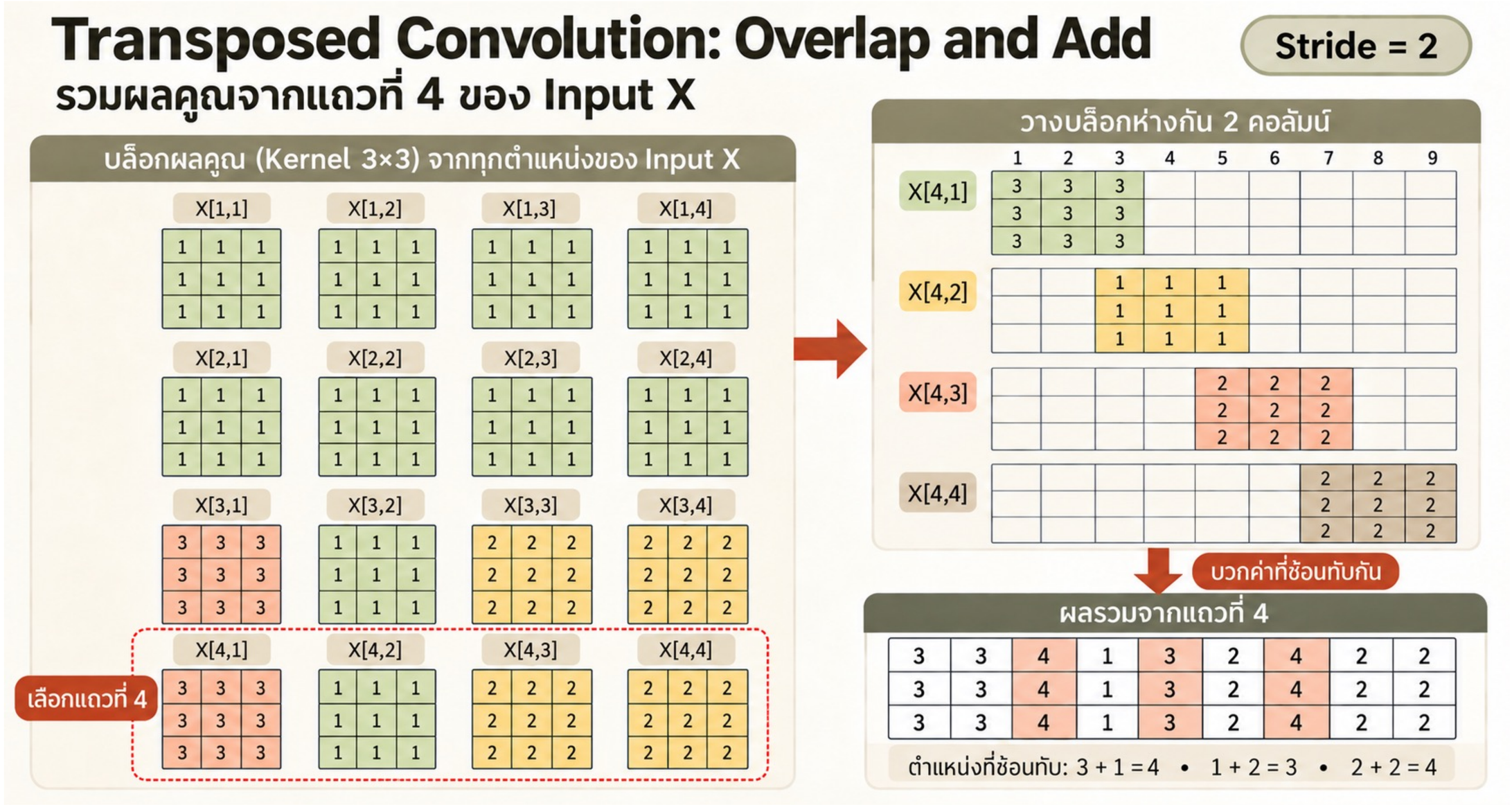
Transposed Convolutional Layer



Transposed Convolutional Layer



Transposed Convolutional Layer



Transposed Convolutional Layer

Transposed Convolution: Overlap and Add

Stride = 2

ผลรวมแนวนอนจากทั้ง 4 แถวของ Input X

บล็อกผลคูณ (Kernel 3×3) จากทุกตำแหน่งของ Input X

X[1,1]

1	1	1
1	1	1
1	1	1

X[1,2]

1	1	1
1	1	1
1	1	1

X[1,3]

1	1	1
1	1	1
1	1	1

X[1,4]

1	1	1
1	1	1
1	1	1

X[2,1]

1	1	1
1	1	1
1	1	1

X[2,2]

1	1	1
1	1	1
1	1	1

X[2,3]

1	1	1
1	1	1
1	1	1

X[2,4]

1	1	1
1	1	1
1	1	1

X[3,1]

3	3	3
3	3	3
3	3	3

X[3,2]

1	1	1
1	1	1
1	1	1

X[3,3]

2	2	2
2	2	2
2	2	2

X[3,4]

2	2	2
2	2	2
2	2	2

X[4,1]

3	3	3
3	3	3
3	3	3

X[4,2]

1	1	1
1	1	1
1	1	1

X[4,3]

2	2	2
2	2	2
2	2	2

X[4,4]

2	2	2
2	2	2
2	2	2

ผลรวมจากแถวที่ 1

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ผลรวมจากแถวที่ 2

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ผลรวมจากแถวที่ 3

3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

ผลรวมจากแถวที่ 4

3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

Transposed Convolutional Layer

Transposed Convolution: Vertical Overlap and Add

Stride = 2

ผลรวมจากแถวที่ 1

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ผลรวมจากแถวที่ 2

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ผลรวมจากแถวที่ 3

3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

ผลรวมจากแถวที่ 4

3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

เลื่อนลง 2 แถว แล้วบวกส่วนที่ซ้อนทับ

1	1	1	2	1	2	1	2	1	1
2	1	1	2	1	2	1	2	1	1
3	2	2	4	2	4	2	4	2	2
4	1	1	2	1	2	1	2	1	1
5	1	1	2	1	2	1	2	1	1

แถวที่ 3: ผลรวมจากส่วนที่ซ้อนทับ

1 + 1 = 2 • 2 + 2 = 4

ผลลัพธ์ชั่วคราว 5 × 9

Transposed Convolutional Layer

Transposed Convolution: Vertical Overlap and Add

Stride = 2

ผลรวมจากแถวที่ 1

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ผลรวมจากแถวที่ 2

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ผลรวมจากแถวที่ 3

3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

ผลรวมจากแถวที่ 4

3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

เพิ่มผลรวมจากแถวที่ 3 โดยเลื่อนลง 2 แถว

1	1	1	2	1	2	1	2	1	1
2	1	1	2	1	2	1	2	1	1
3	2	2	4	2	4	2	4	2	2
4	1	1	2	1	2	1	2	1	1
5	1+3	1+3	2+4	1+1	2+3	1+2	2+4	1+2	1+2
6	3	3	4	1	3	2	4	2	2
7	3	3	4	1	3	2	4	2	2

แถวที่ 5: บวกส่วนที่ซ้อนทับ

4	4	6	2	5	3	6	3	3

ผลลัพธ์ชั่วคราว 7 × 9

Transposed Convolutional Layer

Transposed Convolution: Vertical Overlap and Add

Stride = 2

ผลรวมจากแถวที่ 1

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ผลรวมจากแถวที่ 2

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1

ผลรวมจากแถวที่ 3

3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

ผลรวมจากแถวที่ 4

3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

เพิ่มผลรวมจากแถวที่ 4 โดยเลื่อนลง 2 แถว

1	1	1	2	1	2	1	2	1	1
2	1	1	2	1	2	1	2	1	1
3	2	2	4	2	4	2	4	2	2
4	1	1	2	1	2	1	2	1	1
5	4	4	6	2	5	3	6	3	3
6	3	3	4	1	3	2	4	2	2
7	3+3	3+3	4+4	1+1	3+3	2+2	4+4	2+2	2+2
8	3	3	4	1	3	2	4	2	2
9	3	3	4	1	3	2	4	2	2

แถวที่ 7: บวกส่วนที่ซ้อนทับ

6	6	8	2	6	4	8	4	4

ผลลัพธ์ขนาด 9 × 9

Transposed Convolutional Layer

Input X =

1	1	1	1
1	1	1	1
3	1	2	2
3	1	2	2

Kernel K =

1	1	1
1	1	1
1	1	1

Padding = 0 • Stride = 2

Output Y (9 × 9)

1	1	2	1	2	1	2	1	1
1	1	2	1	2	1	2	1	1
2	2	4	2	4	2	4	2	2
1	1	2	1	2	1	2	1	1
4	4	6	2	5	3	6	3	3
3	3	4	1	3	2	4	2	2
6	6	8	2	6	4	8	4	4
3	3	4	1	3	2	4	2	2
3	3	4	1	3	2	4	2	2

ขนาดผลลัพธ์แต่ละด้าน

$$N_{out} = (N_{in} - 1) \times s - 2p + k$$

เมื่อ dilation = 1 และ output padding = 0

ความหมายของตัวแปร

- N_{in} : ขนาดอินพุตแต่ละด้าน
- s : stride
- p : padding
- k : ขนาด kernel แต่ละด้าน
- N_{out} : ขนาดเอาต์พุตแต่ละด้าน

แทนค่า $N_{in} = 4,\ s = 2,\ p = 0,\ k = 3$

$$N_{out} = (4 - 1) \times 2 - 2(0) + 3 = 9$$

ดังนั้น Output มีขนาด 9 × 9

Transposed Convolutional Layer

```python
import torch
import torch.nn as nn
import seaborn as sn
import matplotlib.pyplot as plt

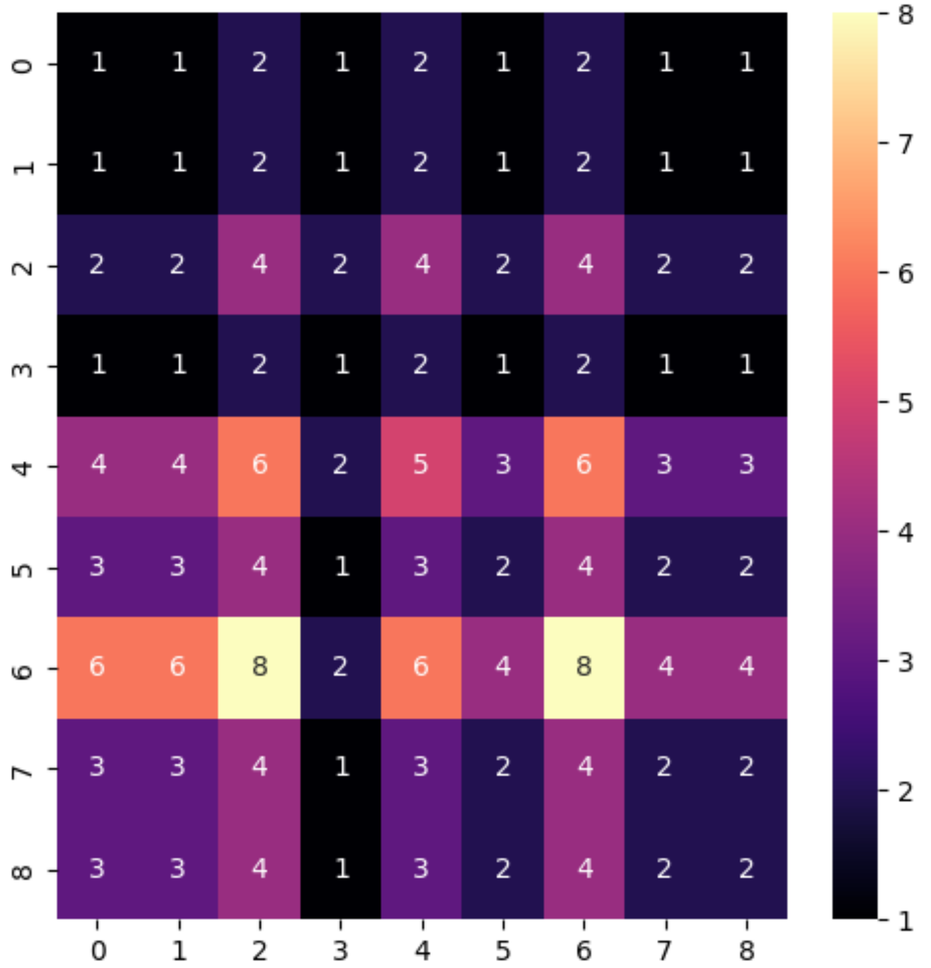
# 1. สร้างอินพุตและเคอร์เนล
X = torch.tensor([[1.0, 1.0, 1.0, 1.0],
                  [1.0, 1.0, 1.0, 1.0],
                  [3.0, 1.0, 2.0, 2.0],
                  [3.0, 1.0, 2.0, 2.0]])          # อินพุตขนาด 4x4 ค่า = 1
X = X.reshape(1,1,4,4)
K = torch.ones((1, 1, 3, 3))                      # kernel ขนาด 3x3 ค่า = 1

# 2. สร้าง Transposed Convolution Layer
tconv = nn.ConvTranspose2d(in_channels=1, out_channels=1, kernel_size=3, stride=2, padding=0, bias=False)

# กำหนดค่า weight = K
tconv.weight.data = K

# 3. คำนวณผลลัพธ์
out = tconv(X)          # ได้ Tensor ขนาด 9x9
out = out.squeeze()     # ตัด batch/channel ออก เหลือ 2D matrix


# 4. วาด heatmap
plt.figure(figsize=(6,6))
sn.heatmap(out.detach().numpy(), annot=True, cmap="magma", cbar=True)
plt.show()
```



หลักการทำงาน Transposed Convolutions

ตัวดำเนินการ Transposed Convolutions ซึ่งเป็นการดำเนินการตรงข้ามกับ Convolution ปกติ กล่าวคือ Transposed Convolution จะพยายามเพิ่มขนาดของภาพเข้าไป (up-sampling) แทนที่จะลดขนาดลง (down-sampling) อย่าง Convolution ทั่วไป การทำงานของ Transposed Convolution นี้มีความสำคัญโดยเฉพาะในการสร้างภาพหรือข้อมูลที่มีความละเอียดสูงขึ้น เช่น ในกระบวนการถอดรหัสใน Autoencoder หรือในการสร้างภาพใหม่ใน Generative Adversarial Networks (GANs) การกำหนดค่า Padding และ Stride ที่แตกต่างกันจะมีผลกระทบต่อขนาดของผลลัพธ์ที่ได้จากการทำ Transposed Convolution

- Padding ทำหน้าที่ในการเพิ่มค่าเทียมรอบ ๆ ข้อมูลเริ่มต้น โดยค่า Padding จะช่วยควบคุมขอบของภาพที่ได้จากการคำนวณคอนโวลูชัน ยิ่งค่า Padding มากขึ้น ขนาดของภาพผลลัพธ์จะยิ่งขยายใหญ่ขึ้น
- Stride เป็นการกำหนดจำนวนก้าวในการเลื่อนตัวกรองไปตามข้อมูล โดย Stride ที่มากขึ้นจะทำให้ผลลัพธ์มีการขยายขนาดมากขึ้นเช่นกัน และอาจมีช่องว่างเพิ่มขึ้นระหว่างพิกเซลของภาพผลลัพธ์ แต่ Stride ที่น้อยจะทำให้การขยายภาพมีความละเอียดสูงกว่า

การกำหนดค่า Padding และ Stride ที่แตกต่างกันส่งผลต่อคุณสมบัติของภาพที่ได้อย่างมาก นักศึกษาจึงต้องเข้าใจว่าค่าต่าง ๆ เหล่านี้สามารถควบคุมขนาดและลักษณะของข้อมูลที่ถูกขยายได้อย่างไร

หมายเหตุ ไม่หมุนตัวกรอง

ตัวอย่างโปรแกรม Transposed Convolutions

```
import torch
from torch import nn
import seaborn as sn

X = torch.tensor([[1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0]])
K = torch.tensor([[1.0, 1.0, 1.0], [1.0, 1.0, 1.0], [1.0, 1.0, 1.0]])

X = X.reshape(1, 1, 4, 4)
K = K.reshape(1, 1, 3, 3)

tconv = nn.ConvTranspose2d(1, 1, kernel_size=3, padding=2, stride=2, bias=False)
tconv.weight.data = K

out1 = tconv(X)
out1 = out1.squeeze()

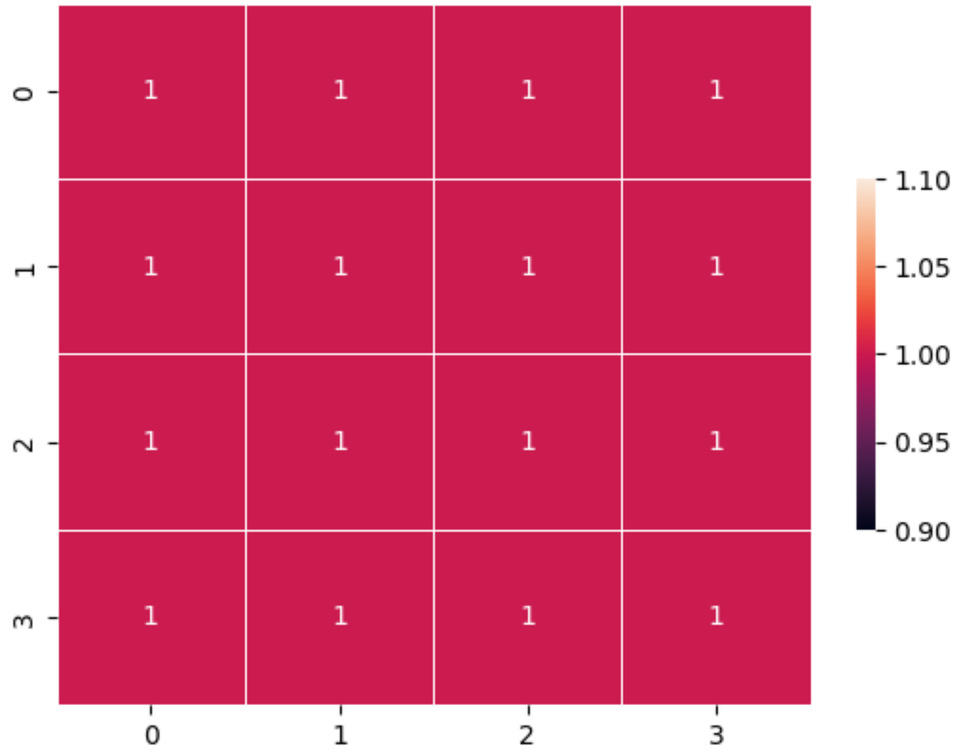
sn.heatmap(out1.detach().numpy(), annot=True)
```

ตัวอย่างการสร้างข้อมูลและตัวกรอง

```
X = torch.tensor([[1.0, 1.0, 1.0, 1.0],      // สร้างข้อมูล
                  [1.0, 1.0, 1.0, 1.0],
                  [1.0, 1.0, 1.0, 1.0],
                  [1.0, 1.0, 1.0, 1.0]])

sn.heatmap(X.detach().numpy(), annot=True, linewidths=.5, cbar_kws={"shrink": .5})
```

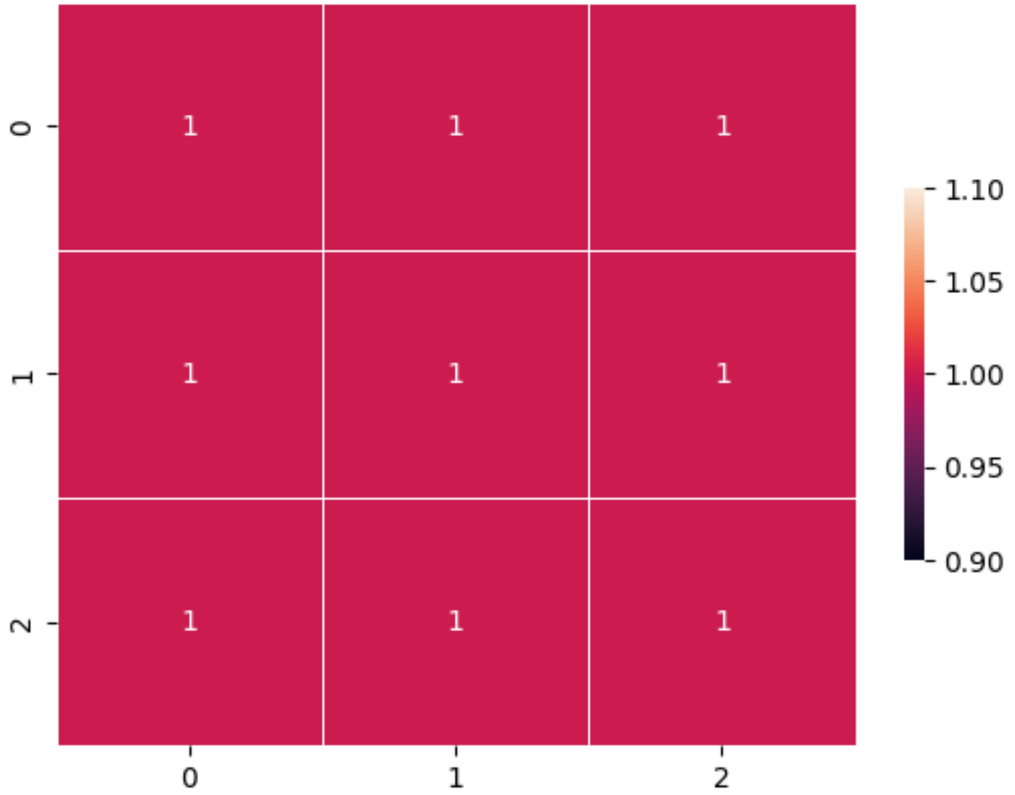
<Axes: >




```
[36] K = torch.tensor([[1.0, 1.0, 1.0],    // สร้าง Filter
                       [1.0, 1.0, 1.0],
                       [1.0, 1.0, 1.0]])

     sn.heatmap(K.detach().numpy(), annot=True, linewidths=.5, cbar_kws={"shrink": .5})
```

<Axes: >



ตัวอย่างการทำงาน Transposed Convolutions

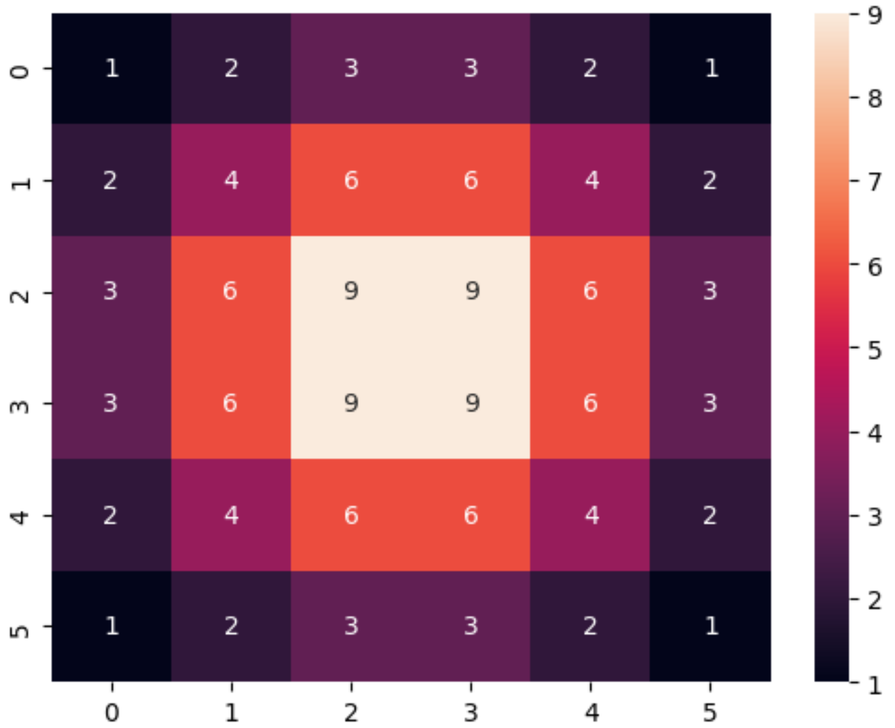
```
X = X.reshape(1, 1, 4, 4)
K = K.reshape(1, 1, 3, 3)

tconv = nn.ConvTranspose2d(1, 1, kernel_size=3, padding=0, stride=1, bias=False)
tconv.weight.data = K

out1 = tconv(X)
out1 = out1.squeeze()

sn.heatmap(out1.detach().numpy(), annot=True)
```

```
<Axes: >
```



Padding = 0, Stride = 1

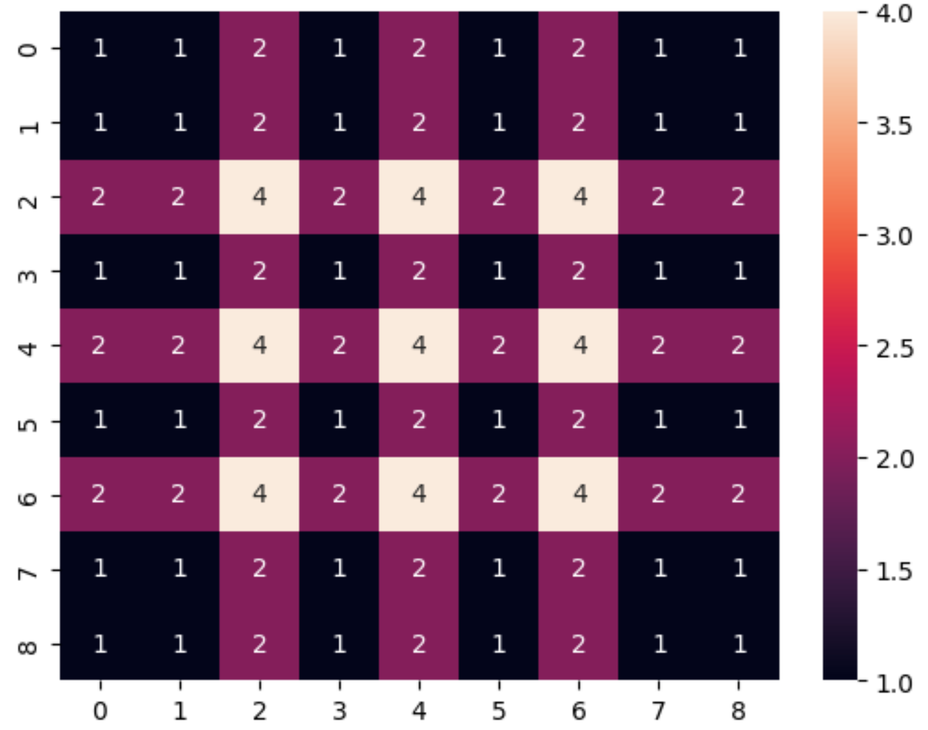
```
X = X.reshape(1, 1, 4, 4)
K = K.reshape(1, 1, 3, 3)

tconv = nn.ConvTranspose2d(1, 1, kernel_size=3, padding=0, stride=2, bias=False)
tconv.weight.data = K

out1 = tconv(X)
out1 = out1.squeeze()

sn.heatmap(out1.detach().numpy(), annot=True)
```

```
<Axes: >
```



Padding = 0, Stride = 2

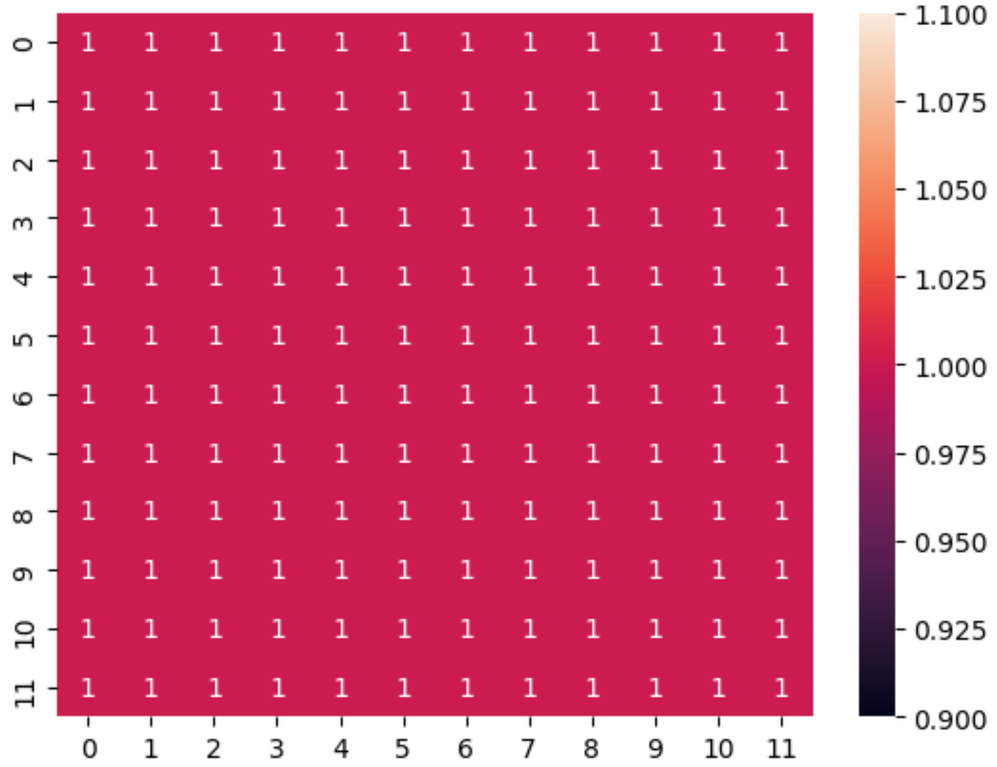
ตัวอย่างการทำงาน Transposed Convolutions

```
tconv = nn.ConvTranspose2d(1, 1, kernel_size=3, padding=0, stride=3, bias=False)
tconv.weight.data = K

out1 = tconv(X)
out1 = out1.squeeze()

sn.heatmap(out1.detach().numpy(), annot=True)
```

```
<Axes: >
```



Padding = 0, Stride = 3

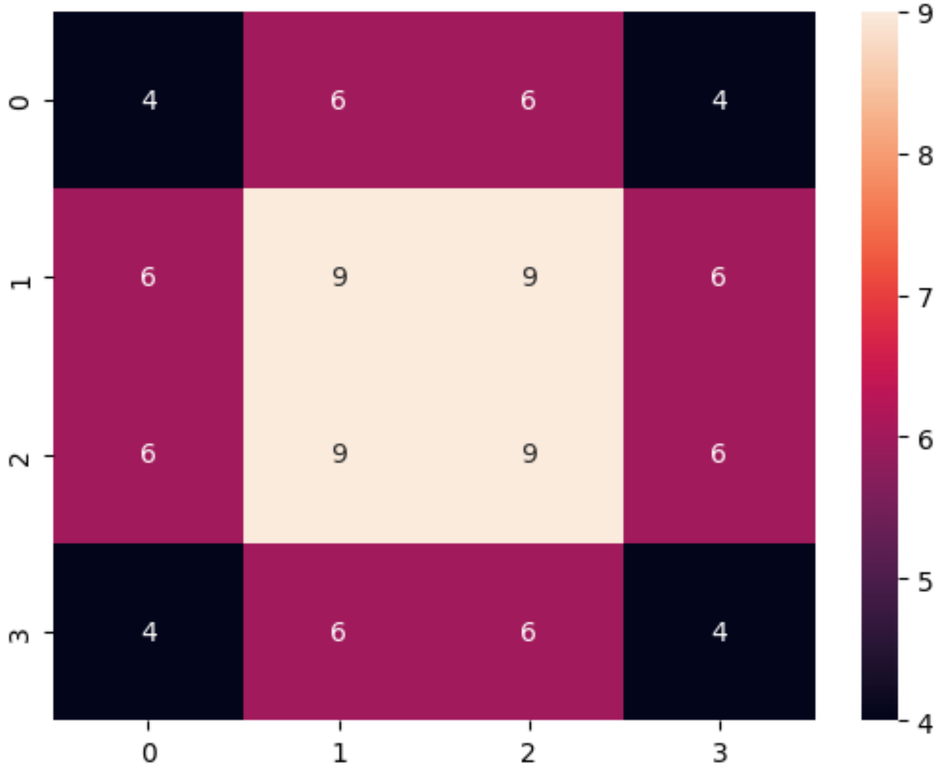
ตัวอย่างการทำงาน Transposed Convolutions

```
tconv = nn.ConvTranspose2d(1, 1, kernel_size=3, padding=1, stride=1, bias=False)
tconv.weight.data = K

out1 = tconv(X)
out1 = out1.squeeze()

sn.heatmap(out1.detach().numpy(), annot=True)
```

```
<Axes: >
```



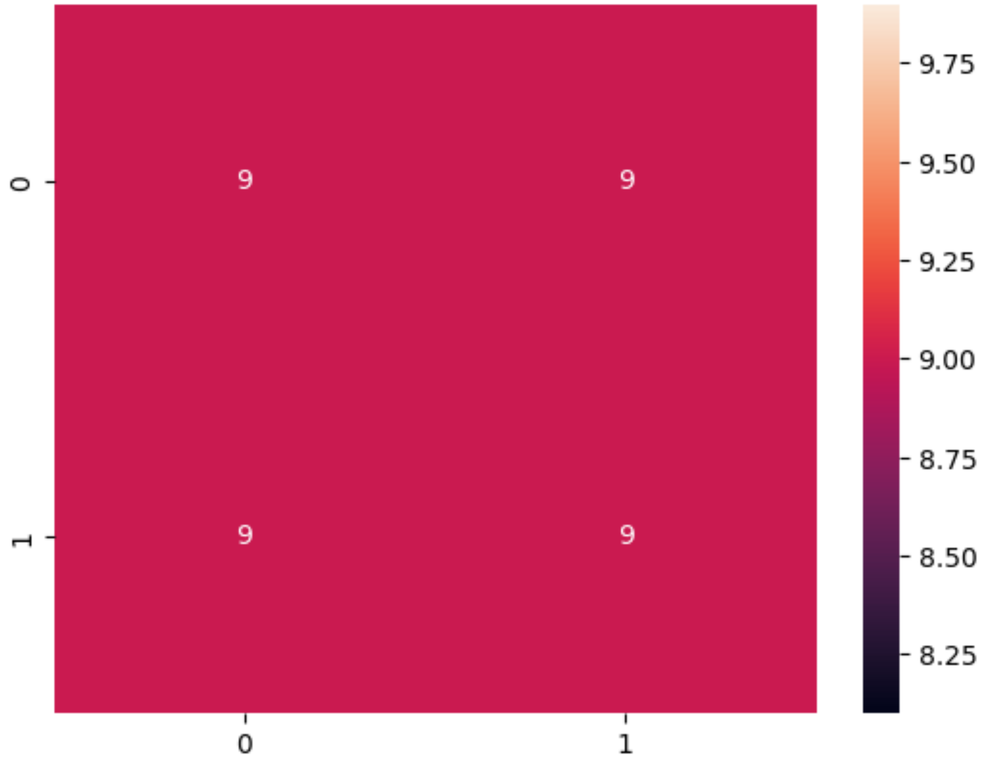
Padding = 1, Stride = 1

```
tconv = nn.ConvTranspose2d(1, 1, kernel_size=3, padding=2, stride=1, bias=False)
tconv.weight.data = K

out1 = tconv(X)
out1 = out1.squeeze()

sn.heatmap(out1.detach().numpy(), annot=True)
```

```
<Axes: >
```



Padding = 2, Stride = 1

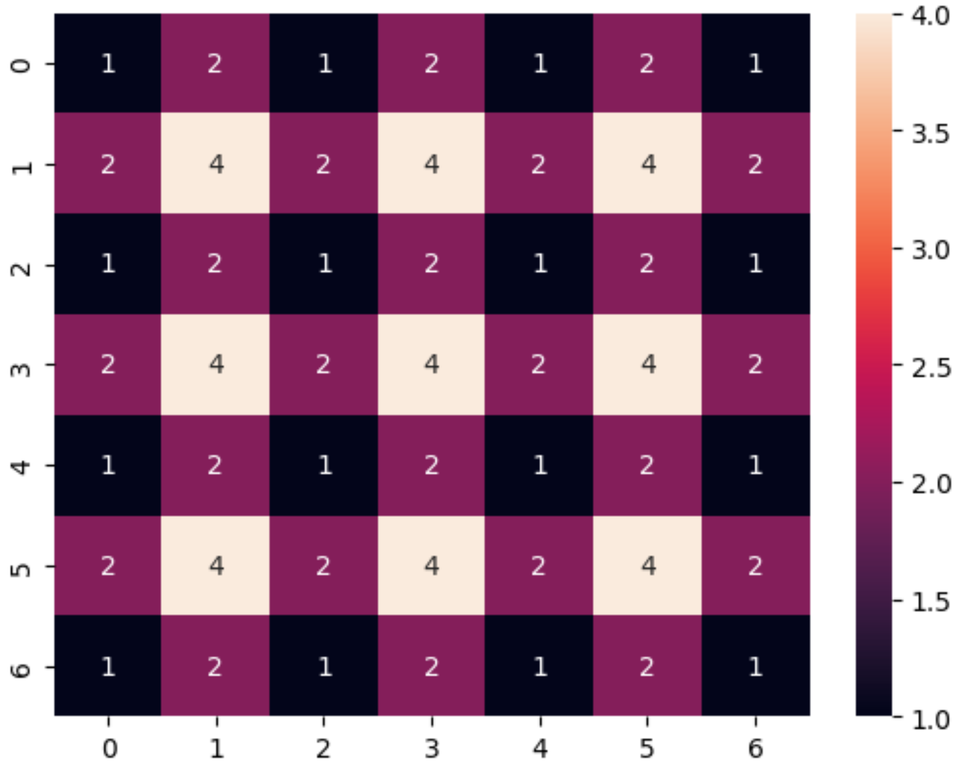
ตัวอย่างการทำงาน Transposed Convolutions

```
tconv = nn.ConvTranspose2d(1, 1, kernel_size=3, padding=1, stride=2, bias=False)
tconv.weight.data = K

out1 = tconv(X)
out1 = out1.squeeze()

sn.heatmap(out1.detach().numpy(), annot=True)
```

```
<Axes: >
```



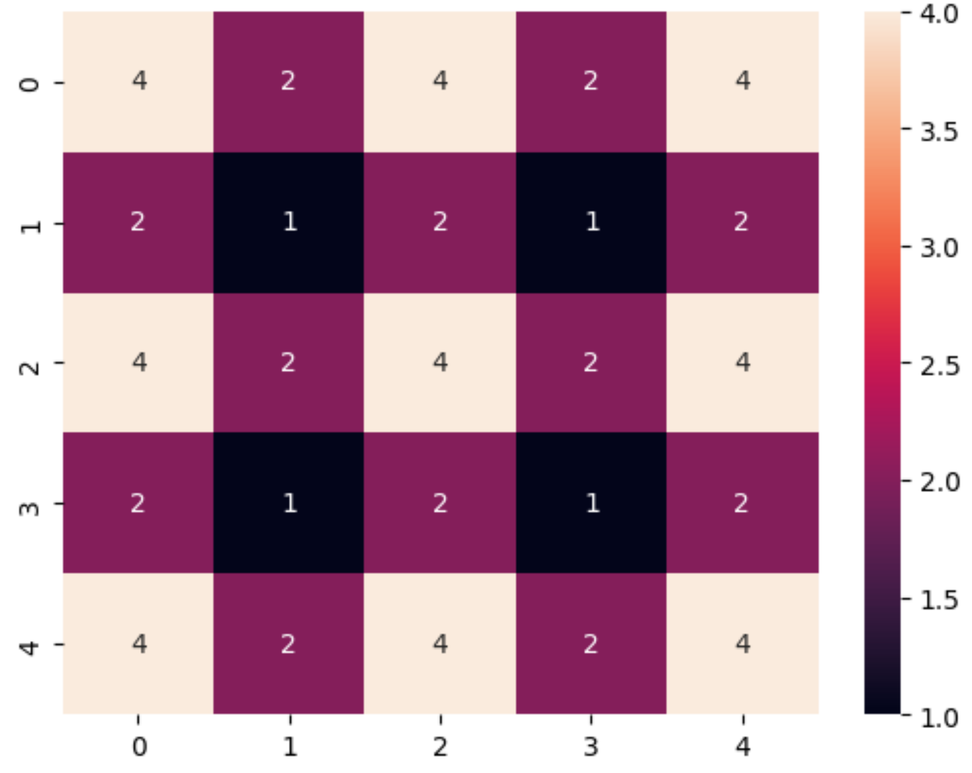
Padding = 1, Stride = 2

```
tconv = nn.ConvTranspose2d(1, 1, kernel_size=3, padding=2, stride=2, bias=False)
tconv.weight.data = K

out1 = tconv(X)
out1 = out1.squeeze()

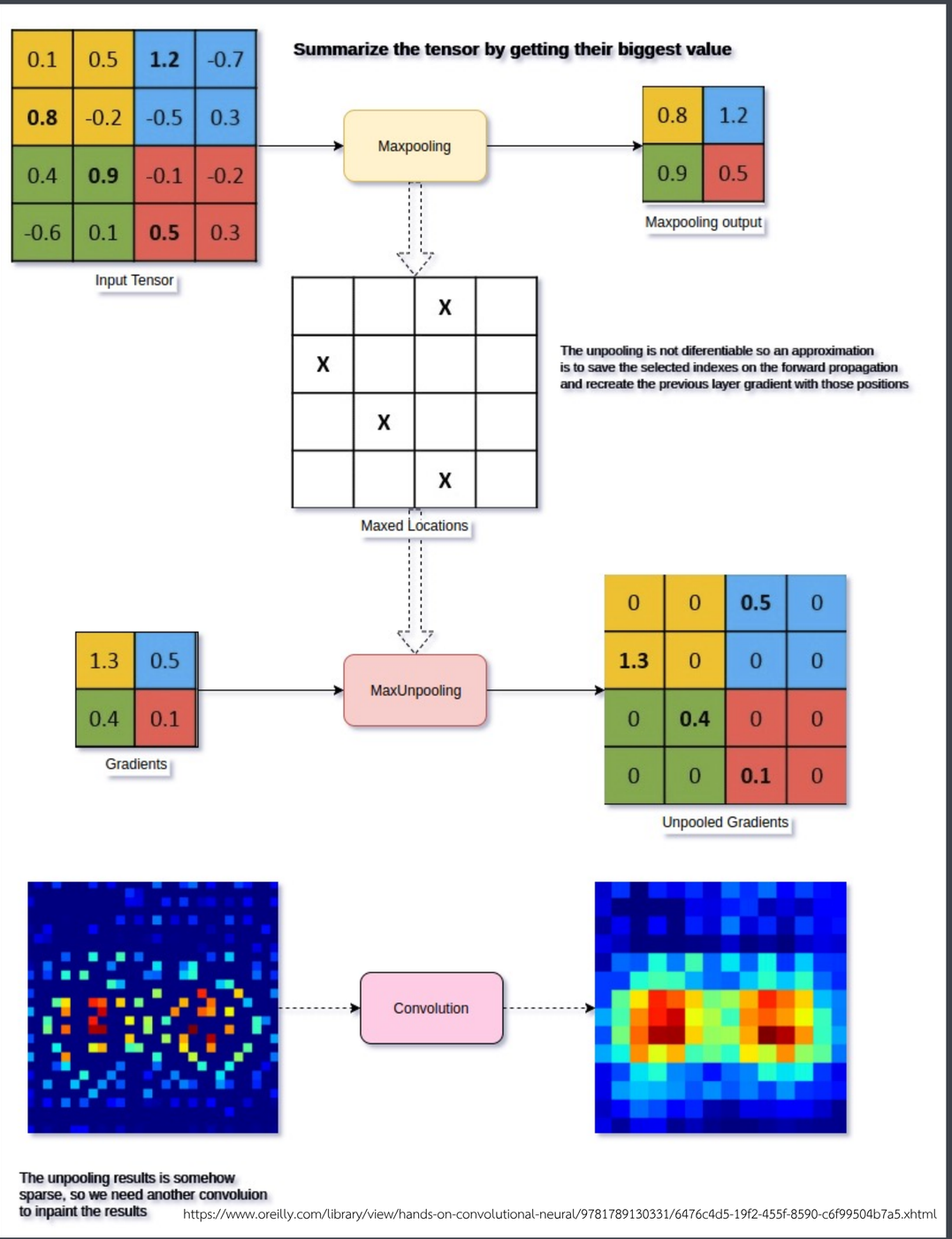
sn.heatmap(out1.detach().numpy(), annot=True)
```

```
<Axes: >
```



Padding = 2, Stride = 2

การยกเลิกการพูลลิ่ง (Unpooling)

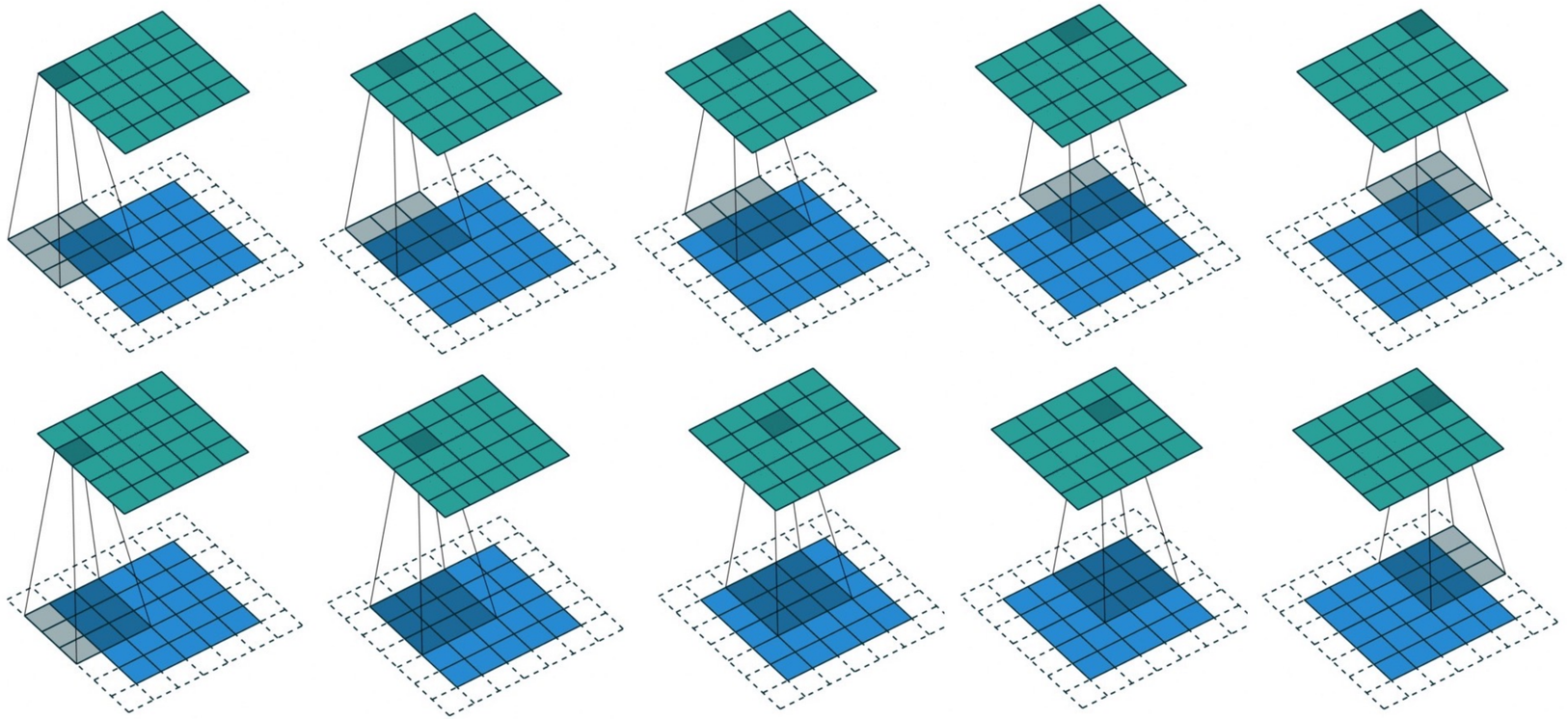


Max Unpooling คือกระบวนการ "ย้อนกลับ" การทำ Max Pooling โดยอาศัยข้อมูลตำแหน่ง (Pooling Indices หรือ Mask) ที่จดไว้ตั้งแต่ตอนทำ Max Pooling ว่าค่าที่ถูกเลือก (maximum) มาจากตำแหน่งใด เมื่อทำ Max Unpooling ค่าดังกล่าวจะถูกนำกลับไปใส่ในตำแหน่งเดิม ส่วนตำแหน่งอื่น ๆ จะถูกเติมด้วยค่า 0

แนวคิดหลักของ Max Unpooling คือการทำหน้าที่เป็น Upsampler เพื่อกู้คืนโครงสร้างเชิงพื้นที่ของข้อมูลที่ถูกย่อขนาดลงด้วย Max Pooling ให้กลับมามีขนาดใหญ่ขึ้น อย่างไรก็ตาม วิธีการนี้มักไม่ถูกใช้งานมากนักในปัจจุบัน แม้จะเคยมีงานวิจัยบางส่วนเลือกใช้ในอดีต เหตุผลก็คือ แม้จะได้ข้อมูลที่ถูกขยายขนาดกลับมา แต่ยังคงจำเป็นต้องใช้เลเยอร์ Convolution ตามมาช่วย "สร้าง" รายละเอียดของผลลัพธ์ให้ครบถ้วนและใกล้เคียงกับข้อมูลต้นฉบับ

เทคนิค Padding

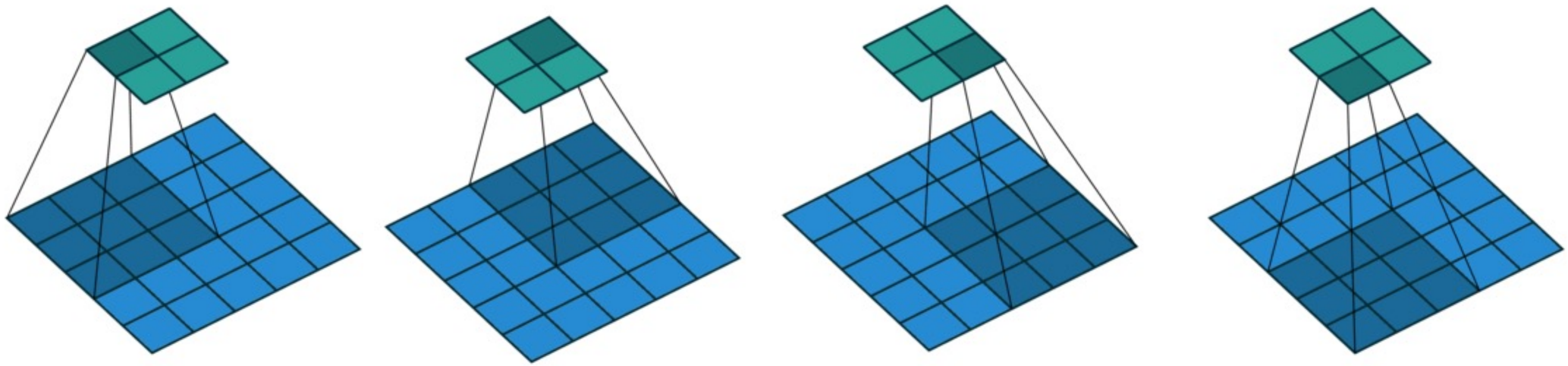
เทคนิค Padding ได้รับความนิยมนำมาใช้เพื่อเพิ่มมิติของ Input เพื่อไม่ให้มิติของ Output จาก Convolution ลดลง โดยปกติบริเวณขอบของ Input จะเติมข้อมูลด้วย 0 ซึ่งจะไม่ส่งผลต่อ Dot Product



อ้างอิง https://medium.com/@marsxiang/convolutions-transposed-and-deconvolution-6430c358a5b6

เทคนิค Stride

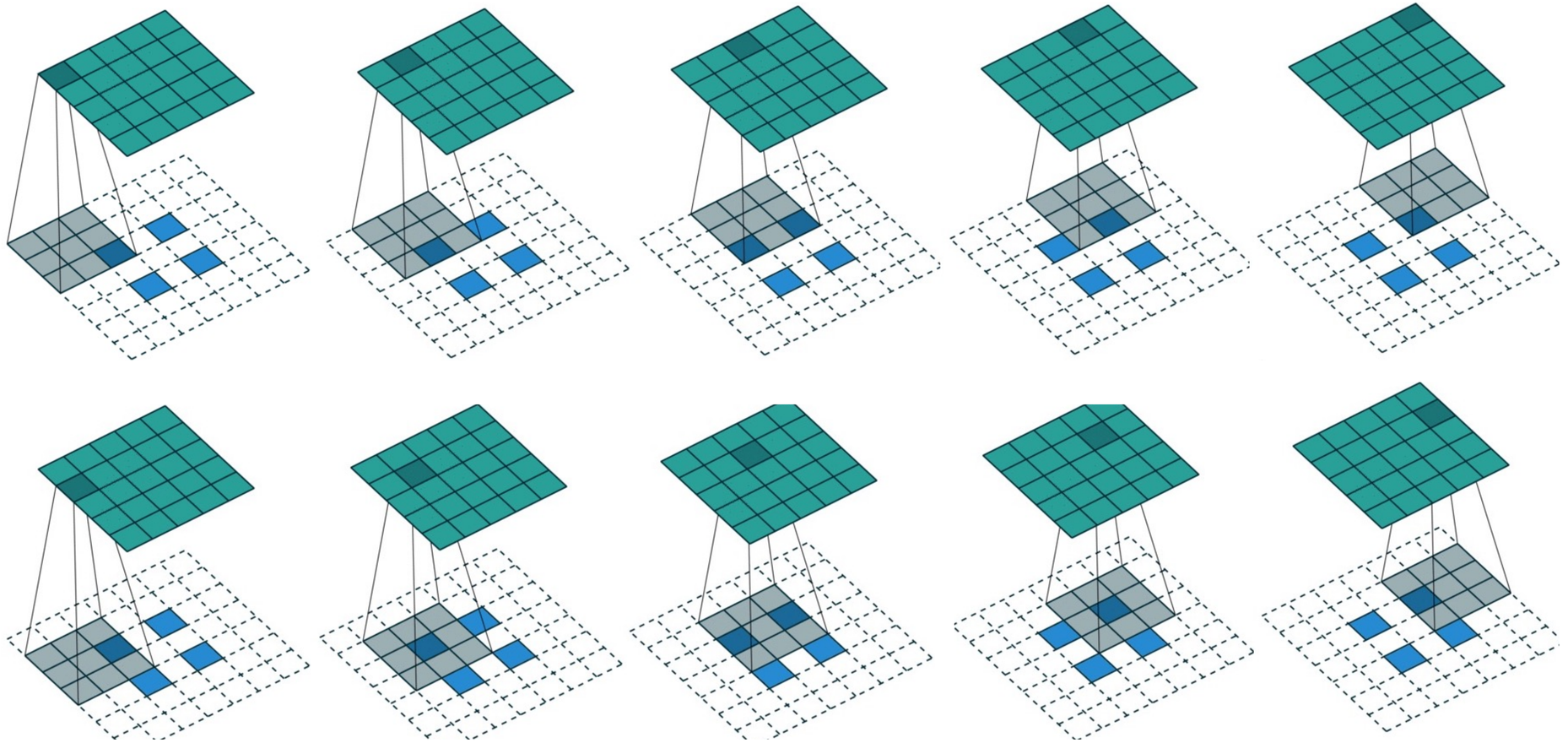
เทคนิค Stride คือการกำหนดขนาดความกว้างในการขยับในทุกครั้ง โดยที่ Stride ยิ่งมีค่าสูงทำให้การบีบอัดผลลัพธ์ (Output) เพิ่มมากขึ้น โดยส่วนใหญ่จะกำหนดค่า Stride เท่ากับหนึ่ง



อ้างอิง https://medium.com/@marsxiang/convolutions-transposed-and-deconvolution-6430c358a5b6

เทคนิค Stride (Cont.)

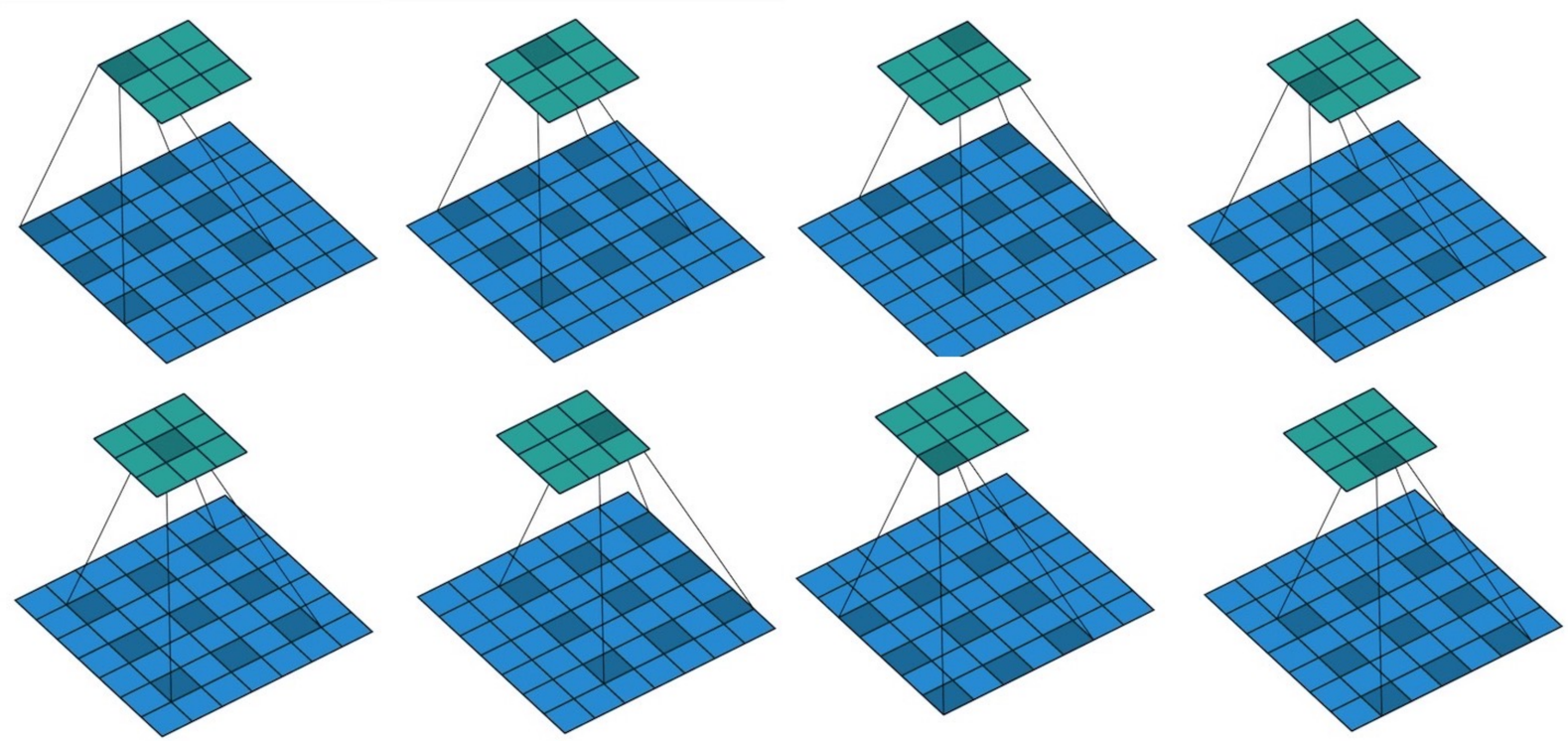
ถ้ากำหนด Strides แบบ 2 x 2 จะทำให้เพิ่มขนาดของ Input จาก 3 x 3 เป็น Output ขนาด 5 x 5



อ้างอิง https://medium.com/@marsxiang/convolutions-transposed-and-deconvolution-6430c358a5b6

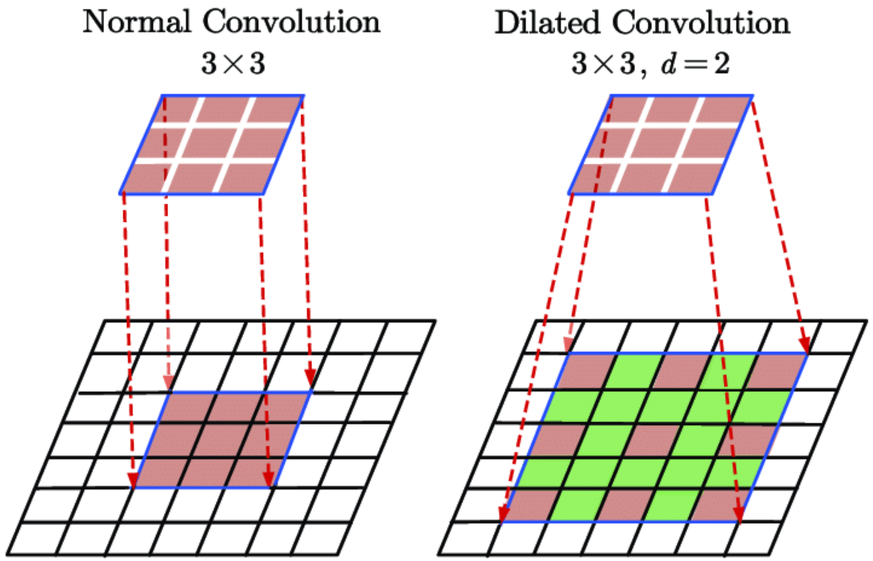
เทคนิค Dilation

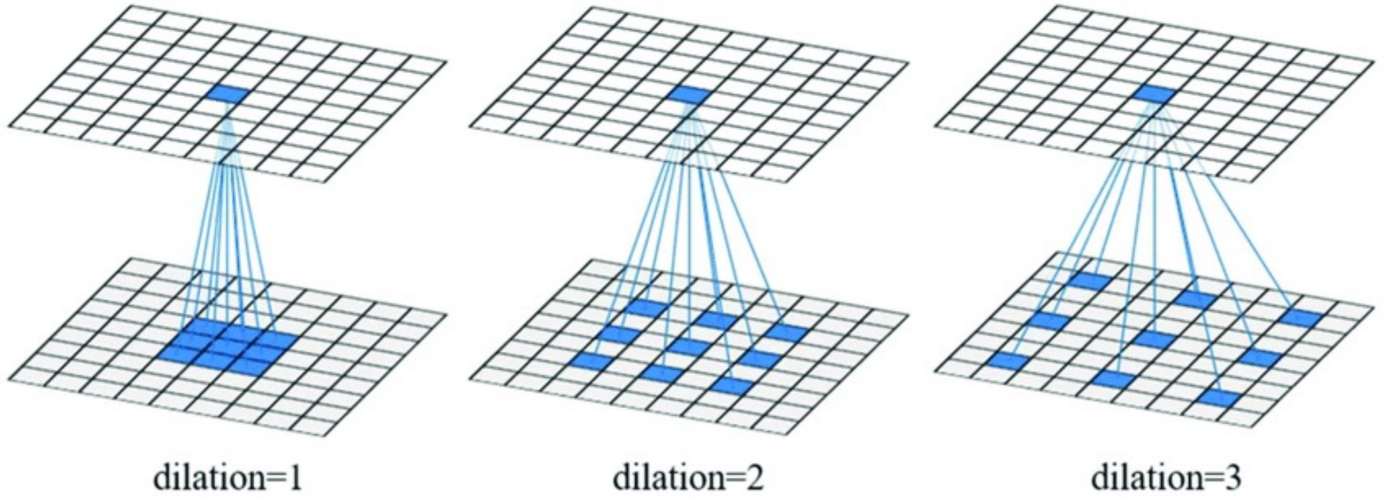
เทคนิค Dilations ใช้คำนวณและควบคุมขนาดของ Output แต่จุดประสงค์หลักเพื่อขยายขอบเขตของ Kernel สามารถมองเห็นเพื่อจับรายละเอียดของภาพในรูปแบบที่ใหญ่ขึ้นได้



อ้างอิง https://medium.com/@marsxiang/convolutions-transposed-and-deconvolution-6430c358a5b6

เทคนิค Dilation



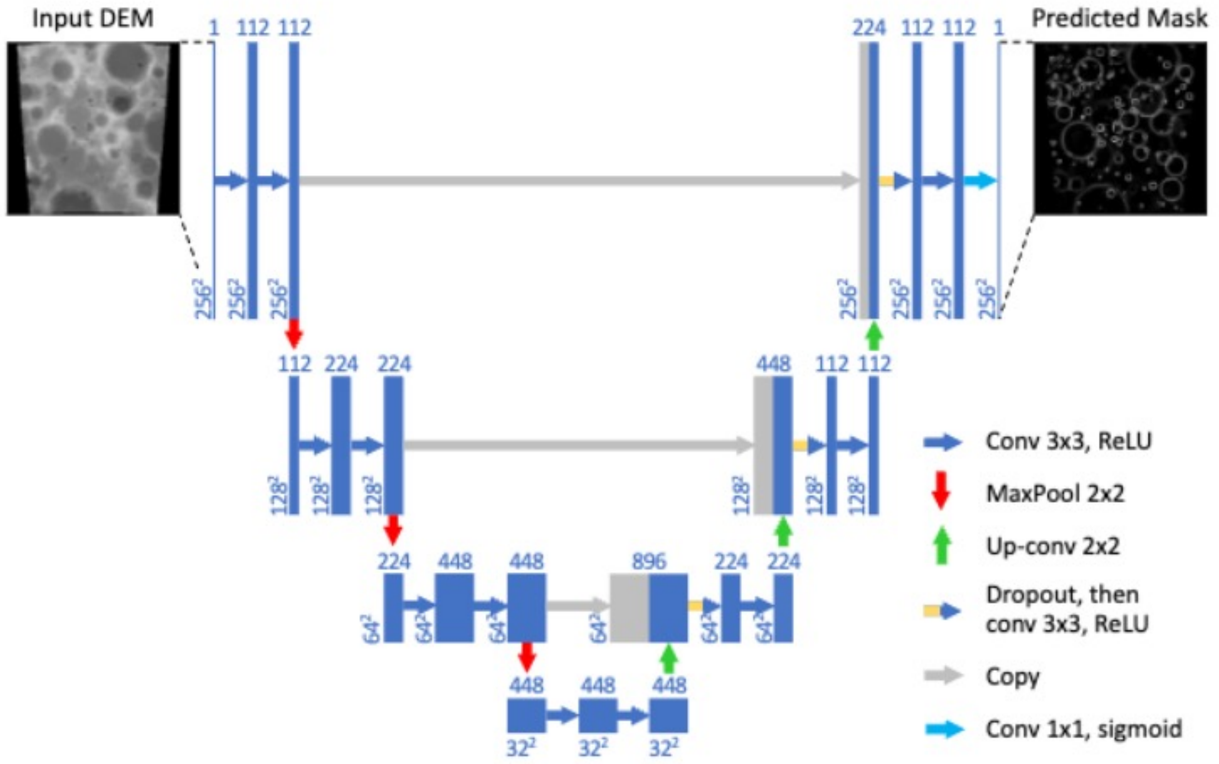


https://www.researchgate.net/publication/338780378/ และ https://www.researchgate.net/publication/336002670

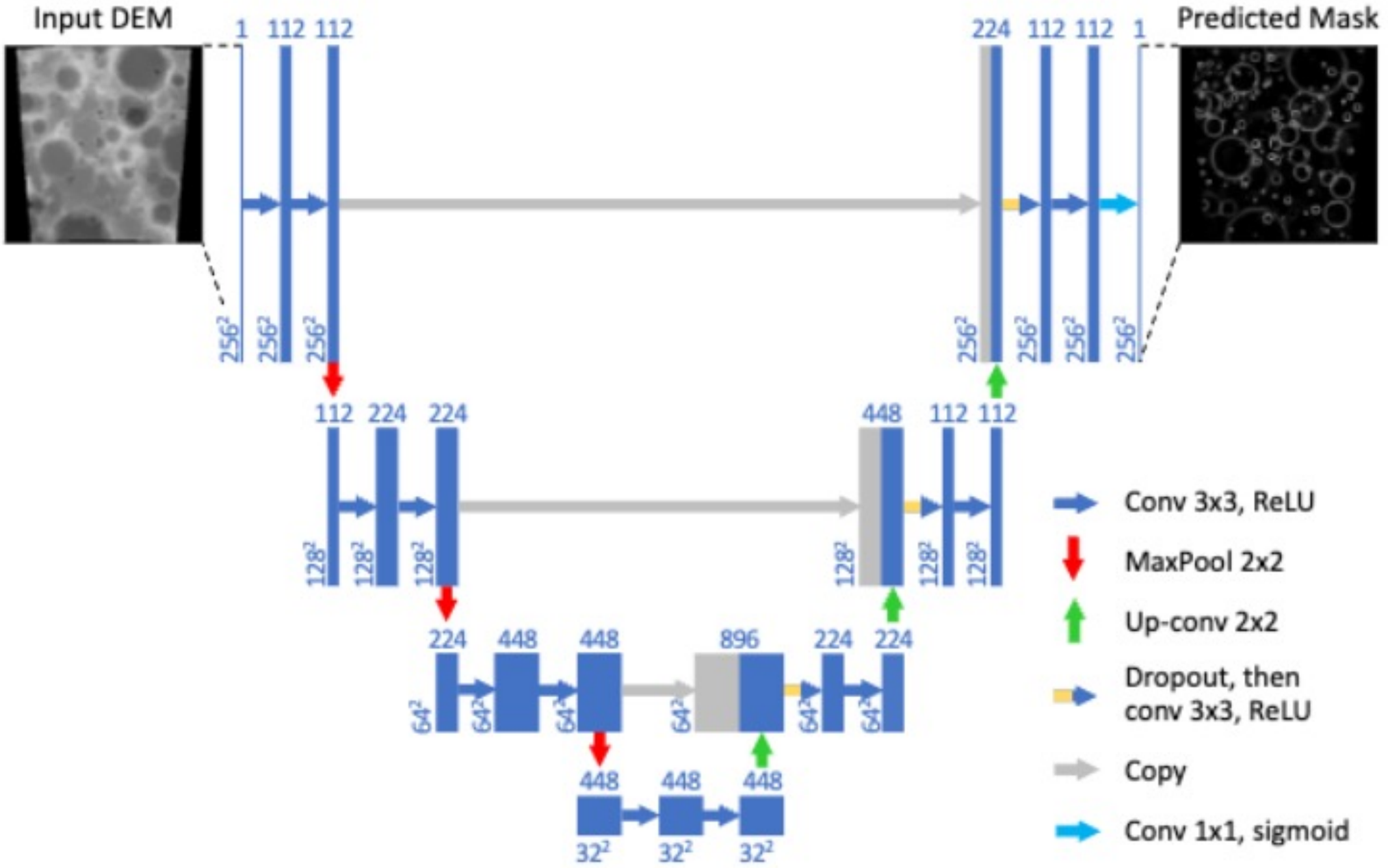
สถาปัตยกรรม U-Net

Encoder ประกอบด้วยชุดของเลเยอร์การคอนโวลูชัน (Convolution) และการพูลลิ่ง (Pooling) ที่ค่อย ๆ ลดขนาดของภาพอินพุตเพื่อดึงลักษณะเฉพาะในหลายสเกล (Scale) ออกมา ใน Encoder ขนาด (Width และ Height) ของภาพจะค่อย ๆ ลดลง ในขณะที่ ความลึก (Dept หรือ Channel) ค่อย ๆ เพิ่มขึ้น ซึ่งหมายความว่าเครือข่ายจะเรียนรู้ข้อมูล "WHAT" ในภาพ แต่จะสูญเสียข้อมูล "WHERE" ไป

Decoder ประกอบด้วยชุดของเลเยอร์การคอนโวลูชันและการอัพแซมปลิง (Upsampling) ที่ขยายแผนที่ลักษณะเฉพาะไปยังขนาดของภาพอินพุตเดิม พร้อมทั้งรวมลักษณะเฉพาะที่มีความละเอียดสูง (Copy and Crop) จาก Encoder สิ่งนี้ทำให้ Decoder สามารถสร้างมาสก์ (Mark) การแบ่งส่วนที่มีขนาดเดียวกับภาพอินพุตเดิม ใน Decoder ขนาดของภาพ (Width และ Height) จะค่อย ๆ เพิ่มขึ้นในขณะที่ความลึกค่อย ๆ ลดลง ซึ่งหมายความว่าเครือข่ายจะเรียนรู้ข้อมูล "WHERE" ในภาพ โดยค่อย ๆ ใช้การอัพแซมปลิง



อ้างอิง https://doi.org/10.5281/zenodo.1133969.

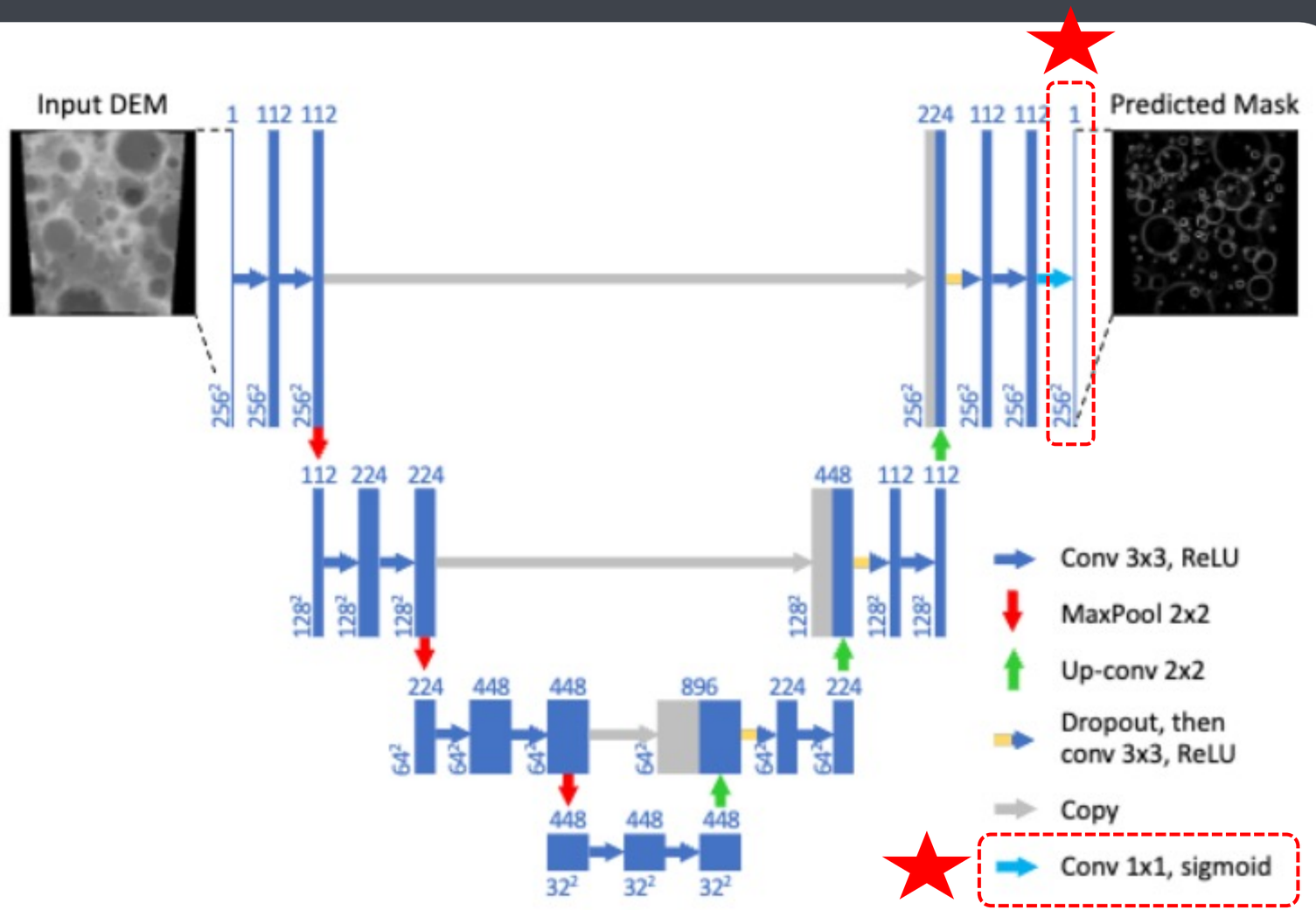

Input DEM
Predicted Mask
1
112
112
224
112
112
1
256²
256²
256²
256²
256²
256²
256²
256²
112
224
224
448
112
112
128²
128²
128²
128²
128²
128²
224
448
448
896
224
224
64²
64²
64²
64²
64²
64²
448
448
448
32²
32²
32²
Conv 3x3, ReLU
MaxPool 2x2
Up-conv 2x2
Dropout, then
conv 3x3, ReLU
Copy
Conv 1x1, sigmoid

สถาปัตยกรรม U-Net สำหรับ Binary

จากรูปเป็นสถาปัตยกรรมที่แบ่งแยกเพียง 2 คลาสแบบ 1 พิกเซลเป็นได้เพียง 1 คลาส ในกรณีนี้ Final Layer สามารถสร้างได้ 2 รูปแบบ

- **แบบที่ 1** ตัวกรอง ณ Final Layer จำนวน 1 ตัวและกำหนด Activation Function เป็นแบบ Sigmoid***

- **แบบที่ 2** ตัวกรอง ณ Final Layer จำนวน 2 ตัวและกำหนด Activation Function เป็นแบบ Softmax



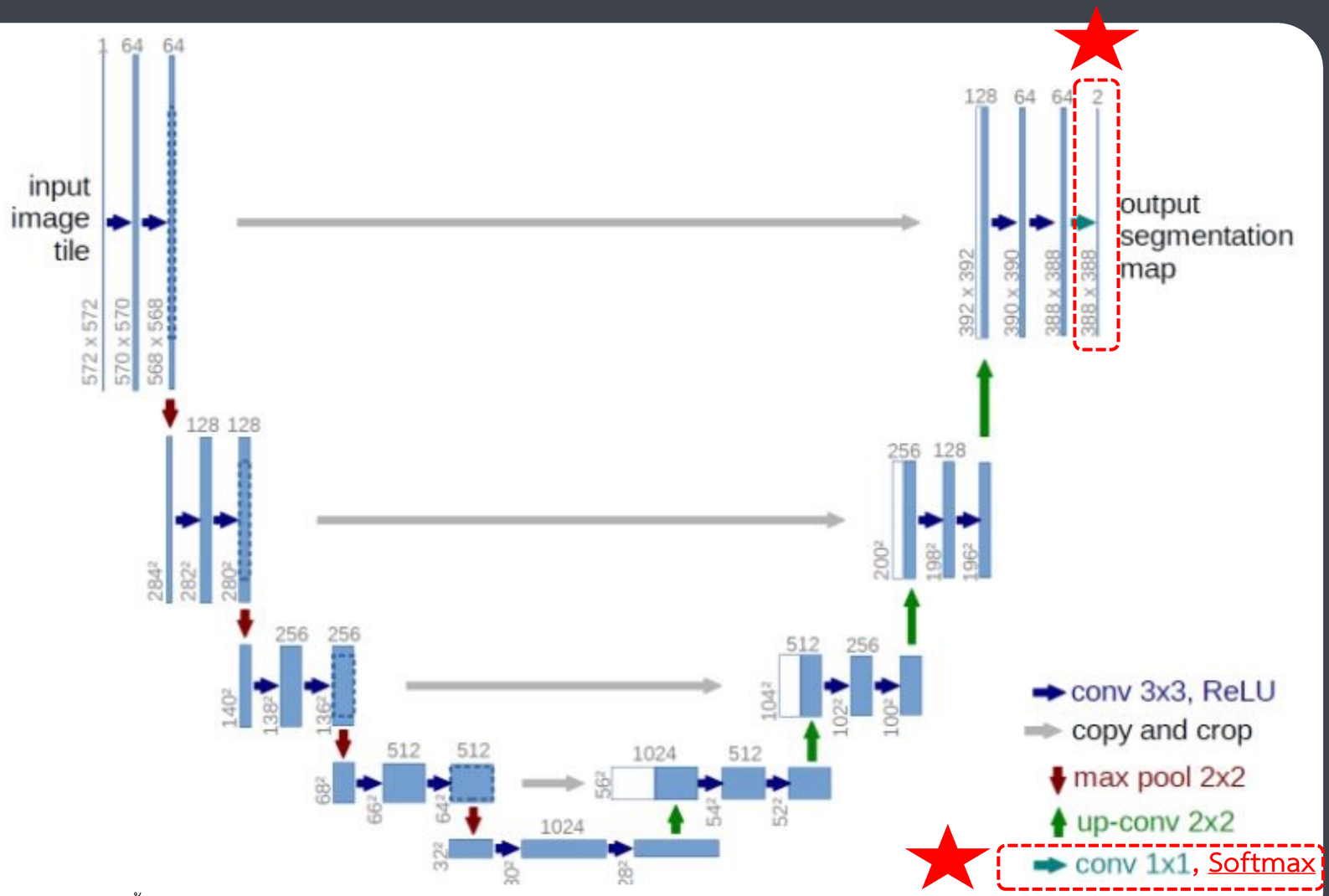
อ้างอิง https://doi.org/10.5281/zenodo.1133969.

สถาปัตยกรรม U-Net สำหรับ Binary

จากรูปเป็นสถาปัตยกรรมที่แบ่งแยกเพียง 2 คลาสแบบ 1 พิกเซลเป็นได้เพียง 1 คลาส ในกรณีนี้ Final Layer สามารถสร้างได้ 2 รูปแบบ

- **แบบที่ 1** ตัวกรอง ณ Final Layer จำนวน 1 ตัวและกำหนด Activation Function เป็นแบบ Sigmoid

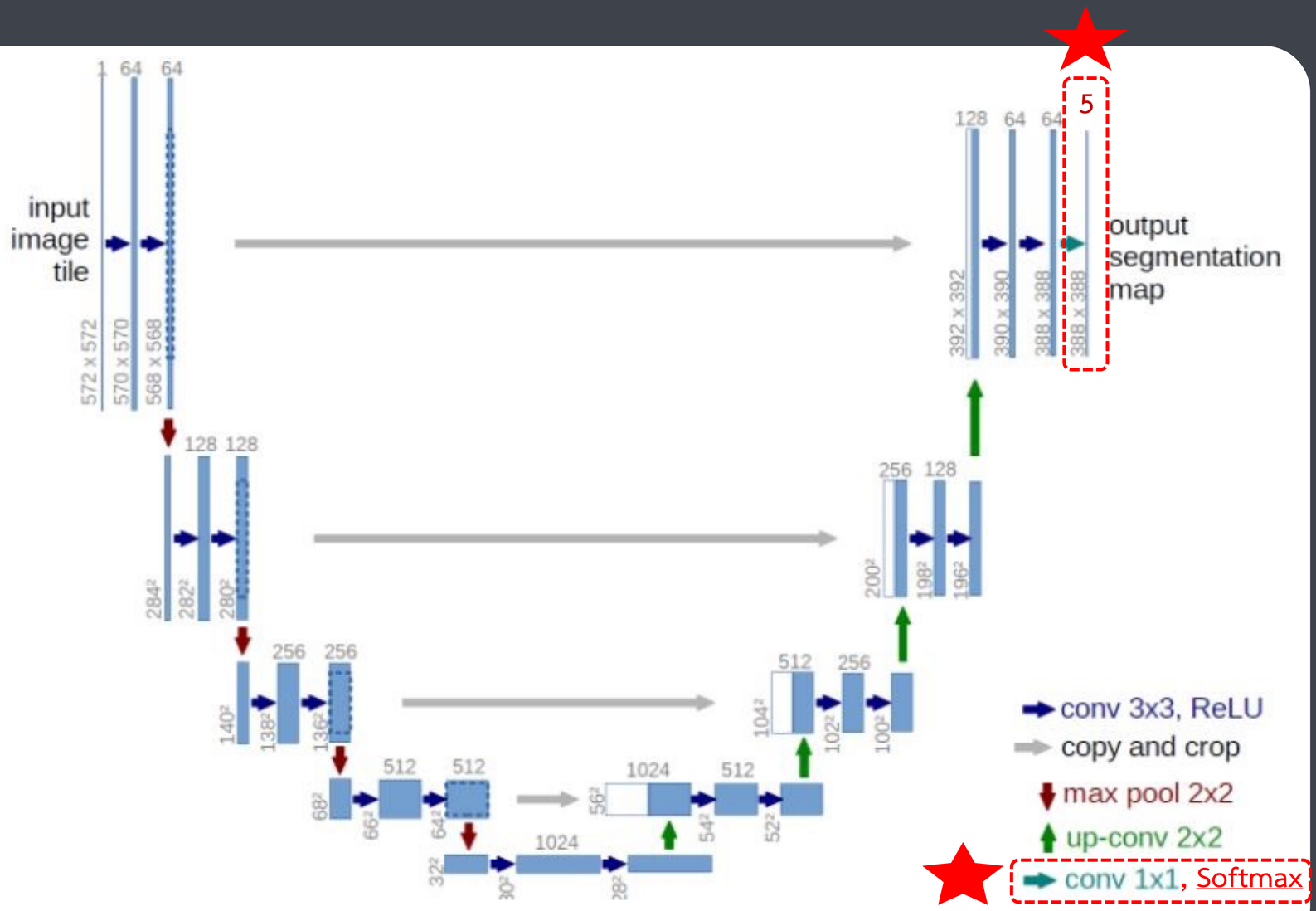
- **แบบที่ 2** ตัวกรอง ณ Final Layer จำนวน 2 ตัวและกำหนด Activation Function เป็นแบบ Softmax***



อ้างอิง https://www.analyticsvidhya.com/blog/2021/01/auto-encoders-for-computer-vision-an-endless-world-of-possibilities/

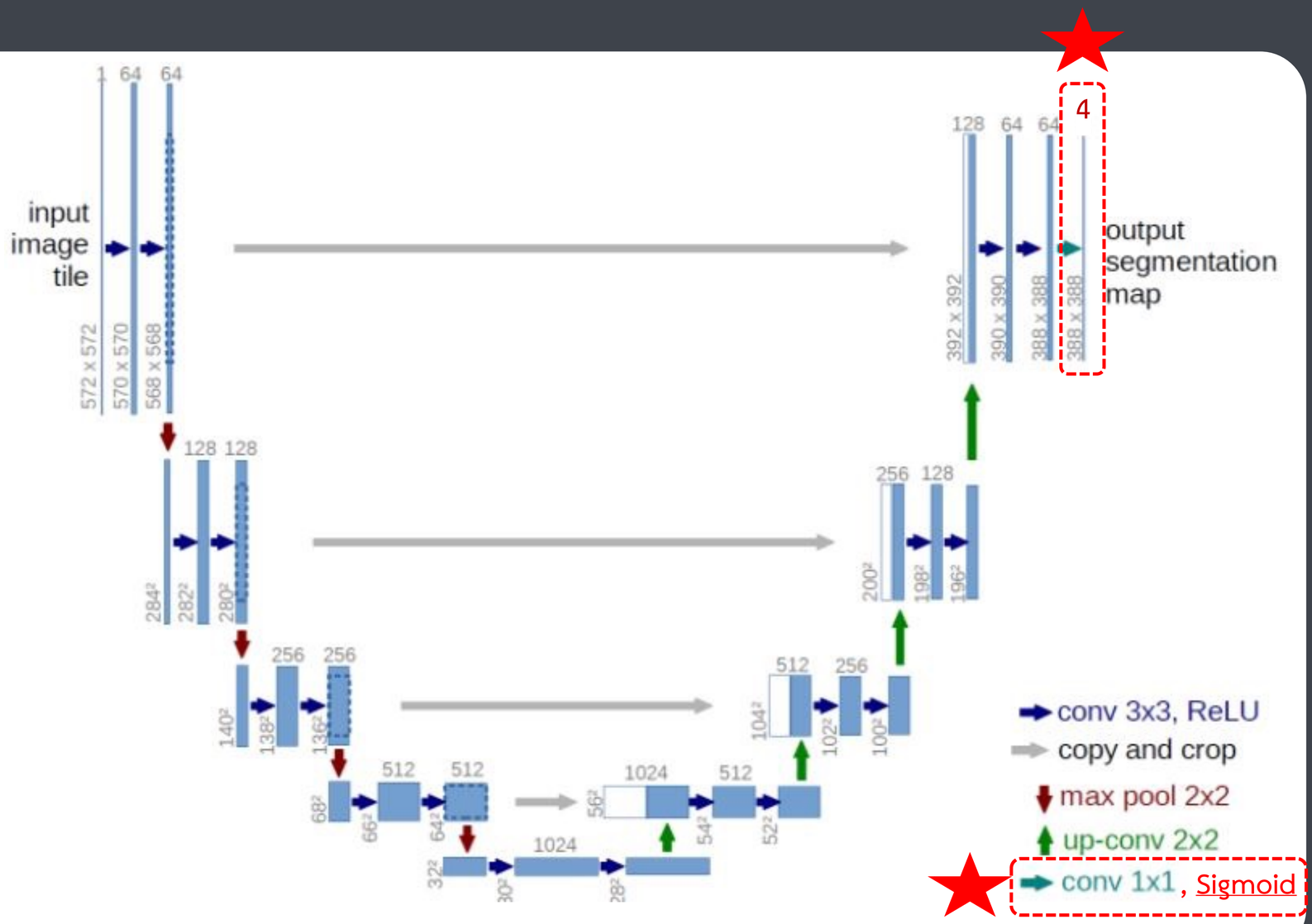
สถาปัตยกรรม U-Net สำหรับ Multiclass

แต่ถ้าเป็นสถาปัตยกรรมที่แบ่งแยกจำนวน 5 คลาสแบบ 1 พิกเซลเป็นได้เพียง 1 คลาส (Multiclassification) ดังนั้น Final Layer จะมี Filter จำนวน 5 ตัวและ Activation Function เป็นแบบ Softmax



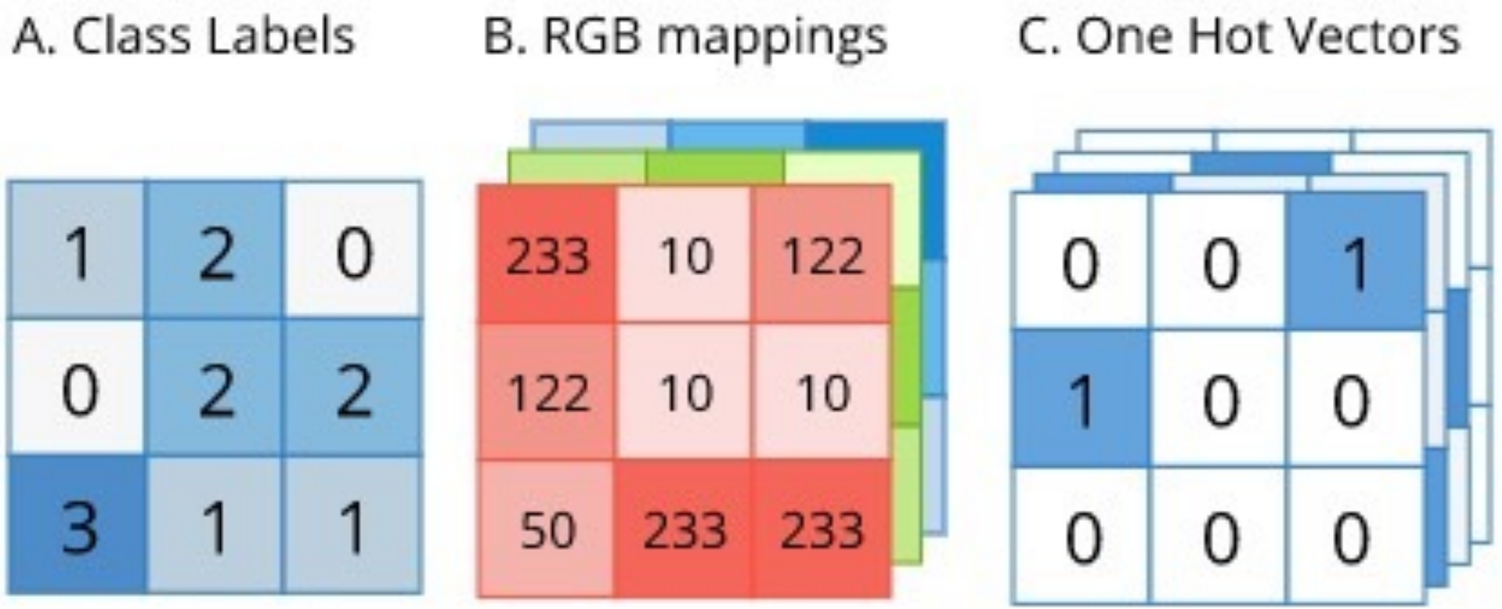
สถาปัตยกรรม U-Net สำหรับ Multilabel

แต่ถ้าเป็นสถาปัตยกรรมที่แบ่งแยกจำนวน 4 คลาสแบบ 1 พิกเซลเป็นได้มากกว่า 1 คลาส (Multilabel) ดังนั้น Final Layer จะมี Filter จำนวน 4 ตัว และ Activation Function เป็นแบบ Sigmoid



Inputs, Labels และ Outputs

ข้อมูลป้ายกำกับสำหรับการฝึกอบรมอาจอยู่ในรูปแบบที่แตกต่างกันหลายรูปแบบ ขึ้นอยู่กับชุดข้อมูลที่คุณดาวน์โหลด



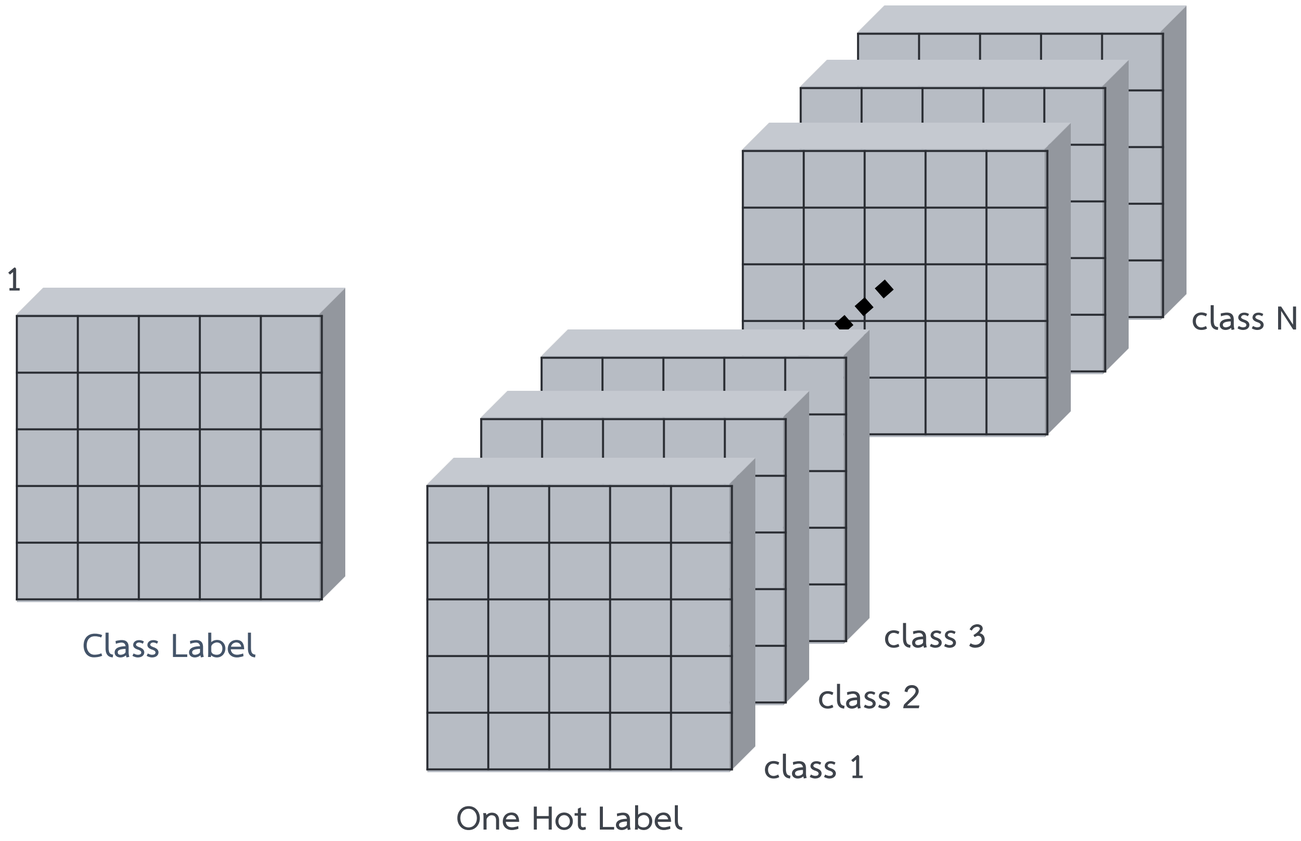
Softmax หรือ Linear
สำหรับงานทางด้าน Multiclass

Sigmoid
สำหรับงานทางด้าน Multilabel

อ้างอิง http://ronny.rest/tutorials/module/seg_01/segmentation_03_inputs_outputs/

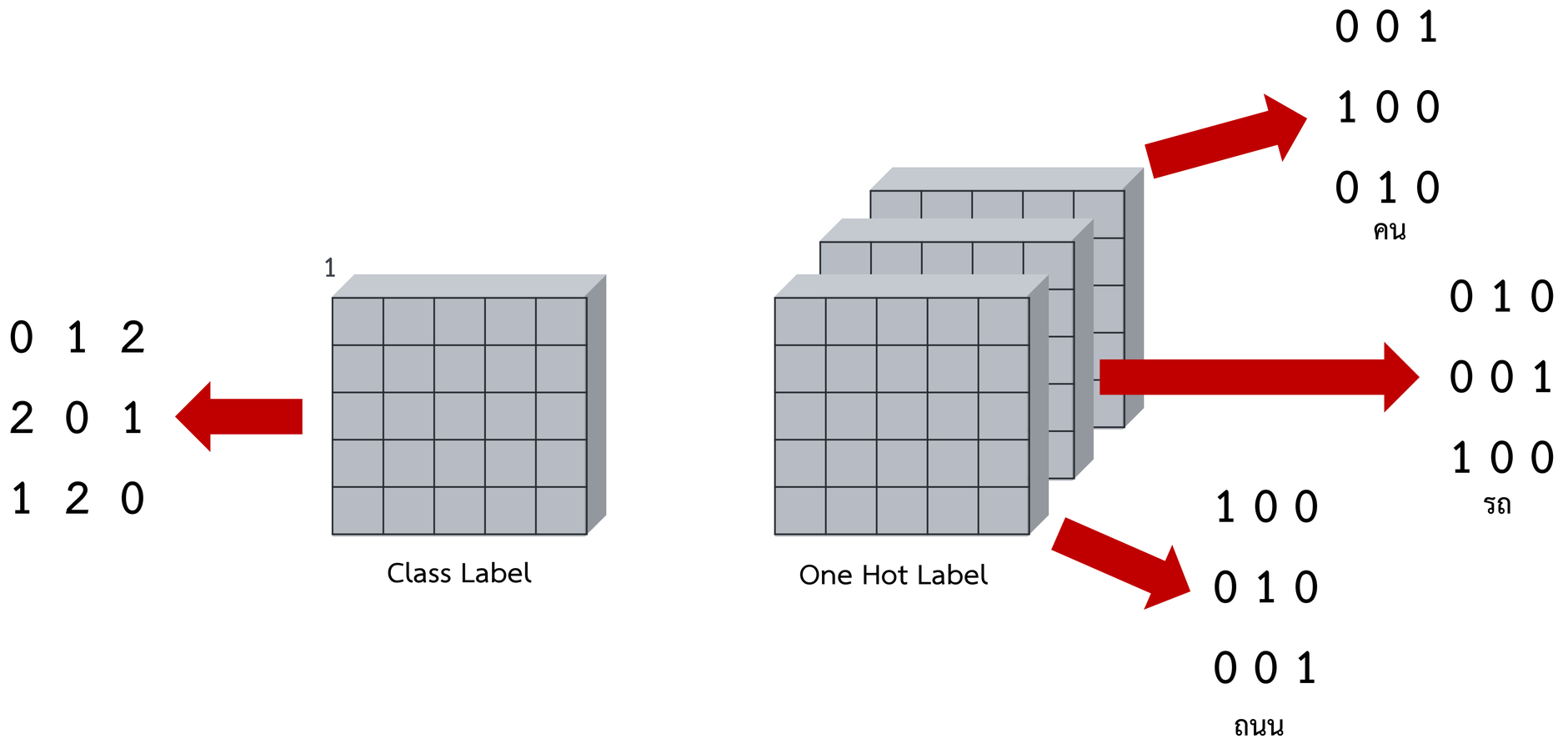
รูปแบบของ Labels ในงาน Autoencoder

สมมติว่ามี N classes และ Label แบบ Class Labels คือ

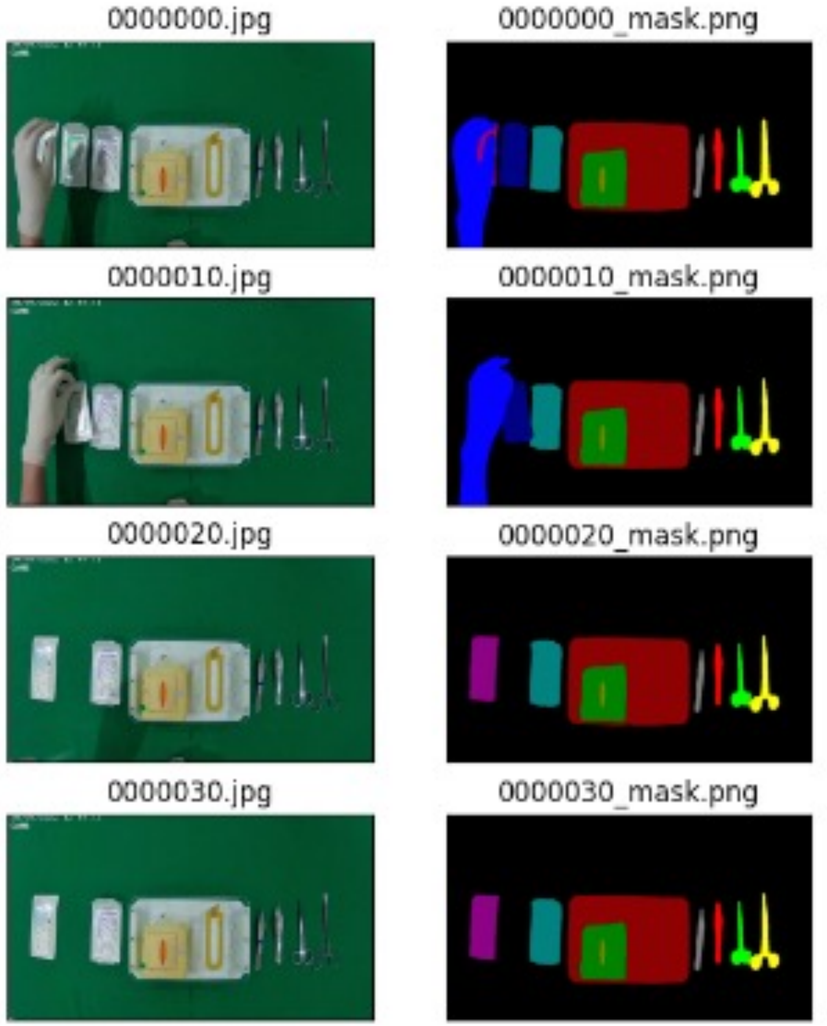


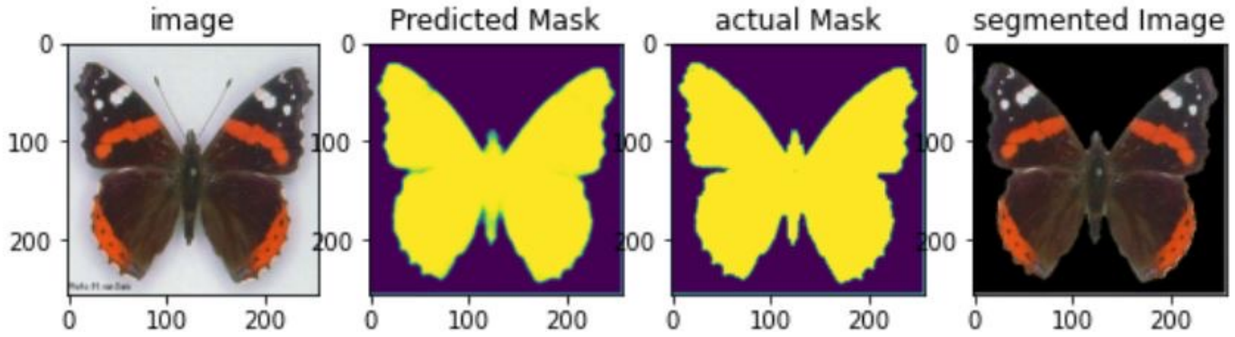
รูปแบบของ Labels ในงาน Autoencoder

สมมติว่ามี 3 classes (ถนน = 0, รถ = 1, คน = 2) และ Label แบบ Class Labels คือ



Inputs, Labels และ Outputs



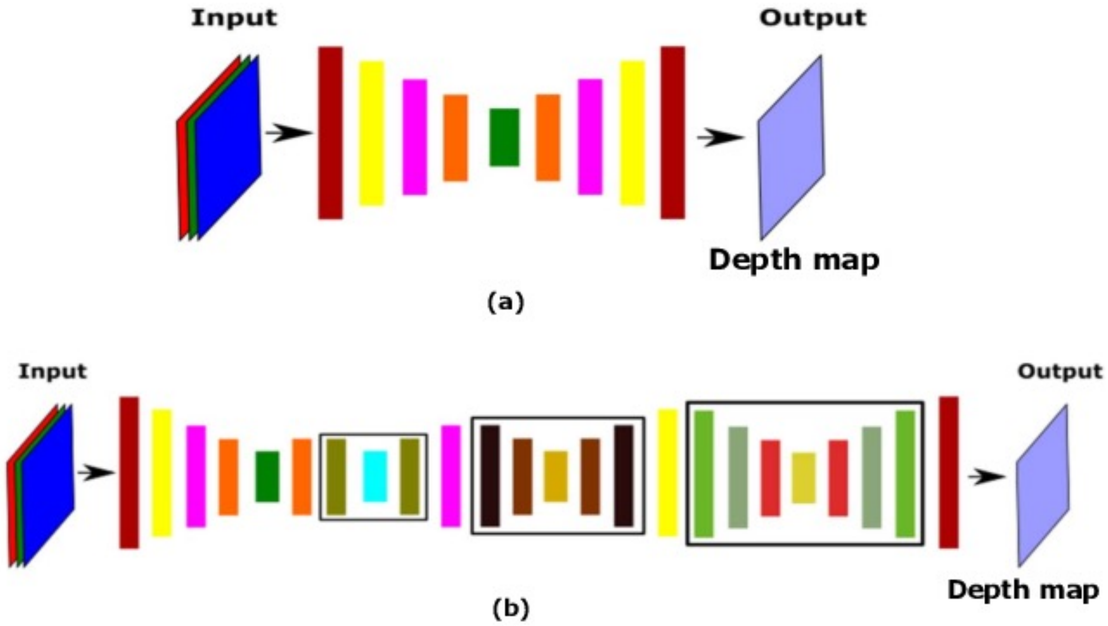


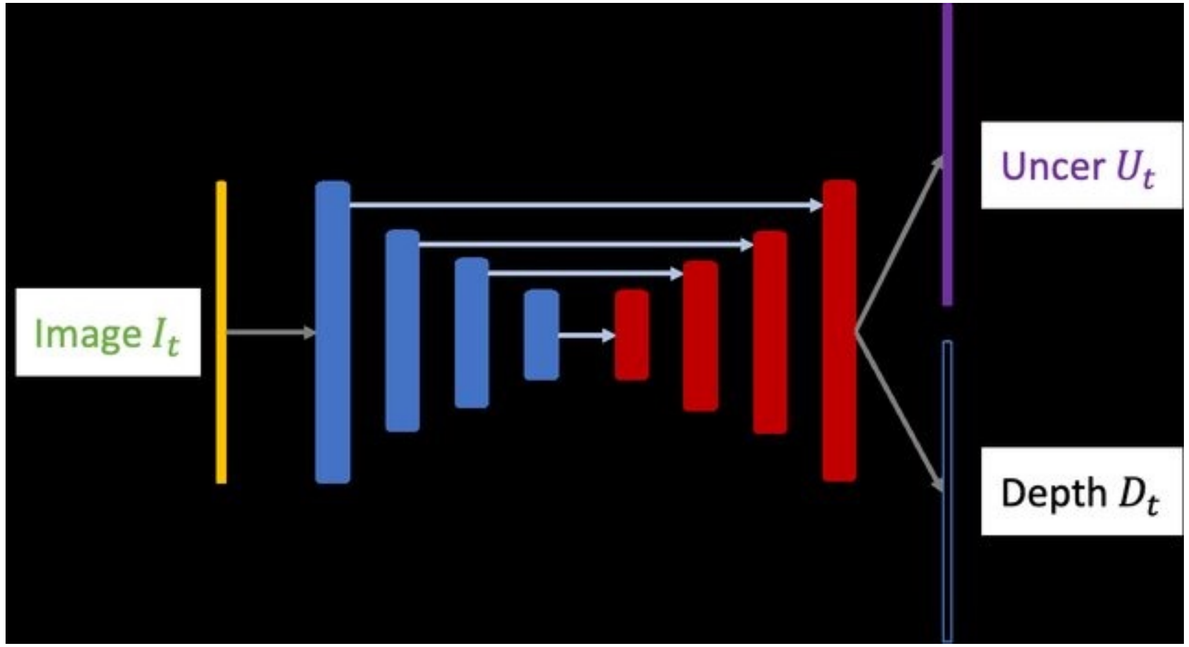
https://www.analyticsvidhya.com/blog/2021/01/auto-encoders-for-computer-vision-an-endless-world-of-possibilities/



https://diode-dataset.org/

สถาปัตยกรรม U-Net





Yusiong, John Paul Tan and Prospero C. Naval. “AsiANet: Autoencoders in Autoencoder for Unsupervised Monocular Depth Estimation.” *2019 IEEE Winter Conference on Applications of Computer Vision (WACV)* (2019): 443-451.

https://www.researchgate.net/publication/360462577_CNN-Augmented_Visual-Inertial_SLAM_with_Planar_Constraints

การประยุกต์ใช้งาน CNN สำหรับ Image

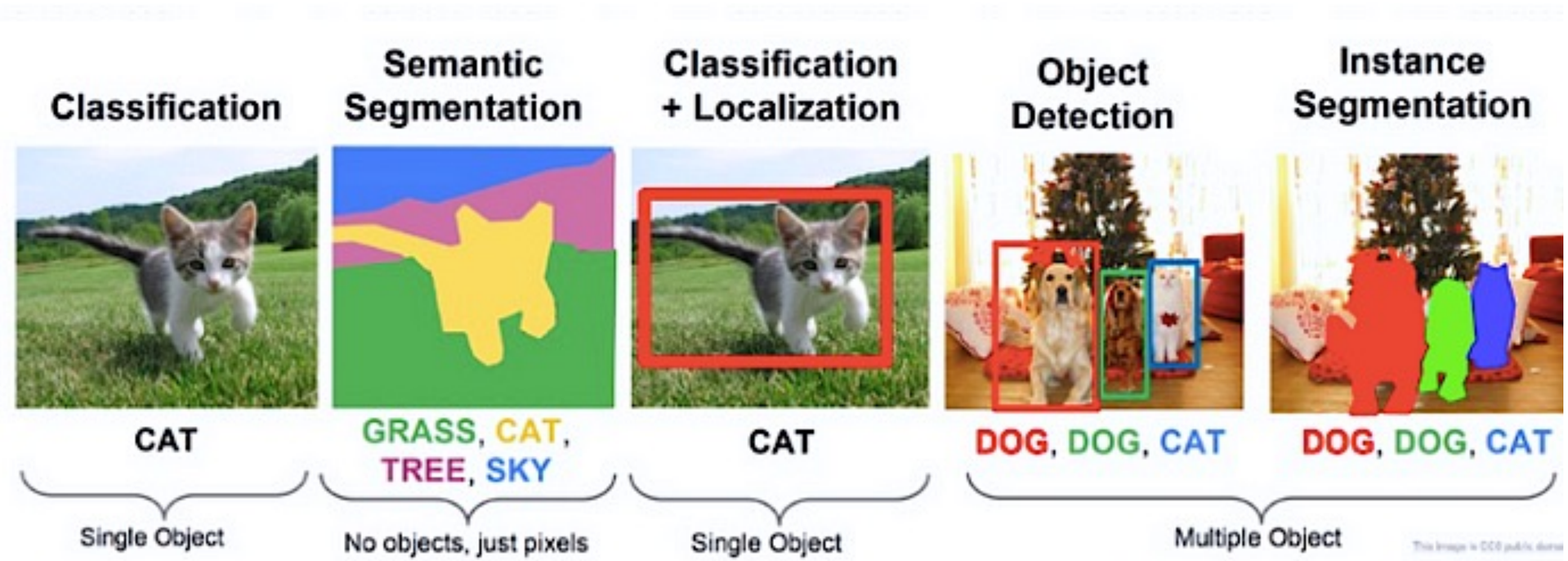
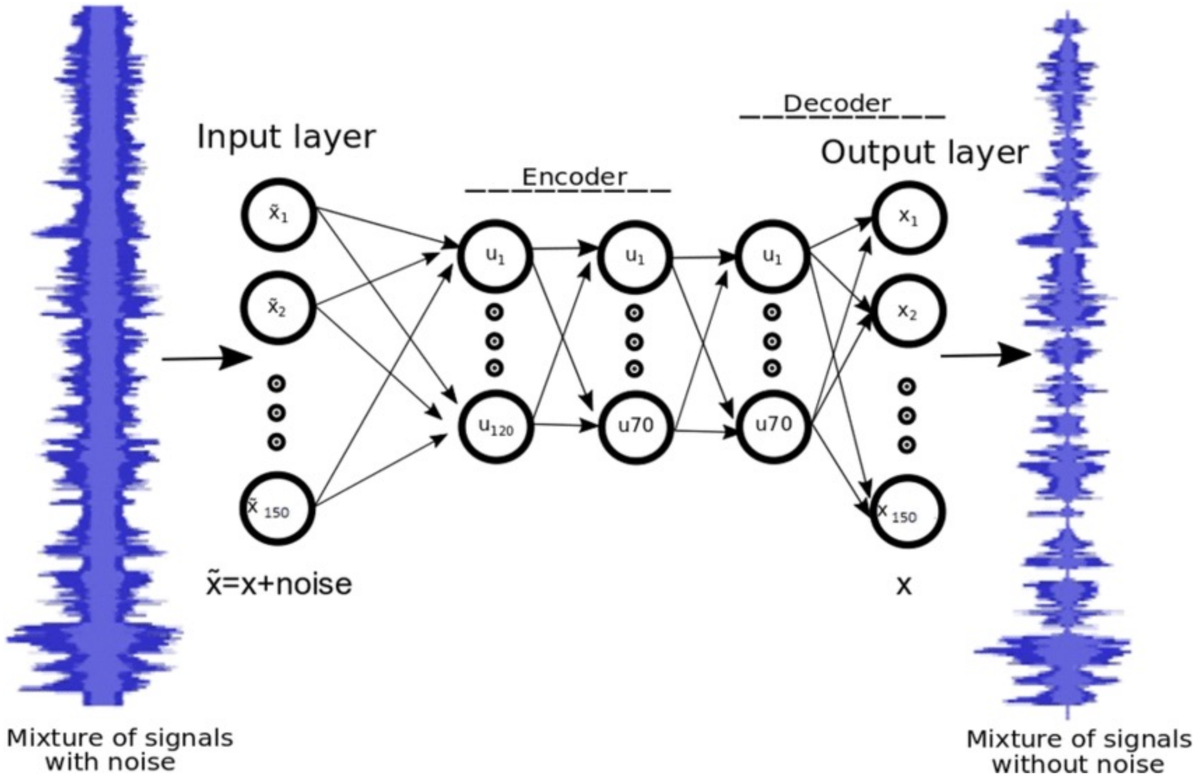


Figure 1: Different computer vision tasks. Source: Introduction to Artificial Intelligence and Computer Vision Revolution (https://www.slideshare.net/darian_f/introduction-to-the-artificial-intelligence-and-computer-vision-revolution).

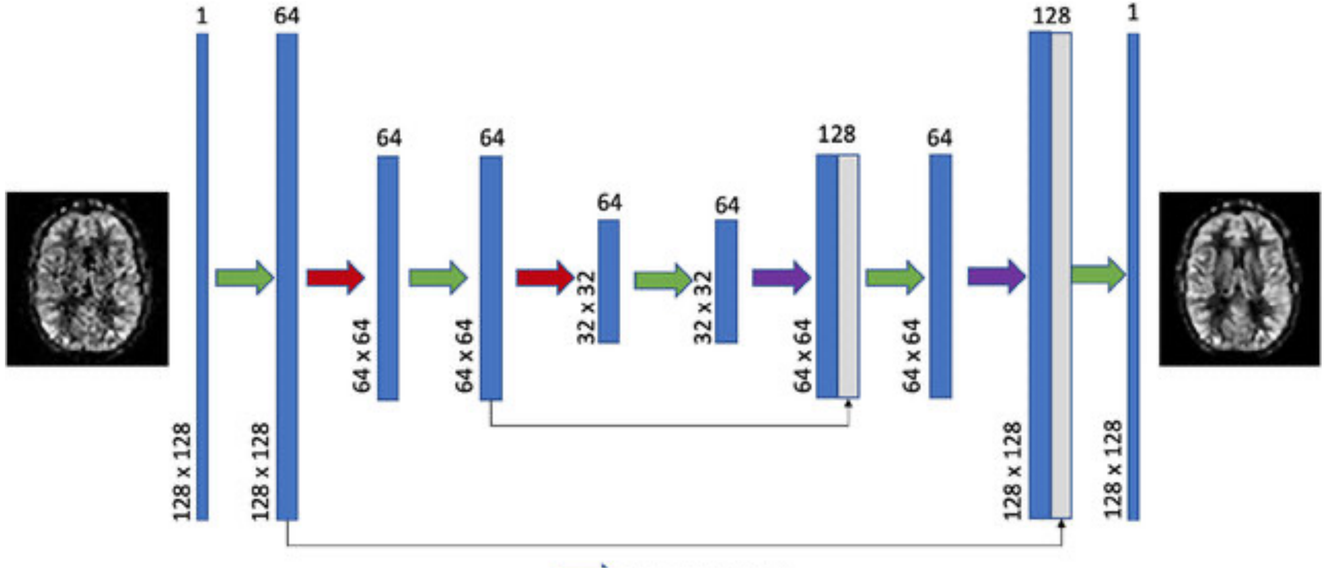
Denoising Autoencoder

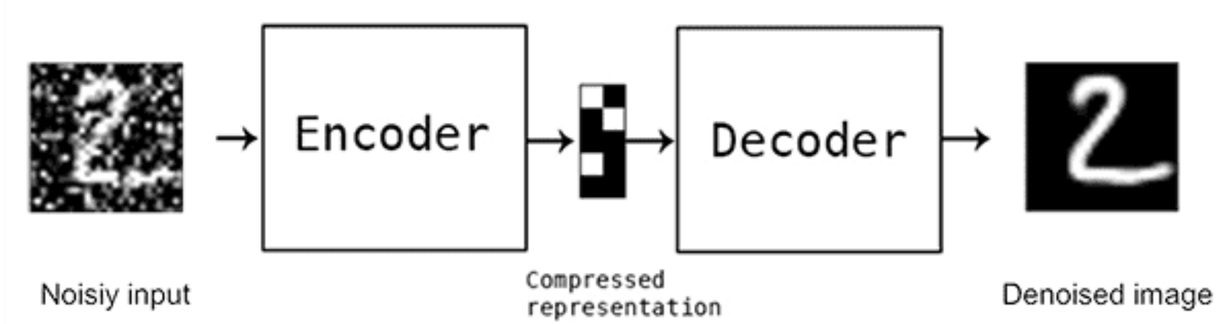


สถาปัตยกรรม Autoencoder ย่อยรูปแบบหนึ่งที่ออกแบบมาสำหรับขจัดสิ่งรบกวนออกจากภาพ ได้รับการคิดค้นโดย Vincent และคณะในปี ค.ศ. 2008 โดยฝึกสอนแบบจำลองด้วย Input ที่มีลักษณะเป็น Corrupted Data และ Output เป็น Clean Data ซึ่ง Denoising Autoencoder จะสกัด Code ที่มีความทนทานต่อสิ่งรบกวน หรือสามารถกล่าวได้ว่าจะสกัด Code มาจากส่วน Data เป็นหลักและอาจจะมีส่วนของ noise ปะปนมาด้วยเล็กน้อย ซึ่งสถาปัตยกรรมมีโครงสร้างคล้าย Vanilla Autoencoder แต่จะฝึกสอนด้วยปกติและมีสัญญาณรบกวนปะปนด้วย

อ้างอิง 10.1007/s10470-019-01446-6.

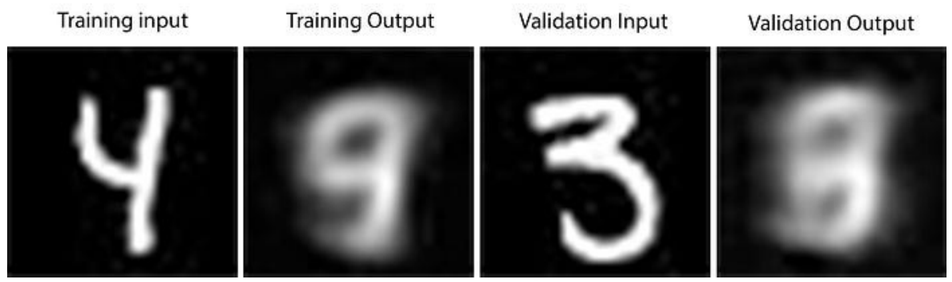
Denoising Autoencoder



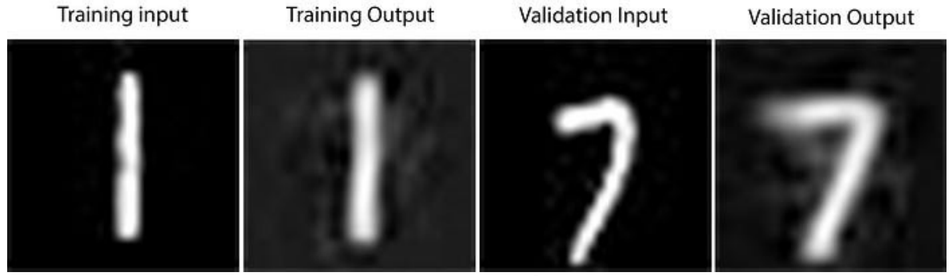


อ้างอิง (1) https://pyimagesearch.com/2020/02/24/denoising-autoencoders-with-keras-tensorflow-and-deep-learning/ และ (2) DOI: 10.1002/jmri.27255

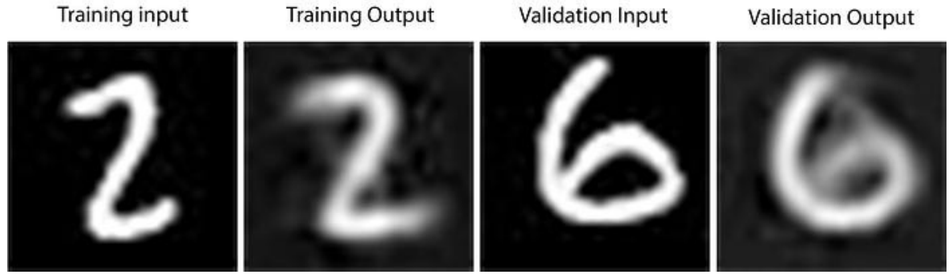
ตัวอย่าง Image Reconstruction



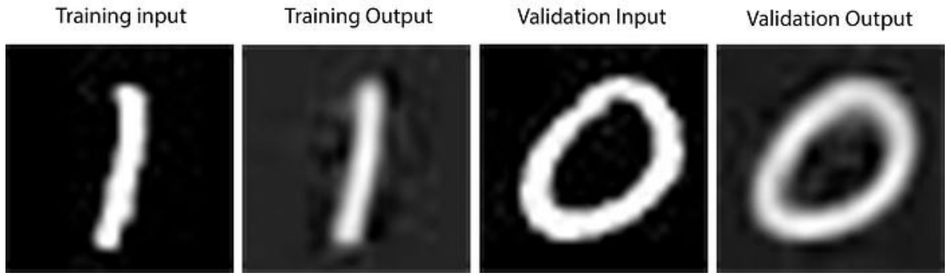
Epoch ที่ 1



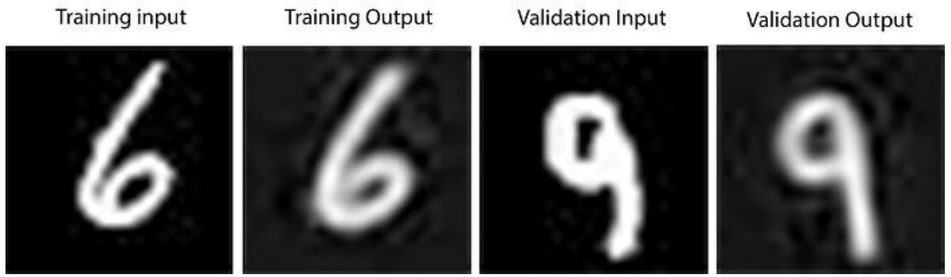
Epoch ที่ 5



Epoch ที่ 10



Epoch ที่ 25



Epoch ที่ 50

ภาพที่ได้จากแบบจำลอง Autoencode สำหรับงานด้าน Image Reconstruction ของแต่ละ Epoch

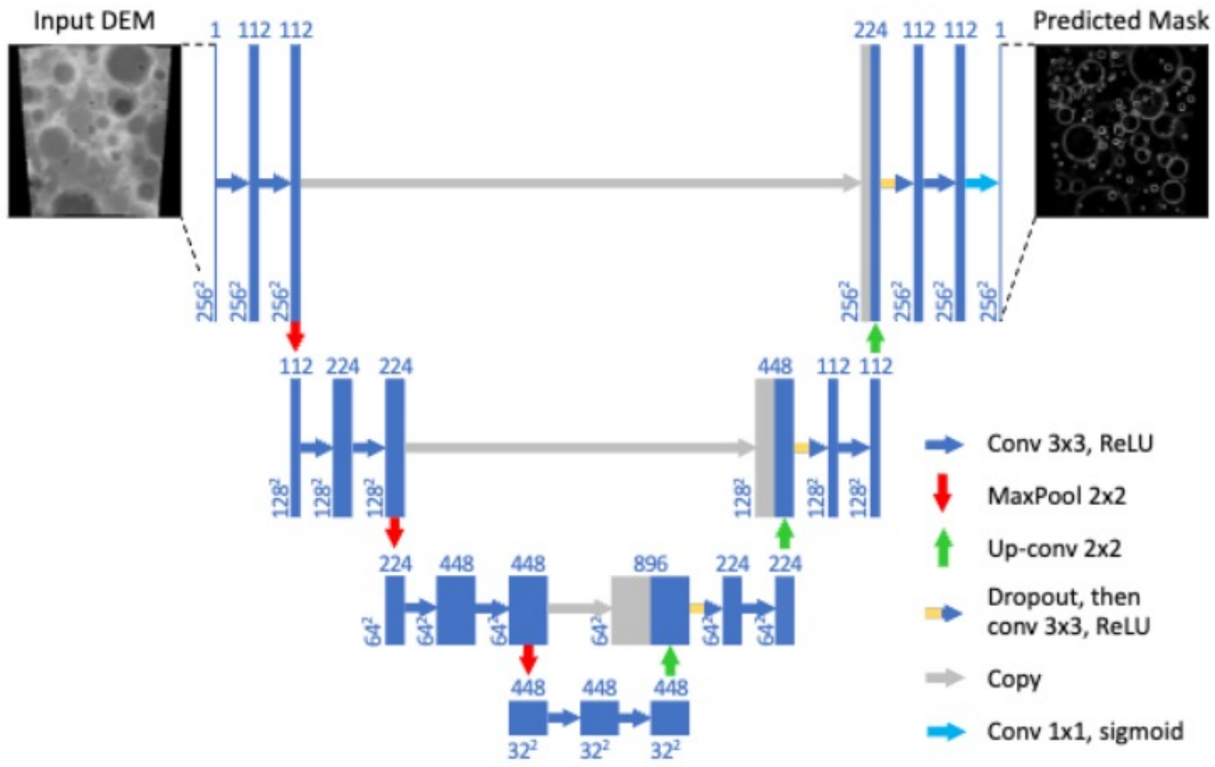
อ้างอิง https://medium.com/@dopplerz/autoencoder-โมเดลที่เรียนรู้ที่จะเป็นตัวเอง-d2edeb7381a2



หัวข้อ

- แนวคิดสถาปัตยกรรม Autoencoder
- **สถาปัตยกรรม Autoencoder ที่น่าสนใจ**
- วิธีการใช้งานชุดคำสั่ง Segmentation Models

สถาปัตยกรรม U-net



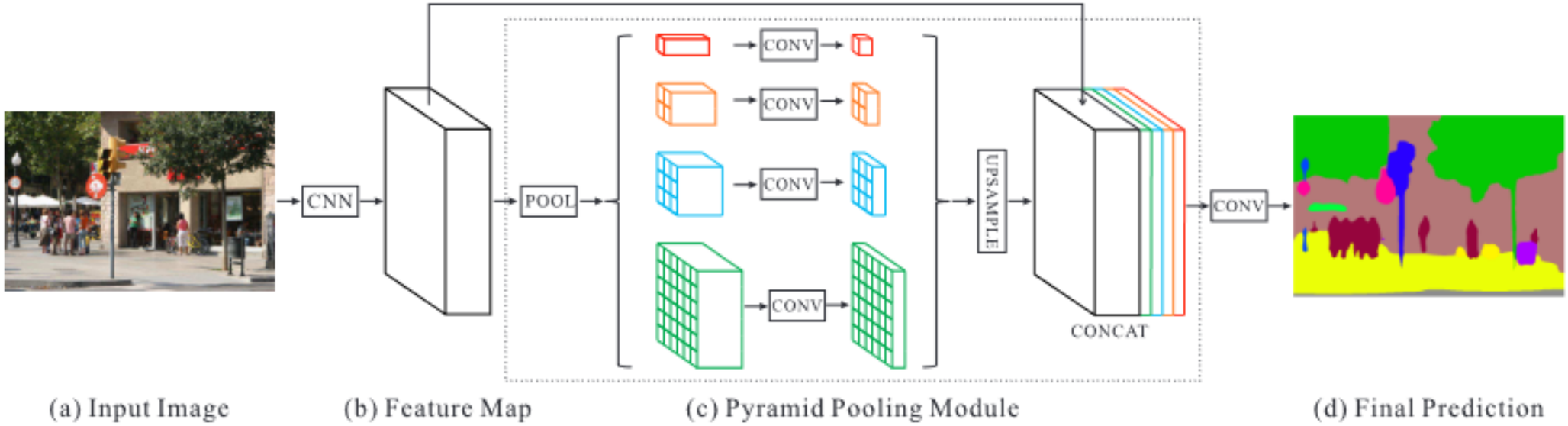
U-Net ได้รับการพัฒนาขึ้นโดย Olaf Ronneberger และคณะในปี 2015 เพื่อการแบ่งส่วนภาพ (Image Segmentation) โดยเฉพาะในการประมวลผลทางการแพทย์ (Medical Image Processing) โดยมีโครงสร้างแบบ Encoder-Decoder ซึ่งสามารถดึงลักษณะเชิงพื้นที่และรวมข้อมูลเชิงพื้นที่ที่มีความละเอียดสูงได้อย่างมีประสิทธิภาพ ซึ่งประกอบไปด้วย 3 องค์ประกอบหลัก ได้แก่

- Encoder (Contraction Path) ทำหน้าที่ดึงลักษณะเชิงพื้นที่โดยใช้ Convolutional Layers และ Max Pooling Layers เพื่อลดขนาดของฟีเจอร์แมปและเพิ่มความลึกของข้อมูล ช่วยให้สร้างแผนที่ลักษณะ (feature map) ที่มีความละเอียดสูงจากภาพอินพุต
- Decoder (Expansion Path) ทำหน้าที่ถอดรหัสลักษณะเชิงพื้นที่และเพิ่มขนาดของฟีเจอร์แมปกลับไปสู่ขนาดเดิมโดยใช้ Transposed Convolutional Layers ช่วยให้รวมข้อมูลจากหลายระดับความลึกและสร้างผลลัพธ์ที่มีความละเอียดสูง
- Skip Connections ใช้การเชื่อมต่อข้ามเลเยอร์ระหว่าง Encoder และ Decoder เพื่อรวมข้อมูลเชิงพื้นที่ที่มีความละเอียดสูงจาก Encoder กับข้อมูลที่ถอดรหัสใน Decoder ทำให้ช่วยลดการสูญเสียข้อมูลและเพิ่มความแม่นยำในการแบ่งส่วนภาพ

การรวมข้อมูลจากหลายระดับความลึกด้วยการใช้ Skip Connections ทำให้ U-Net สามารถรวมข้อมูลเชิงพื้นที่จากหลายระดับความลึกเข้าด้วยกัน ซึ่งเพิ่มความแม่นยำในการแบ่งส่วนภาพ อีกทั้งโครงสร้างแบบ Encoder-Decoder ยังช่วยให้สามารถดึงลักษณะเชิงพื้นที่ที่สำคัญจากภาพและถอดรหัสลักษณะเชิงพื้นที่เหล่านี้เพื่อสร้างผลลัพธ์ที่มีความละเอียดสูง

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. arXiv preprint arXiv:1505.04597.

สถาปัตยกรรม PSPNet



สถาปัตยกรรม Pyramid Scene Parsing Network (PSPNet) สร้างโดย Hengshuang Zhao, Jianping Shi, Xiaojuan Qi, Xiaogang Wang, Jiaya Jia ในปี 2017 เพื่อการแบ่งส่วนภาพ (Image Segmentation) PSPNet ได้รับการออกแบบมาเพื่อเพิ่มประสิทธิภาพในการแบ่งส่วนภาพโดยใช้เทคนิคการรวมลักษณะเชิงพื้นที่ (spatial features) จากหลายสเกล (multi-scale) โดยอาศัย Pyramid Pooling Module โมดูลนี้ช่วยในการจับลักษณะเฉพาะที่แตกต่างกันจากหลายสเกล โดยการแบ่งแผนที่ลักษณะ (feature map) ออกเป็นส่วนย่อย (subregions) เช่น 1x1, 2x2, 3x3 เป็นต้น จากนั้น ทำการคอนโวลูชัน (conv 1x1) ในแต่ละส่วนย่อยเพื่อลดมิติของข้อมูล และรวมผลลัพธ์จากแต่ละส่วนย่อยเข้าด้วยกัน ซึ่งการใช้ Pyramid Pooling Module ทำให้ PSPNet สามารถจับลักษณะเชิงพื้นที่ที่มีความละเอียดสูงจากหลายสเกล ทำให้ช่วยเพิ่มความแม่นยำในการแบ่งส่วนภาพ นอกจากนี้ PSPNet ยังพบจุดเด่นอีกมากมาย อาทิเช่น

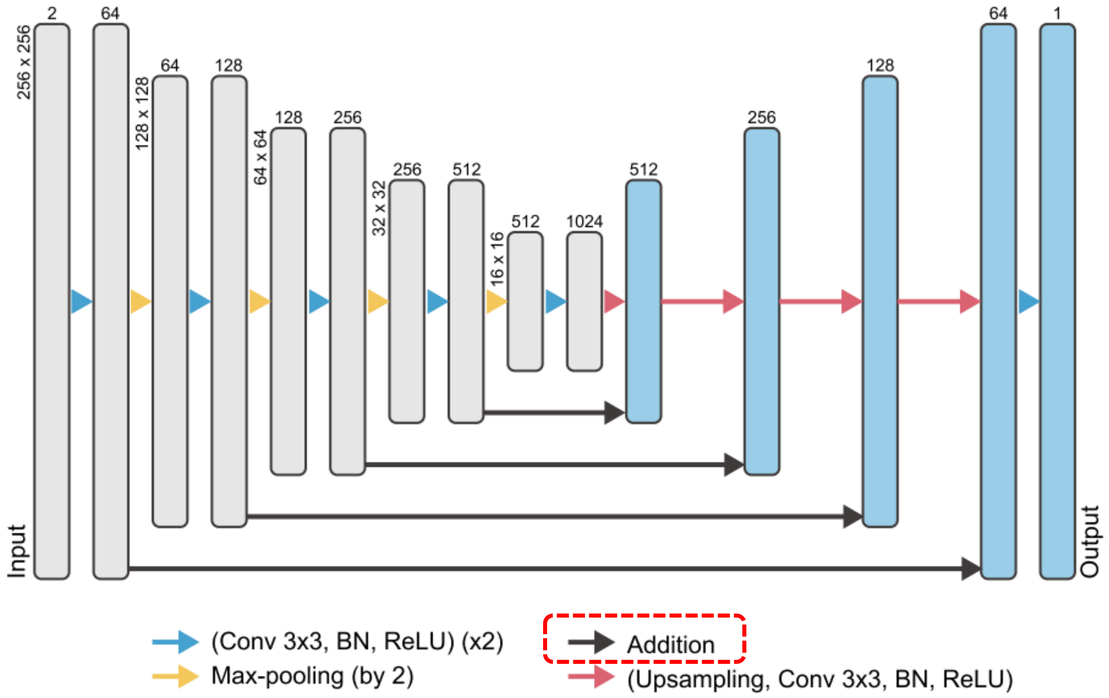
- Multi-Scale Feature Extraction: ความสามารถในการดึงลักษณะเชิงพื้นที่จากหลายสเกล ช่วยให้สามารถจัดการกับวัตถุที่มีขนาดและรูปร่างที่แตกต่างกันในภาพเดียวได้อย่างมีประสิทธิภาพ
- Pyramid Pooling Module: การใช้โมดูลนี้ช่วยให้สามารถรวมข้อมูลจากหลายสเกลเข้าด้วยกัน ซึ่งเพิ่มความแม่นยำในการแบ่งส่วนภาพ
- การใช้ Dice Loss: ช่วยให้การฝึกสอนโมเดลมีประสิทธิภาพมากขึ้น โดยเฉพาะในกรณีที่ข้อมูลมีการกระจายที่ไม่สมดุล

Zhao, H., Shi, J., Qi, X., Wang, X., & Jia, J. (2017). Pyramid Scene Parsing Network. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) 2017, 2881-2890.

สถาปัตยกรรม Linknet

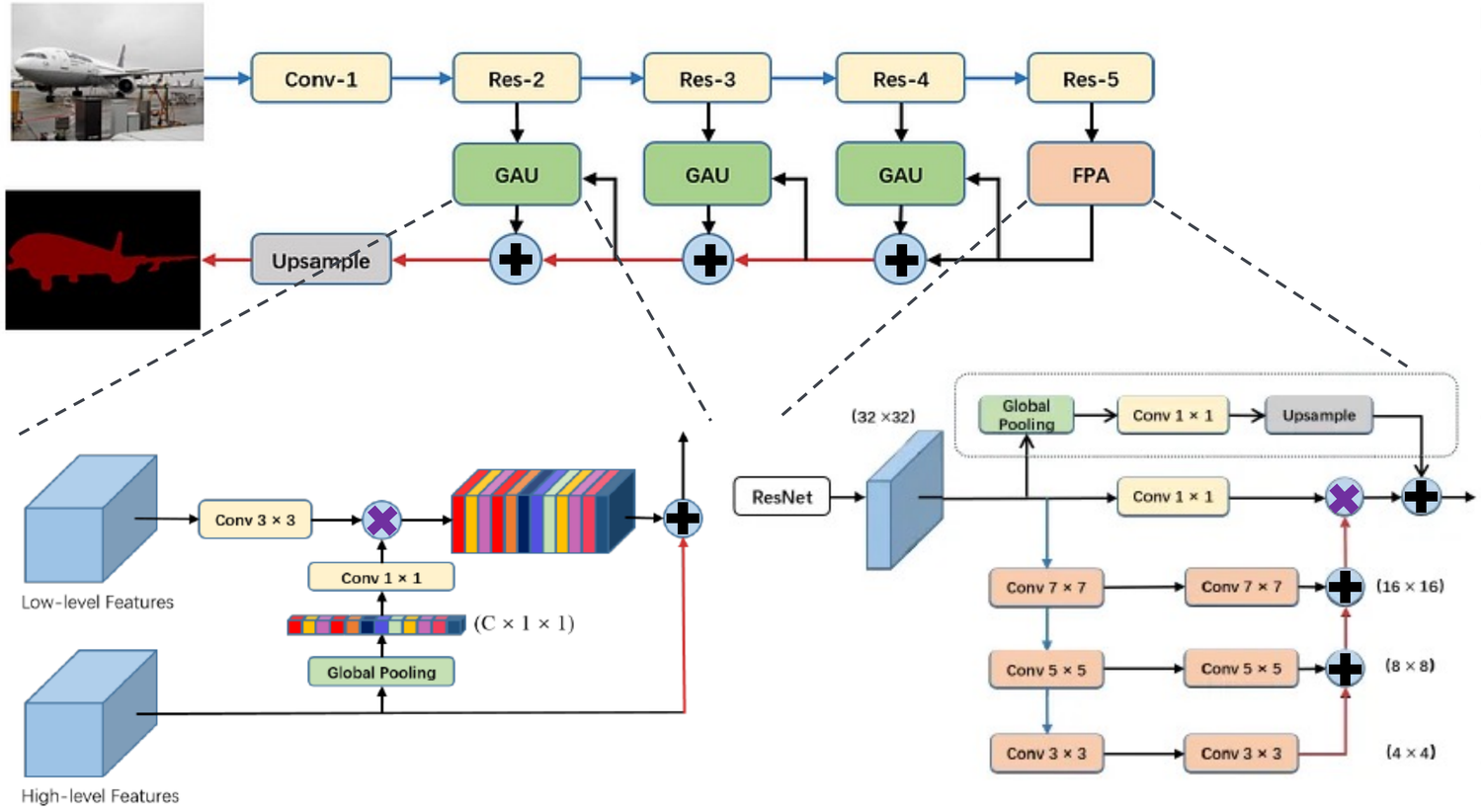
Linknet ถูกสร้างโดย Abhishek Chaurasia และคณะ ในปี 2017 เพื่อใช้ในงานการแบ่งส่วนเชิงความหมาย (semantic segmentation) Linknet ถูกออกแบบมาเพื่อให้มีประสิทธิภาพสูงในการประมวลผล และสามารถทำงานได้รวดเร็ว อีกทั้ง ยังใช้ทรัพยากรน้อยเนื่องจากการออกแบบที่เน้นความเรียบง่าย ทำให้โมเดลใช้ทรัพยากรในการประมวลผลน้อยลง ซึ่งเหมาะสมกับการใช้งานในระบบที่มีข้อจำกัดเรื่องทรัพยากร นอกจากนี้ ยังมีการนำหลักการเชื่อมต่อข้ามเลเยอร์ (Skip Connection) เข้ามาช่วยลดการสูญเสียข้อมูลและเพิ่มความแม่นยำในการทำนายผลลัพธ์ Linknet ได้รับการพัฒนามาจาก 5 ส่วน ได้แก่

- **Initial Block** ทำหน้าที่ในการเตรียมข้อมูลอินพุตให้พร้อมสำหรับการประมวลผลในส่วนถัดไป รวมถึงการปรับขนาดและการปรับค่าพิกเซลของภาพ
- **Encoder Block** ทำหน้าที่ดึงลักษณะเฉพาะจากภาพ (Feature Extraction) โดยใช้ Convolutional Layers คล้ายกับส่วน Encoder ในโมเดลการเรียนรู้เชิงลึกอื่น อาทิเช่น U-Net
- **Decoder Block** ทำหน้าที่ถอดรหัสลักษณะเฉพาะ (Decode Feature) กลับไปเป็นภาพ โดยอาศัย Transposed Convolutions หรือ Deconvolutions เพื่อเพิ่มขนาดของแผนที่ลักษณะ (Feature Map) กลับไปสู่ขนาดเดิม
- **Final Block** ทำหน้าที่ทำนายผลลัพธ์สุดท้ายของการแบ่งส่วนภาพ (Predict Output) นิยมใช้การคอนโวลูชันขนาด 1x1 เพื่อทำการแปลงแผนที่ลักษณะเป็นแผนที่ผลลัพธ์
- Skip Connection ช่วยส่งข้อมูลบางส่วนจากเลเยอร์ Encoder ไปยังเลเยอร์ Decoder เพื่อช่วยลดการสูญเสียข้อมูลและเพิ่มประสิทธิภาพของการทำนายผลลัพธ์



- Yokoya, Naoto et al. (2020). Breaking Limits of Remote Sensing by Deep Learning From Simulated Data for Flood and Debris-Flow Mapping. IEEE Transactions on Geoscience and Remote Sensing. PP. 1-15..
- Chaurasia, A., & Culurciello, E. (2017). LinkNet: Exploiting Encoder Representations for Efficient Semantic Segmentation. arXiv preprint arXiv:1707.03718.

สถาปัตยกรรม PAN



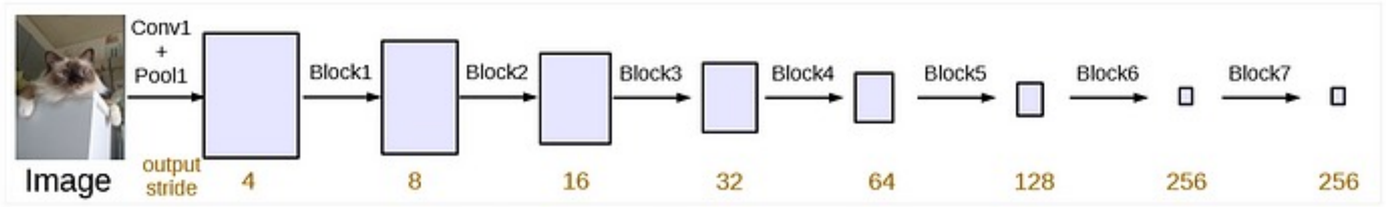
เส้นสีน้ำเงินแสดงถึงการ downsample ซึ่งเป็นการลดขนาดของ Feature map ขณะที่ เส้นสีแดงแสดงถึงการ upsample ซึ่งเป็นการเพิ่มขนาดของ Feature map สุดท้าย PAN จะทำ Batch Normalization หลังจากแต่ละเลเยอร์คอนโวลูชัน เป็นกระบวนการที่ช่วยปรับปรุงการฝึกโมเดลให้มีเสถียรภาพและเร็วขึ้นโดยการทำให้ค่าฟีเจอร์ต่าง ๆ มีการกระจายที่เหมาะสมในแต่ละแบตช์

Pyramid Attention Network (PAN) ได้รับการพัฒนาขึ้นโดย Hanchao Li, Pengfei Xiong, Jie An, Lingxue Wang ในปี 2018 PAN ถูกออกแบบมาเพื่อเพิ่มประสิทธิภาพในการแบ่งส่วนภาพโดยการรวมข้อมูลจากหลายสเกลและให้ความสำคัญกับบริบทที่มีความสำคัญของภาพ ซึ่งประกอบไปด้วย 3 องค์ประกอบหลัก ได้แก่

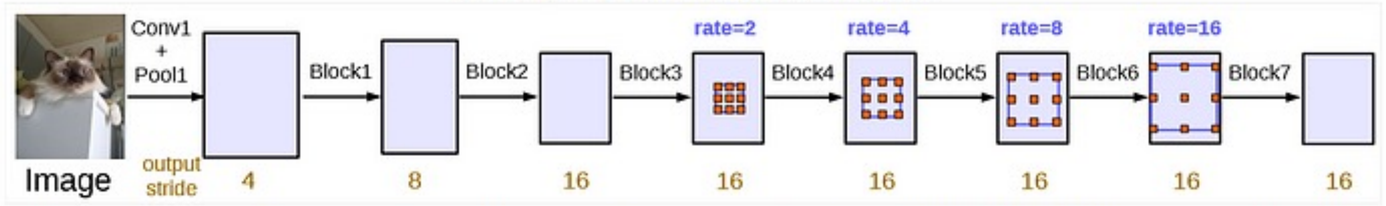
- **Encoder Block** ทำหน้าที่ดึงลักษณะเฉพาะจากภาพ (Feature Extraction) โดยใช้ Convolutional Layers คล้ายกับ Encoder ในโมเดลการเรียนรู้เชิงลึกอื่น อาทิเช่น U-net, ResNet เป็นต้น
- **Feature Pyramid Attention (FPA)** ทำหน้าที่รวมลักษณะเชิงพื้นที่ (Features) จากหลายระดับ (Multi-scale Features) โดยการแบ่ง Feature Map ออกเป็นส่วนย่อยต่างๆ และรวมข้อมูลจากหลายระดับเข้าด้วยกัน เพื่อให้ได้ลักษณะเชิงพื้นที่ที่ครอบคลุมและมีความละเอียดสูงขึ้น FPA ช่วยให้สามารถจับลักษณะเชิงพื้นที่จากหลายระดับ ทำให้สามารถจัดการกับวัตถุที่มีขนาดและรูปร่างที่แตกต่างกันในภาพเดียวได้อย่างมีประสิทธิภาพ
- **Global Attention Upsample (GAU)** ทำหน้าที่ให้ความสำคัญกับบริเวณหรือข้อมูลที่มีความสำคัญสูง (Global Context) และนำข้อมูลเหล่านี้ไปช่วยปรับน้ำหนักให้กับลักษณะเชิงพื้นที่ระดับต่ำ (Low-level Features) เพื่อช่วยเพิ่มความแม่นยำในการแบ่งส่วนภาพโดยการใช้บริบทที่สำคัญจากภาพทั้งหมด

Li, H., Xiong, P., An, J., & Wang, L. (2018). Pyramid Attention Network for Semantic Segmentation. arXiv preprint arXiv:1805.10180. และ https://sh-tsang.medium.com/review-pan-pyramid-attention-network-for-semantic-segmentation-semantic-segmentation-8d94101ba24a

สถาปัตยกรรม DeeplabV3

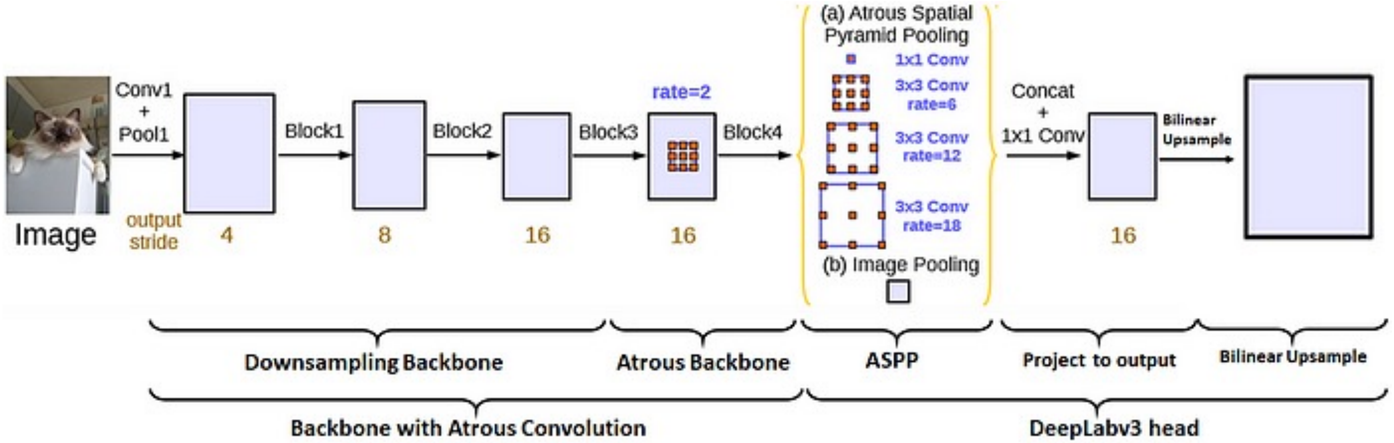


(a) Going deeper without atrous convolution.



(b) Going deeper with atrous convolution. Atrous convolution with $rate > 1$ is applied after block3 when $output_stride = 16$.

Figure 3. Cascaded modules without and with atrous convolution.



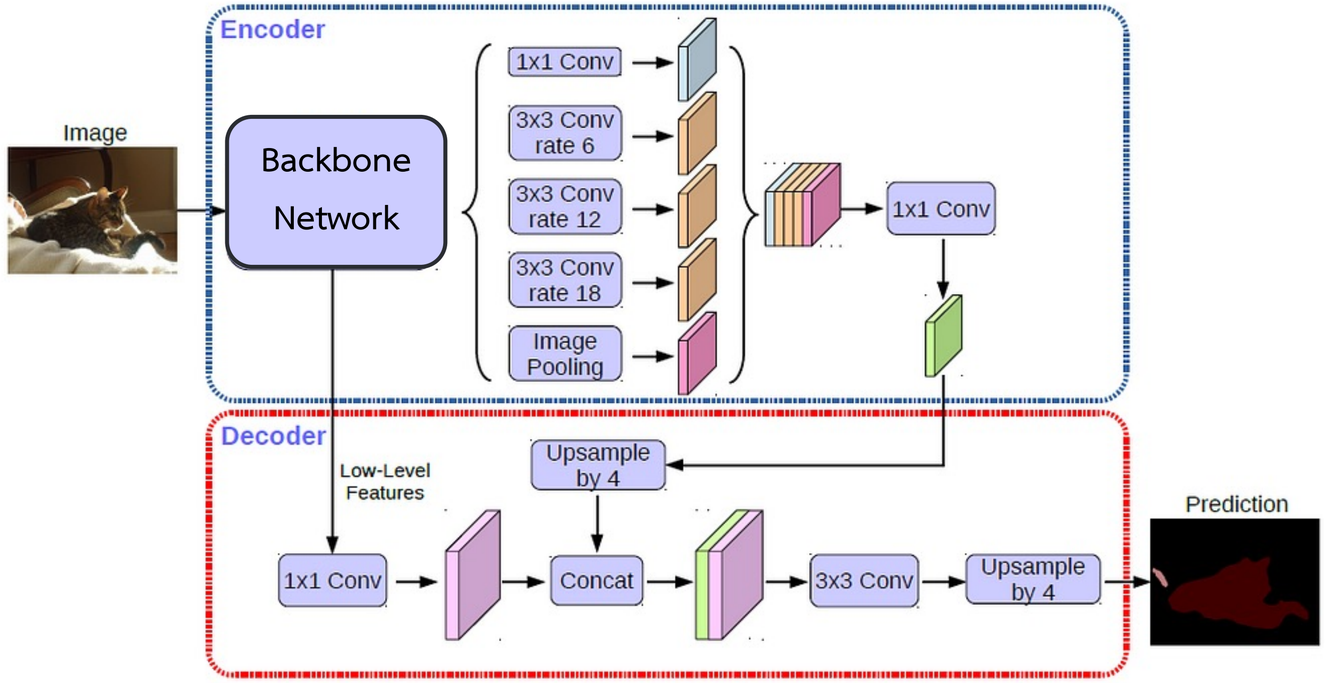
DeepLabV3 สร้างโดย Liang-Chieh Chen และคณะ ในปี 2017 เป็นโมเดลที่ใช้ในการแบ่งส่วนภาพ โดยมีการนำเทคนิคการคอนโวลูชันที่พัฒนาขึ้นเพื่อให้สามารถดึงลักษณะเชิงพื้นที่จากหลายระดับได้อย่างมีประสิทธิภาพ ซึ่งประกอบไปด้วย 2 องค์ประกอบหลัก ได้แก่

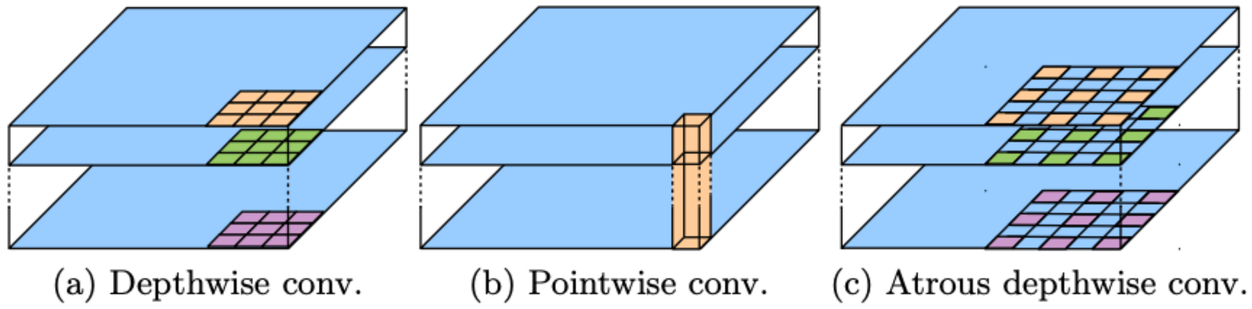
- **Atrous Convolution หรือ Dilated Convolution** เป็นเทคนิคที่ปรับความห่างระหว่างพิกเซลในการคอนโวลูชัน ทำให้สามารถรับลักษณะเชิงพื้นที่ที่กว้างขึ้นโดยไม่สูญเสียความละเอียดของภาพ ทำให้ช่วยเพิ่มฟิลด์รับรู้ (receptive field) ของฟิลเตอร์คอนโวลูชันโดยไม่ต้องเพิ่มจำนวนพารามิเตอร์ ช่วยให้สามารถจับลักษณะเชิงพื้นที่ได้ดีขึ้น
- **Atrous Spatial Pyramid Pooling (ASPP)** เป็นการใช้ Atrous Convolution หลายสเกลในการดึงลักษณะเชิงพื้นที่ (Multi-scale Feature Extraction) โดยการคอนโวลูชันที่มีขนาด Dilated ต่างๆ กัน ส่งผลให้สามารถรวมลักษณะเชิงพื้นที่จากหลายสเกลเข้าด้วยกัน ทำให้สามารถจัดการกับวัตถุที่มีขนาดและรูปร่างแตกต่างกันในภาพเดียวได้อย่างมีประสิทธิภาพ

ดังนั้น DeepLabV3 สามารถดึงลักษณะเชิงพื้นที่ที่กว้างขึ้นด้วยการใช้ Atrous Convolution ทำให้สามารถดึงลักษณะเชิงพื้นที่ที่กว้างขึ้นโดยไม่สูญเสียความละเอียดของภาพ อีกทั้ง ยังสามารถรวมลักษณะเชิงพื้นที่จากหลายสเกลด้วย ASPP ทำให้สามารถรวมลักษณะเชิงพื้นที่จากหลายสเกลเข้าด้วยกัน ซึ่งเพิ่มความแม่นยำในการแบ่งส่วนภาพ

Chen, L.-C., Papandreou, G., Schroff, F., & Adam, H. (2017). Rethinking Atrous Convolution for Semantic Image Segmentation. arXiv preprint arXiv:1706.05587.. และ https://medium.com/@itberrios6/deeplabv3-c0c8c93d25a4

สถาปัตยกรรม DeeplabV3+





(a) Depthwise conv. (b) Pointwise conv. (c) Atrous depthwise conv.

DeepLabV3+ พัฒนาขึ้นโดย Liang-Chieh Chen และคณะในปี 2018 ซึ่งเป็นการพัฒนาต่อยอดจากสถาปัตยกรรม DeepLabV3 โดยการเพิ่มโมดูล Decoder เพื่อพัฒนาโครงสร้างให้เป็นแบบ encoder-decoder ซึ่งช่วยให้การแบ่งส่วนภาพมีความละเอียดและแม่นยำมากขึ้น ซึ่งประกอบไปด้วย 3 องค์ประกอบหลัก ได้แก่

- **Atrous Convolution**
- **Atrous Spatial Pyramid Pooling (ASPP)**
- **Decoder Module** ทำหน้าที่ถอดรหัสลักษณะเชิงพื้นที่ที่ได้จาก Encoder ให้กลับไปมีขนาดเท่ากับ Input เพื่อทำนายผลการแบ่งส่วนภาพ สิ่งนี้ช่วยเพิ่มความละเอียดและความแม่นยำในการแบ่งส่วนภาพ ทำให้ผลลัพธ์สุดท้ายมีความละเอียดสูงและตรงกับขนาดของภาพต้นฉบับ

การที่ DeepLabV3+ มีโครงสร้างแบบ Encoder-Decoder ช่วยให้การประมวลผลลักษณะเชิงพื้นที่และการถอดรหัสลักษณะเชิงพื้นที่ทำได้อย่างมีประสิทธิภาพมากขึ้นด้วยการใช้โมดูล Decoder ทำให้ผลลัพธ์สุดท้ายมีความละเอียดสูงและตรงกับขนาดของภาพต้นฉบับ

Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., & Adam, H. (2018). Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation. arXiv preprint arXiv:1802.02611. และ https://sh-tsang.medium.com/review-deeplabv3-atrous-separable-convolution-semantic-segmentation-a625f6e83b90



หัวข้อ

- แนวคิดสถาปัตยกรรม Autoencoder
- สถาปัตยกรรม Autoencoder ที่น่าสนใจ
- **วิธีการใช้งานชุดคำสั่ง Segmentation Models**

ชุดคำสั่ง Segmentation Models คืออะไร



ชุดคำสั่ง Segmentation Models ถูกพัฒนาขึ้นด้วยภาษา Python โดยคุณ Pavel Iakubovskii สำหรับงานทางด้าน Image Segmentation ที่ประยุกต์ใช้ชุดคำสั่ง PyTorch โดยเป็นชุดคำสั่งสำเร็จรูป (High level API) ที่มีทั้งโมเดลที่ฝึกฝนแล้วให้เลือกใช้งาน ตัวชี้วัด (Metric) รวมถึง Loss ที่สามารถนำมาใช้งานได้



หัวข้อ

- แนวคิดสถาปัตยกรรม Autoencoder
- ชุดคำสั่ง Segmentation Models คืออะไร
- วิธีการใช้งาน Segmentation Models



วิธีการใช้งาน Segmentation Models

วิธีการใช้งาน Segmentation Models

- วิธีการติดตั้งจาก The Python Package Index (PyPI) ใช้ Package manager ของ Python

```
pip install segmentation-models-pytorch
```

- **วิธีการติดตั้งจาก Github โดยตรง**

  ในบางครั้งวิธีการนี้จะได้เวอร์ชันที่ใหม่กว่าติดตั้งจาก The Python Package Index (PyPI)

```
$ pip install git+https://github.com/qubvel/segmentation_models.pytorch
```

วิธีการใช้งาน Segmentation Models

สำหรับการสร้างโมเดล Image Segmentation สามารถประกอบไปด้วย 5 ขั้นตอนดังนี้

ข้อ	คำอธิบาย
1	กำหนด Dataset ที่จะนำมาใช้ในการฝึกสอนโมเดล
2	กำหนด Convolutional Neural Network สำหรับการฝึกสอนโมเดล
3	เลือก Loss Function และ Optimizer ที่เหมาะสมกับงานของเรา
4	การฝึกสอนโมเดล
5	การทดสอบและวัดผลโมเดล

อ้างอิง : https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

วิธีการใช้งาน Segmentation Models

```
from torch.utils.data import Dataset
from PIL import Image
import os

class CustomImageDataset(Dataset):
    def __init__(self, img_dir, mask_dir, transform=None):
        self.img_dir = img_dir
        self.mask_dir = mask_dir
        self.img_id = os.listdir(img_dir)
        self.transform = transform

    def __len__(self):
        return len(self.img_dir)

    def __getitem__(self, idx):
        img_path =  os.path.join(self.img_dir, self.img_id[idx])
        mask_path = os.path.join(self.mask_dir, self.img_id[idx])

        # Load image
        image = Image.open(img_path).convert('RGB')
        mask = Image.open(mask_path)

        # Convert to black and white
        thresh = 128
        fn = lambda x: 255 if x > thresh else 0
        mask = mask.convert('L').point(fn,mode='1')

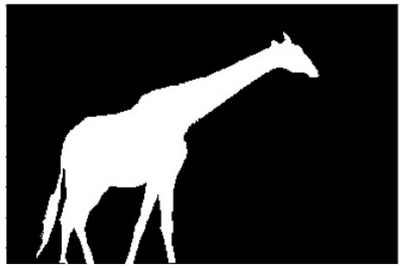
        if self.transform:
            image, mask = self.transform(image, mask)
        return image, mask
```

1. กำหนด Dataset ที่ใช้ในการฝึกสอนโมเดล

ในงาน Image Segmentation จะมีรายละเอียดที่แตกต่างจากวิธีการตรวจจับภาพอื่น ๆ คือ เฉลย (Label) ที่นิยมเรียกว่า Mask จะมีขนาดเท่ากับภาพที่นำมาฝึกสอน โดยลักษณะเฉลยจะประกอบด้วยค่า 0 หรือ 1 เพื่อบอกบริเวณวัตถุ



Base



Binary Mask

โดยภาพต้นฉบับและ Mask สามารถใช้ transform ตัวเดียวกันได้ ซึ่งคำสั่ง transform ต้องใช้ version 2 ขึ้นไป

อ้างอิงภาพ : https://raw.githubusercontent.com/dmlc/webdata/master/mxnet/doc/tutorials/data_aug/outputs/with_mask/masks.png

วิธีการใช้งาน Segmentation Models

```
import torchvision.transforms.v2 as transforms
import numpy as np
import torch

transform = transforms.Compose([
    transforms.ToImage(),
    transforms.ToDtype(torch.float32, scale=True),
    ])

train_img_dir = './RetinaBloodVessel/train/image’
train_mask_dir = './RetinaBloodVessel/train/mask’

val_img_dir = './RetinaBloodVessel/val/image’
val_mask_dir = './RetinaBloodVessel/val/mask'

train_dataset = CustomImageDataset(train_img_dir , train_mask_dir ,
transform=transform)
valid_dataset = CustomImageDataset(val_img_dir , val_mask_dir ,
transform=transform)

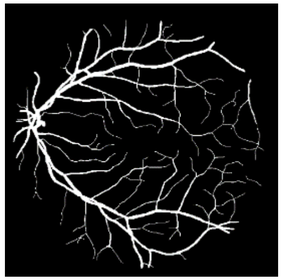
train_loader = torch.utils.data.DataLoader(train_dataset,
batch_size=8, shuffle=True, num_workers=0)
valid_loader = torch.utils.data.DataLoader(valid_dataset,
batch_size=8, shuffle=False, num_workers=0)
```

ตัวอย่างการนำมาใช้กับชุดข้อมูล Retina Blood Vessel จากเว็บไซต์ Kaggle

ตัวอย่างภาพ



ภาพ



Mask

ชุดข้อมูล : https://www.kaggle.com/datasets/abdallahwagih/retina-blood-vessel

วิธีการใช้งาน Segmentation Models

```
import matplotlib.pyplot as plt

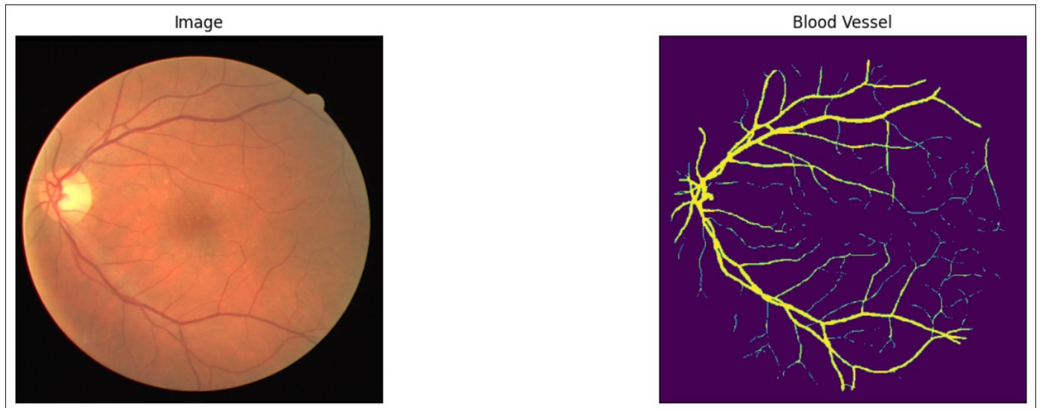
# helper function for data visualization
def visualize(**images):
    """PLot images in one row."""
    n = len(images)
    plt.figure(figsize=(16, 5))
    for i, (name, image) in enumerate(images.items()):
        plt.subplot(1, n, i + 1)
        plt.xticks([])
        plt.yticks([])
        plt.title(' '.join(name.split('_')).title())
        plt.imshow(image)
    plt.show()

image, mask = train_dataset[0] # get some sample
image, mask = image.numpy(), mask.numpy()
image, mask = image.transpose(1, 2, 0), mask.transpose(1, 2, 0)

visualize(
    image = image,
    blood_vessel = mask,
)
```

ตัวอย่างการนำภาพและ Mask ออกมาแสดงจากที่เรานำไปสร้าง (Build) เป็น Dataset สำหรับ Pytorch แล้ว

ตัวอย่างผลลัพธ์





วิธีการใช้งาน Segmentation Models

```
import torch
import numpy as np
import segmentation_models_pytorch as smp
from segmentation_models_pytorch import utils

ENCODER = 'resnet50'
ENCODER_WEIGHTS = 'imagenet'
CLASSES = ['blood_vessel']
ACTIVATION = 'sigmoid'
DEVICE = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# create segmentation model with pretrained encoder
model = smp.FPN(
    encoder_name=ENCODER,
    encoder_weights=ENCODER_WEIGHTS,
    classes=len(CLASSES),
    activation='sigmoid')
```

2. กำหนด Convolutional Neural Network สำหรับการฝึกสอนโมเดล

สำหรับการสร้างโมเดล สามารถกำหนดรายละเอียด โดยใส่เป็นพารามิเตอร์ลงในโมเดลที่ต้องการได้เลย

ตัวอย่าง ENCODER

ENCODER	WEIGHTS
resnet50	imagenet / ssl / swsl
densenet121	imagenet
inceptionresnetv2	imagenet / imagenet+background
efficientnet-b0	imagenet
mobilenet_v2	imagenet

สำหรับ ENCODER เพิ่มเติม:

https://github.com/qubvel/segmentation_models.pytorch



วิธีการใช้งาน Segmentation Models

```
import torch
import numpy as np
import segmentation_models_pytorch as smp
from segmentation_models_pytorch import utils

ENCODER = 'resnet50'
ENCODER_WEIGHTS = 'imagenet'
CLASSES = ['blood_vessel']
ACTIVATION = 'sigmoid’
# excute Model on CPU or GPU
DEVICE = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# create segmentation model with pretrained encoder
model = smp.FPN(
    encoder_name=ENCODER,
    encoder_weights=ENCODER_WEIGHTS,
    classes=len(CLASSES),
    activation='sigmoid’)

# or another option
aux_params=dict(
    pooling='avg',             # one of 'avg', 'max'
    dropout=0.5,               # dropout ratio, default is None
    activation='sigmoid',      # activation function, default is None
    classes= len(CLASSES)     # define number of output labels
)
model = smp.Unet(encoder_name=ENCODER, aux_params=aux_params)
```

2. กำหนดสถาปัตยกรรมที่จะเรียกใช้งาน

สำหรับชุดคำสั่ง Segmentation Models มีสถาปัตยกรรมที่ให้เรียกใช้งานได้หลากหลาย ได้แก่

- Unet
- Unet++
- MAnet
- FPN
- DeepLabV3
- DeepLabV3+

สำหรับ Model เพิ่มเติม: https://github.com/qubvel/segmentation_models.pytorch

วิธีการใช้งาน Segmentation Models

```
import torch
import numpy as np
import segmentation_models_pytorch as smp
from segmentation_models_pytorch import utils

ENCODER = 'resnet50'
ENCODER_WEIGHTS = 'imagenet'
CLASSES = ['blood_vessel']
ACTIVATION = 'sigmoid'
# excute Model on CPU or GPU
DEVICE = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# create segmentation model with pretrained encoder
model = smp.FPN(
    encoder_name=ENCODER,
    encoder_weights=ENCODER_WEIGHTS,
    classes=len(CLASSES),
    activation='sigmoid')

# or another option
aux_params=dict(
    pooling='avg',             # one of 'avg', 'max'
    dropout=0.5,               # dropout ratio, default is None
    activation='sigmoid',      # activation function, default is None
    classes= len(CLASSES)      # define number of output labels
)
model = smp.Unet(encoder_name=ENCODER, aux_params=aux_params)
```

```
class ClassificationHead(nn.Sequential):
    def __init__(
        self, in_channels, classes, pooling="avg", dropout=0.2, activation=None
    ):
        if pooling not in ("max", "avg"):
            raise ValueError(
                "Pooling should be one of ('max', 'avg'), got {}.".format(pooling)
            )
        pool = nn.AdaptiveAvgPool2d(1) if pooling == "avg" else nn.AdaptiveMaxPool2d(1)
        flatten = nn.Flatten()
        dropout = nn.Dropout(p=dropout, inplace=True) if dropout else nn.Identity()
        linear = nn.Linear(in_channels, classes, bias=True)
        activation = Activation(activation)
        super().__init__(pool, flatten, dropout, linear, activation)
```

คือการกำหนดวิธีแปลงข้อมูลให้กลายเป็นข้อมูล 1 มิติ ก่อนผ่านกระบวนการ flatten จากนั้น จึงส่งไปทำนายผลลัพธ์ด้วย fully connected ต่อไป

สำหรับ Model เพิ่มเติม: https://github.com/qubvel/segmentation_models.pytorch



วิธีการใช้งาน Segmentation Models

```
import torch
import numpy as np
import segmentation_models_pytorch as smp
from segmentation_models_pytorch import utils

ENCODER = 'resnet50'
ENCODER_WEIGHTS = 'imagenet'
CLASSES = ['XXX', 'yyy', 'zzz']  # Edit Here
ACTIVATION = 'softmax2d'         # Edit Here

# excute Model on CPU or GPU
DEVICE = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# create segmentation model with pretrained encoder
model = smp.FPN(
    encoder_name=ENCODER,
    encoder_weights=ENCODER_WEIGHTS,
    classes=len(CLASSES),
    activation='sigmoid')
```

กำหนดสถาปัตยกรรมสำหรับ Multiclass Classification

ให้นักศึกษากำหนด Activation Function เป็น softmax2d จากนั้น ค่อยกำหนดจำนวนคลาสและชื่อคลาสจากพารามิเตอร์ CLASSES ในรูปแบบ List

สำหรับ Model เพิ่มเติม: https://github.com/qubvel/segmentation_models.pytorch



วิธีการใช้งาน Segmentation Models

```
import torch
import numpy as np
import segmentation_models_pytorch as smp
from segmentation_models_pytorch import utils

ENCODER = 'resnet50'
ENCODER_WEIGHTS = 'imagenet'
CLASSES = ['XXX', 'yyy', 'zzz']  # Edit Here
ACTIVATION = Sigmoid'            # Edit Here

# excute Model on CPU or GPU
DEVICE = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# create segmentation model with pretrained encoder
model = smp.FPN(
    encoder_name=ENCODER,
    encoder_weights=ENCODER_WEIGHTS,
    classes=len(CLASSES),
    activation='sigmoid')
```

กำหนดสถาปัตยกรรมสำหรับ Multilabel Classification

ให้นักศึกษากำหนด Activation Function เป็น Sigmoid จากนั้น ค่อยกำหนดจำนวนคลาสและชื่อคลาสจากพารามิเตอร์ CLASSES ในรูปแบบ List

สำหรับ Model เพิ่มเติม: https://github.com/qubvel/segmentation_models.pytorch

วิธีการใช้งาน Segmentation Models

```
import torch
import numpy as np
import segmentation_models_pytorch as smp
from segmentation_models_pytorch import utils

ENCODER = 'resnet50'
ENCODER_WEIGHTS = 'imagenet'
CLASSES = ['XXX']                   # Edit Here
ACTIVATION = Sigmoid'               # Edit Here

# excute Model on CPU or GPU
DEVICE = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# create segmentation model with pretrained encoder
model = smp.FPN(
    encoder_name=ENCODER,
    encoder_weights=ENCODER_WEIGHTS,
    classes=len(CLASSES),
    activation='sigmoid')
```

กำหนดสถาปัตยกรรมสำหรับ Binary Classification

ให้นักศึกษากำหนด Activation Function เป็น Sigmoid จากนั้น ค่อยกำหนดจำนวนคลาสและชื่อคลาสจากพารามิเตอร์ CLASSES ในรูปแบบ List ซึ่งจะมีเพียง 1 คลาสเท่านั้น

สำหรับ Model เพิ่มเติม: https://github.com/qubvel/segmentation_models.pytorch

วิธีการใช้งาน Segmentation Models

```
import torch
import numpy as np
import segmentation_models_pytorch as smp
from segmentation_models_pytorch import utils

ENCODER = 'resnet50'
ENCODER_WEIGHTS = 'imagenet'
CLASSES = ['XXX', 'YYYY']          # Edit Here
ACTIVATION = Softmax'              # Edit Here

# excute Model on CPU or GPU
DEVICE = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# create segmentation model with pretrained encoder
model = smp.FPN(
    encoder_name=ENCODER,
    encoder_weights=ENCODER_WEIGHTS,
    classes=len(CLASSES),
    activation='sigmoid')
```

กำหนดสถาปัตยกรรมสำหรับ Binary Classification

ให้นักศึกษากำหนด Activation Function เป็น Sigmoid จากนั้น ค่อยกำหนดจำนวนคลาสและชื่อคลาสจากพารามิเตอร์ CLASSES ในรูปแบบ List ซึ่งจะมีเพียง 2 คลาสเท่านั้น

สำหรับ Model เพิ่มเติม: https://github.com/qubvel/segmentation_models.pytorch



วิธีการใช้งาน Segmentation Models

```
# Dice loss for image segmentation task. It supports binary,
multiclass, and multilabel cases.
loss = utils.losses.DiceLoss()

# Jaccard loss for image segmentation task. It supports binary,
multiclass, and multilabel cases.
# loss = utils.losses.JaccardLoss()


optimizer = torch.optim.Adam([
    dict(params=model.parameters(), lr=0.0001),
])
```

3. เลือก Loss Function และ Optimizer ที่เหมาะสม

สำหรับชุดคำสั่ง Segmentation Models มี Loss Function สำหรับการทำ Image Segmentation ให้เรียกใช้งานได้จากชุดคำสั่งโดยตรง ได้แก่ Jaccard Loss หรือ Dice loss ซึ่ง Loss Function ทั้ง 2 ตัว ได้รับการออกแบบมาสำหรับงานแบ่งส่วนภาพ (image segmentation) ที่รองรับการทำงานในกรณีแบบไบนารี (binary), มัลติคลาส (multiclass) และมัลติเลเบล (multilabel)

จากนั้น นักศึกษาค่อยกำหนด Optimizer และ Learning Rate ตามความเหมาะสมของงาน



วิธีการใช้งาน Segmentation Models

```
metrics = [
    utils.metrics.IoU(threshold=0.5),
]

# create epoch runners
# it is a simple loop of iterating over dataloader`s samples
# verbose คือการกำหนดการแสดงข้อมูลรายละเอียดระหว่างการทำงาน โดยที่ True คือแสดงและ False คือไม่แสดง

train_epoch = smp.utils.train.TrainEpoch(
    model,
    loss=loss,
    metrics=metrics,
    optimizer=optimizer,
    device=DEVICE,
    verbose=True,
)

valid_epoch = smp.utils.train.ValidEpoch(
    model,
    loss=loss,
    metrics=metrics,
    device=DEVICE,
    verbose=True,
)
```

4. การฝึกสอนโมเดล

ในการฝึกสอนโมเดลของชุดข้อมูล Segmentation Models จะใช้ฟังก์ชันสำเร็จรูปของชุดคำสั่ง โดยรับอาร์กิวเมนต์เป็นรายละเอียดของการฝึกสอน และการ Validation พร้อมกับ Metric ที่ต้องใช้สำหรับการวัดผลระหว่างการ Validation ได้แก่

- F1 Score
- IoU Score
- Accuracy
- Precision
- Recall

อ้างอิง https://github.com/qubvel/segmentation_models.pytorch/blob/master/segmentation_models_pytorch/utils/metrics.py

วิธีการใช้งาน Segmentation Models

```
metrics = [
    utils.metrics.IoU(threshold=0.5),
]

# create epoch runners
# it is a simple loop of iterating over dataloader`s samples
# verbose คือการกำหนดการแสดงข้อมูลรายละเอียดระหว่างการทำงาน โดยที่ True คือแสดงและ False คือไม่แสดง

train_epoch = smp.utils.train.TrainEpoch(
    model,
    loss=loss,
    metrics=metrics,
    optimizer=optimizer,
    device=DEVICE,
    verbose=True,
)

valid_epoch = smp.utils.train.ValidEpoch(
    model,
    loss=loss,
    metrics=metrics,
    device=DEVICE,
    verbose=True,
)
```

```
Epoch: 0
valid: 100%|██████████| 2/2 [00:00<00:00,  3.50it/s, dice_loss - 0.7835, iou_score - 0.122]

Epoch: 1
valid: 100%|██████████| 2/2 [00:00<00:00,  3.67it/s, dice_loss - 0.7753, iou_score - 0.1265]

Epoch: 2
valid: 100%|██████████| 2/2 [00:00<00:00,  3.64it/s, dice_loss - 0.8025, iou_score - 0.1076]

Epoch: 3
valid: 100%|██████████| 2/2 [00:00<00:00,  3.65it/s, dice_loss - 0.757, iou_score - 0.1328]
```

หมายเหตุ ถ้ากำหนดให้ verbose มีค่าเป็น True โปรแกรมจะแสดงข้อมูลรายละเอียดระหว่างการฝึกสอนแบบจำลอง แต่ถ้า verbose มีค่าเป็น False โปรแกรมจะไม่แสดงข้อมูลรายละเอียดระหว่างการทำงาน

อ้างอิง https://github.com/qubvel/segmentation_models.pytorch/blob/master/segmentation_models_pytorch/utils/metrics.py



วิธีการใช้งาน Segmentation Models

```
# train model for 40 epochs
max_score = 0
n_epoch = 40

for i in range(n_epoch):

    print('\nEpoch: {}'.format(i))
    train_logs = train_epoch.run(train_loader)
    valid_logs = valid_epoch.run(valid_loader)

    # do something (save model, change lr, etc.)
    # if max_score < valid_logs['iou_score']:
    #     max_score = valid_logs['iou_score']
    #     torch.save(model, './best_model.pth')
    #     print('Model saved!')

    # reduce learning rate from 0.0001 to 0.00001
    if i == 25:
        optimizer.param_groups[0]['lr'] = 1e-5
        print('Decrease decoder learning rate to 1e-5!’)

torch.save(model, './best_model.pth')
print('Model saved!')
```

หลังจากนั้น นำโมเดลไปฝึกสอนด้วยชุดคำสั่งทางด้านซ้ายมือ เมื่อการฝึกสอนเสร็จสิ้นตามจำนวนรอบที่กำหนด (epoch) โปรแกรมจะบันทึกโมเดลโดยอัตโนมัติ นอกจากนี้นักศึกษายังสามารถกำหนดให้โปรแกรมบันทึกเฉพาะโมเดลที่มีประสิทธิภาพดีที่สุดได้ โดยการเอาเครื่องหมายความคิดเห็น (uncomment) ออกจากส่วนโปรแกรมที่เป็นสีเขียว และโปรแกรมจะลดค่า learning rate โดยอัตโนมัติเมื่อการฝึกสอนเกินกว่า 25 รอบ



วิธีการใช้งาน Segmentation Models

```
transform = transforms.Compose([
    transforms.ToImage(),
    transforms.ToDtype(torch.float32, scale=True),
    ])

DEVICE = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
best_model = torch.load('./best_model.pth')

img_dir = './RetinaBloodVessel/test/image'
mask_dir = './RetinaBloodVessel/test/mask'

test_dataset = CustomImageDataset(img_dir, mask_dir, transform=transform)
test_dataloader = torch.utils.data.DataLoader(test_dataset, batch_size=1,
shuffle=False, num_workers=0)

loss = utils.losses.DiceLoss()
metrics = [
    utils.metrics.IoU(threshold=0.5),
]

# evaluate model on test set
test_epoch = smp.utils.train.ValidEpoch(
    model=best_model,
    loss=loss,
    metrics=metrics,
    device=DEVICE
)

logs = test_epoch.run(test_dataloader)
```

5. การทดสอบและวัดผลโมเดล

ในการการทดสอบและวัดผลโมเดล สามารถใช้ฟังก์ชันสำเร็จรูปจากชุดคำสั่งเช่นเดียวกับขั้นตอนก่อนหน้า โดยรับอาร์กิวเมนต์เป็นรายละเอียดของการทดสอบ และการ Metric ต่าง ๆ และ Dataloader ที่สร้างมาจากชุดข้อมูลทดสอบ



วิธีการใช้งาน Segmentation Models

ตัวอย่างโปรแกรมเรียกใช้งานแบบจำลอง

```
import torch

# load best saved checkpoint
best_model = torch.load('./best_model.pth’)

transform = transforms.Compose([
    transforms.ToImage(),
    transforms.ToDtype(torch.float32, scale=True)
    ])

img_path = './RetinaBloodVessel/train/image/0.png'
image = Image.open(img_path).convert('RGB')
image = transform(image)
image = image.unsqueeze(0)

predict_mask = best_model.predict(image)
image = image.squeeze().numpy()
predict_mask = predict_mask.squeeze(0).numpy()
predict_mask  = predict_mask.transpose(1, 2, 0)

# Then save predict_mask or visualize
xxx
```